Unterlagen für das obligatorische Fach Informatik

Gymnasium – Einführung in die Programmierung mit Python

Jacques Mock Schindler

19. Oktober 2026

Inhaltsverzeichnis

Welcome **7**
Schedule . . . 7
Hardware and Software Requirements . . . 7
Assessment . . . 8

I Anleitungen **9**

1 Speicherorganisation **10**
1.1 Dateinamen und Pfade . . . 10
1.2 Dateien in der Cloud . . . 11
1.3 Dateistruktur für die Schule . . . 11

2 Arbeitsumgebung (Arbeiten mit Jupyter Notebooks) **12**
2.1 Ausgangslage . . . 12
2.2 Installation von Python . . . 12
2.3 Hello World . . . 13
2.4 Arbeitsumgebung . . . 14
2.5 Öffnen bestehender Jupyter Notebooks . . . 18
2.6 Häufige Fehlermeldungen . . . 19

II Einführung in Python **21**

3 Problemlösung in der Informatik **22**
3.1 Ausgangslage . . . 22
3.2 Beispiel: Tricolore . . . 23
3.3 Beispiel: Österreichische Flagge . . . 24
3.4 Beispiel: Schweizerfahne . . . 25
3.5 Beispiel: Tessiner Wappen . . . 25
3.6 Musterlösungen . . . 25

4 Variabeln in Python **28**
4.1 Vorbemerkungen: Python als Rechner . . . 28
4.2 Variablen . . . 28
4.3 Funktionen in Python . . . 29
4.4 Datentypen . . . 30
4.5 Funktionen mit Type-Hints . . . 34
4.6 Ausgewählte Musterlösungen . . . 34

5 Wiederholungen in Python (For-Loops) 35
5.1 Anwendungsübung zu Wiederholungen in Python . . . 35

6 Programmverzweigungen (Bedingungen) 44
6.1 Anwendungsübung zu Bedingungen in Python . . . 46

7 Programmieren einfacher Funktionen 49
7.1 Kreisfläche . . . 49
7.2 Quotient . . . 49
7.3 Fakultät . . . 49
7.4 Quadratische Gleichungen (Mitternachtsformel) . . . 50
7.5 Musterlösungen . . . 50

8 Prüfungsvorbereitung 54
8.1 Belegung von Variabeln . . . 54
8.2 Funktionen . . . 54
8.3 Musterlösungen . . . 57

9 Datenstrukturen: Listen 59
9.1 Daten in Listen speichern . . . 59
9.2 Index . . . 59
9.3 Überschreiben von Elementen . . . 60
9.4 Schleifen . . . 60
9.5 Länge . . . 61
9.6 Ergänzen und entfernen . . . 61
9.7 Aufgabe: Fibonacci . . . 62
9.8 Aufgabe: zeichne Chomp . . . 62
9.9 Aufgabe: spiele Chomp . . . 63
9.10 Übersicht . . . 64
9.11 Zusatzstoff . . . 64

10 Datenstrukturen: Dictionary 65
10.1 Operationen für Dictionaries . . . 65

11 Beurteilung von Algorithmen 68
11.1 Bubblesort . . . 68
11.2 Mergesort . . . 68

12 Divide and Conquer 70
12.1 Fibonacci-Folge . . . 70
12.2 Endlose Rekursion . . . 71
12.3 Beispiel: Anstossen . . . 71
12.4 Fakultät . . . 74
12.5 Wörter umdrehen . . . 75
12.6 Bonus: Anzahl Möglichkeiten zu Zahlen . . . 75

13 Prüfungsvorbereitung 2 **77**
13.1 Lernziele . . . 77
13.2 Listen . . . 77
13.3 Musterlösungen . . . 79

III Data Representation **82**

14 Number Representation **83**
14.1 One-to-one Number Representation . . . 83
14.2 Symbolic Number Representation . . . 83
14.3 Number Representation and Positional Notation . . . 84
14.4 Number Conversion . . . 85

15 Image Representation in Computers **92**
15.1 Colour Representation . . . 92
15.2 Colour Table . . . 94

16 The Concept of an Algorithm **95**
16.1 Definition . . . 95
16.2 Properties of an Algorithm . . . 95
16.3 Example: Tying Shoe Laces . . . 95
16.4 From Everyday Algorithms to Mathematical Algorithms . . . 96
16.5 The Euclidean Algorithm . . . 96
16.6 Implementing the Euclidean Algorithm in Python . . . 97
16.7 Ways to Describe an Algorithm . . . 98

17 Algorithm Representation: Pseudocode & Struktogramme **99**
17.1 What Is Pseudocode? . . . 99
17.2 Pseudocode Conventions . . . 99
17.3 Worked Example: GCD in Pseudocode . . . 100
17.4 Exercise 1: Python → Pseudocode . . . 101
17.5 Exercise 2: Pseudocode → Python . . . 101
17.6 Exercise 3: Pseudocode → Python . . . 102
17.7 Structure Diagrams (Struktogramme) . . . 103
17.8 Summary: Comparing Notations . . . 105

18 Sorting Algorithms **106**
18.1 The Sorting Problem . . . 106
18.2 Pseudocode Conventions (Quick Reference) . . . 106
18.3 Helper: Visualising Each Step . . . 107
18.4 Your Sorting Algorithm . . . 107
18.5 Named Sorting Algorithms . . . 108
18.6 Selection Sort . . . 109
18.7 Bubble Sort . . . 110
18.8 Testing Your Implementations . . . 112
18.9 Counting Comparisons . . . 113

19 Algorithms and Computational Complexity **115**
19.1 Selection Sort . . . 115
19.2 Big O Notation . . . 115
19.3 Merge Sort: A More Efficient Approach . . . 116
19.4 Comparison of Growth Rates . . . 116

IV Functions **117**

20 Functions: Revision and Consolidation **118**
20.1 Function: A Definition . . . 118
20.2 Practical Example: The VAT Calculator . . . 119
20.3 Coding Exercise: The BMI Calculator . . . 120
20.4 Coding Exercise: The Cuboid Space Diagonal . . . 123

21 Functions II: Calculating Triangles **125**
21.1 Functions within functions . . . 125

22 Classes: Triangle as Object **132**
22.1 Introduction . . . 132
22.2 Enhanced Triangle Class . . . 134
22.3 Coding Task: Circle Class . . . 143

23 Functions: Exercises **147**
23.1 Area of a trapezium . . . 147
23.2 Volume of a right circular cylinder . . . 147
23.3 Progression decisions . . . 148

24 Triangulation: Inheriting from Triangle **150**
24.1 The Surveying Problem . . . 151
24.2 Stage 1 — Helper Functions . . . 151
24.3 Stage 2 — Bundling into a Class . . . 155
24.4 Stage 3 — Inheritance: an Auxiliary Triangle *is a* Triangle . . . 157
24.5 Coding Task . . . 161

25 Interest-Bearing Accounts: Inheriting from Account **162**
25.1 The Design at a Glance . . . 162
25.2 The Design at a Glance . . . 162
25.3 Stage 1 — A Class for a Single Entry . . . 164
25.4 Stage 2 — Bundling into an Account . . . 166
25.5 Stage 3 — Inheritance: a Current Account *is an* Account . . . 167

V Cyber Securtity **173**

26 Caesar Chiffre **174**
26.1 Python Implementation . . . 174

26.2 Caesar Chiffre brechen 176

27 File Handling in Python **179**

28 Caesar-Chiffre knacken **181**
28.1 Brute Force 181
28.2 Statistische Analyse 183

29 Die Vigenère-Chiffre **188**
29.1 Ein wenig Geschichte 188
29.2 Die Idee: Ein Schlüsselwort statt einer Zahl 188
29.3 Verschlüsseln Schritt für Schritt 189
29.4 Das Vigenère-Quadrat 190
29.5 Die Rechnung als Formel 192
29.6 Entschlüsseln 193
29.7 Warum die Häufigkeitsanalyse versagt 193
29.8 Die Schwächen der Vigenère-Chiffre 206

Literaturverzeichnis **211**

Welcome

Here you will find information about computer science lessons.

Schedule

Date	Topic
21.08.2026	Introduction and Revision
28.08.2026	National Surveying I
04.09.2026	National Surveying II
11.09.2026	Cyber Risks
18.09.2026	Cyber Self-Defence I
25.09.2026	Cyber Self-Defence II
23.10.2026	Computer Networks I: Layer Models
30.10.2026	Exam 1: National Surveying and Cyber Security
06.11.2026	Computer Networks II: IP and DNS
13.11.2026	Computer Networks III: NAT
20.11.2026	Computer Networks IV: Top-Level Domains
27.11.2026	Computer Networks V: Monitoring Connections
04.12.2026	Python Sockets I
11.12.2026	Python Sockets II
18.12.2026	Data Structures I: Stack and Queue
08.01.2027	Data Structures II: Queues in Python
15.01.2027	Data Structures III: Binary Search Tree
22.01.2027	Exam 2: Computer Networks and Data Structures
29.01.2027	Databases: Introduction
05.02.2027	Basic SQL Queries and End of Semester

The programme reflects the current state of planning. Changes are to be expected throughout the semester.

Hardware and Software Requirements

A laptop is required for the computer science lessons (you will not be able to solve the tasks set in class using an iPad). You will need administrator rights on your laptop to install the required software.

Furthermore, you must ensure that your battery is sufficiently charged at the start of the lesson to last for a double period.

Assessment

Two written tests are planned per semester. In addition, oral participation will be graded. **Asking questions** is specifically considered as part of oral participation.

The final grade is calculated as a weighted average of the two written tests and the grade for oral participation. The average grade of the two written tests is weighted at 90%, while the grade for oral participation is weighted at 10%.

If anyone would like a personal consultation, you can sign up for a meeting here (Rent a Mock).

Teil I

Anleitungen

1 Speicherorganisation

Informationen sind in Computern in Dateien gespeichert. Die gespeicherten Informationen können dabei ganz unterschiedlicher Art, wie zum Beispiel Texte, Bilder oder Videos, sein. Als Modell, wie man sich Dateien vorstellen kann, hat sich das Bild von Dossiers in Ordnern etabliert. Die einzelnen Dossiers sind die Dateien und die Ordner sind die Strukturen, in denen die Dateien abgelegt sind. Diese Konstruktion kann über mehrere Ebenen hinaus verschachtelt werde (Ordner in Gestellen, die wiederum in einzelnen Räumen stehen, etc.).

1.1 Dateinamen und Pfade

Damit Dateien identifiziert und gefunden werden können, müssen sie einen Namen haben. Grundsätzlich gibt es keine Einschränkungen, wie Dateien benannt werden. Die meisten Dateinamen bestehen allerdings aus zwei Teilen: dem eigentlichen Dateinamen und der Dateinamenserweiterung.

Der eigentliche Dateiname wird idealerweise so festgelegt, dass er einen Rückschluss auf den Inhalt der Datei zulässt.
Die Dateinamenserweiterung ist ein Zusatz, der Auskunft über den Dateityp gibt. Sie steht hinter einem Punkt hinter dem eigentlichen Dateinamen. Auf Windows Rechnern wird die Dateinamenserweiterung im Dateimanager in der Standardeinstellung nicht angezeigt. Um dies zu ändern, muss in den Einstellungen des Dateimanagers die Option “Dateinamenerweiterungen anzeigen” aktiviert werden (Ansicht > Anzeigen > Dateinamenserweiterung).

Damit Dateien besser ausgetauscht werden können, empfiehlt es sich, für die Namen lediglich Buchstaben, Zahlen und Unterstriche (sog. ASCII-Zeichen) zu verwenden.

Damit man Dateien finden kann, muss man wissen, wo sie abgelegt worden sind. Übertragen auf das Modell von Dossiers in Ordnern bedeutet das, zu wissen, welches Dossier in welchem Ordner in welchem Gestell in welchem Raum abgelegt ist. Wie in einem realen Archiv, geht man dabei vom Raum zum Gestell, zum Ordner und schliesslich zum Dossier. In der Informatik wird dieser Weg als Pfad bezeichnet. Auf einem Windows-Rechner beginnt dieser Pfad mit dem sogenannten Laufwerksbuchstaben, gefolgt von einem Doppelpunkt und einem Backslash (\). Aus historischen Gründen ist der Laufwerksbuchstabe auf Windows-Rechnern Standardmässig der Buchstabe `C`.

Ein Beispiel für einen Pfad könnte so aussehen:

```
1 C:\Users\fritz\Documents\text.docx
```

In diesem Beispiel ist `C:` der Laufwerksbuchstabe, `Users` der Ordner, `fritz` der Unterordner, `Documents` der Unterordner von `fritz` und `text.docx` die Datei, die im Ordner `Documents` abgelegt ist. `Users` ist ein von Windows standardmässig angelegter Ordner, in dem die persönlichen Daten der Benutzer abgelegt werden. Der Ordner `fritz` ist der persönliche Ordner des Benutzers `fritz`. Der Ordner `Documents` wird ebenfalls standardmässig von Windows im Ordner jedes Benutzers angelegt. Dem Benutzer `fritz` steht es frei, diesen Ordner zu verwenden und darin Dateien oder Underordner anzulegen.
Der Dateiname `text.docx` verweist mit seiner Dateinamenserweiterung `.docx` auf eine Datei, die mit dem Programm Microsoft Word erstellt worden ist.

1.2 Dateien in der Cloud

Dateien können nicht nur lokal auf dem Computer gespeichert werden. Damit von überall und jederzeit auf Dateien zugegriffen werden kann, werden Dateien in der *Cloud* gespeichert. Dabei handelt es sich um Server in Rechenzentren, die über das Internet erreichbar sind. Beispiele für solche Cloud-Dienst sind OneDrive von Microsoft oder Google Drive.
Ablageorte auf OneDrive werden in Windows direkt in der Verzeichnisstruktur des Betriebssystems angezeigt, solche von Google erhalten auf Windows einen eigenen Laufwerksbuchstaben (`G:\`).

Damit auf die Dateien in der Cloud zugegriffen werden kann, ist eine Internetverbindung erforderlich.

1.3 Dateistruktur für die Schule

Für den schulischen Bedarf erscheint es sinnvoll eine Dateistruktur nach Fächern anzulegen. Im Ordner `Documents` wird dazu für jedes Fach ein eigener Unterordner angelegt. Innerhalb der jeweiligen Fachordner kann eine weitere Struktur nach Semester oder nach Thema sinnvoll sein. Ein Beispiel für die Ordnerstruktur eines Erstklässlers an der KBW kann so aussehen:

```
Documents\
          ├──Schule\
          |       ├──Deutsch
          |       ├──Franz
          |       ├──Mathe
          |       ├──WR
          |       └──...
          ├──Privat\
          |       ├──Rechnungen
          |       └──...
```

2 Arbeitsumgebung (Arbeiten mit Jupyter Notebooks)

2.1 Ausgangslage

In der Informatik geht es darum, wie Informationsverarbeitung mit Hilfe von Computern automatisiert werden kann.
Die Automatisierung der Informationsverarbeitung erfordert die Verwendung von Programmiersprachen. Im Informatikunterricht wird in erster Linie mit der Programmiersprache Python gearbeitet.
Im folgenden findet sich eine Anleitung für die Installation der für den Unterricht erforderlichen Programme.

2.2 Installation von Python

Dieser Abschnitt führt Sie Schritt für Schritt durch die Installation von Python auf einem Windows-Rechner.

> ⚠ **Microsoft Store Falle**
>
> Achten Sie beim Herunterladen von Python darauf, dass Sie sich auf der offiziellen Seite von Python (https://www.python.org) und **nicht** im Microsoft Store befinden. Wenn Sie versehentlich die Python Version aus dem Microsoft Store installiert haben, kann das bei der Arbeit an den Schulprojekten zu Problemen führen.
> Deinstallieren Sie die Microsoft Version von Python und installieren Sie die Version von der offiziellen Website.

1. Laden Sie die neueste Version von Python von der offiziellen Website herunter: python.org.
2. Führen Sie das heruntergeladene Installationsprogramm durch Doppelklick auf die Datei aus. Stellen Sie sicher, dass Sie die Option “Add Python to PATH” aktivieren, bevor Sie auf “Install Now” klicken.

> **Die 'PATH'-Umgebungsvariable**
>
> Stellen Sie sich die PATH-Variable wie ein Adressbuch für die Kommandozeile (Terminal) vor. Wenn Sie einen Befehl wie `python` eingeben, schaut der Computer in diesem Adressbuch nach, wo das entsprechende Programm zu finden ist.
> Indem Sie das Häkchen bei "Add Python to PATH" setzen, fügen Sie die Adresse des Python-Interpreters zu diesem Adressbuch hinzu. Ohne diesen Eintrag weiss der Computer nicht, wo er suchen soll, und meldet, dass er den Befehl nicht kennt.

3. Überprüfen Sie die Installation, indem Sie die Eingabeaufforderung öffnen (Terminal → Windows-Taste + R, dann `cmd` eingeben) und den Befehl `python --version` eingeben. Dies sollte die installierte Python-Version anzeigen.

2.3 Hello World

Es hat sich eingebürgert, dass das erste Programm, das ausgeführt wird, ein Programm ist, das den Text "Hello World" auf dem Bildschirm ausgibt. Um dieser Tradition zu folgen, führen Sie die folgenden Schritte aus:

1. Öffnen Sie ein Terminal (Windows-Taste + R, dann `cmd` eingeben).

> **Das Terminal**
>
> Unter dem Begriff "Terminal" versteht man ein Programm, das eine textbasierte Benutzeroberfläche bereitstellt, um mit dem Betriebssystem zu interagieren. In einem Terminal können Sie Befehle eingeben und erhalten die Ausgaben direkt im Fenster.

2. Geben Sie den Befehl `python` ein, um die Python-Shell zu starten. Die Python Shell sollte ungefähr so, wie das folgende Bild aussehen.

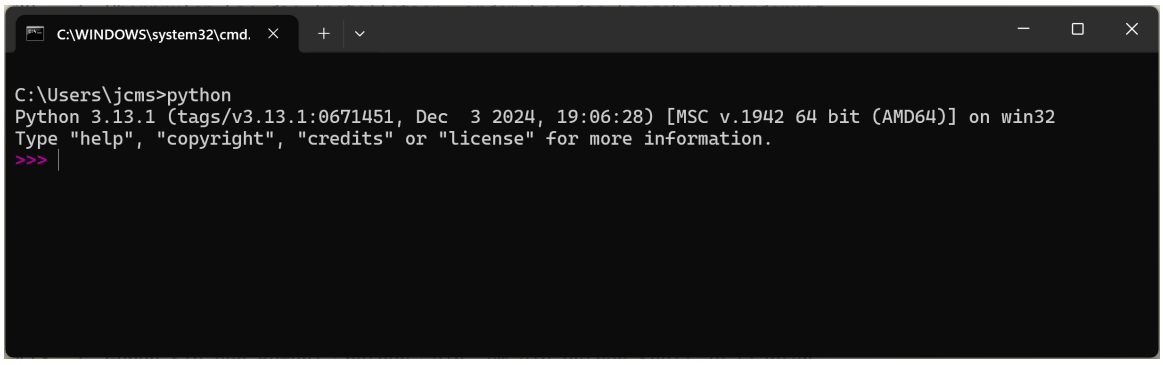


Abbildung 2.1: Python Shell

3. Geben Sie den folgenden Befehl ein und drücken Sie anschliessend die Eingabetaste:

```
1  print("Hello World")
```

Das Resultat sollte wie das folgende Bild aussehen.

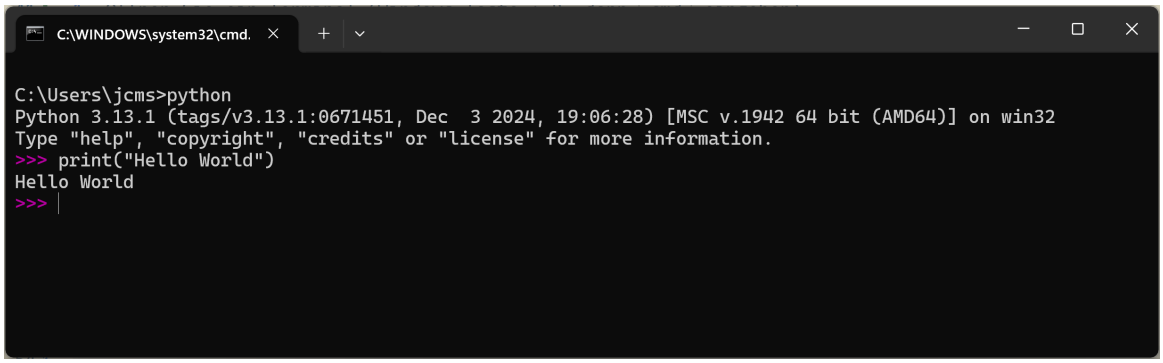


Abbildung 2.2: Python Shell

Gratuliere - Sie haben Ihr erstes Python-Programm erfolgreich ausgeführt!

2.4 Arbeitsumgebung

Im Unterricht wird nicht direkt in der Python-Shell gearbeitet, sondern mit sogenannten Jupyter Notebooks. Jupyter Notebooks ermöglichen es, in der gleichen Datei sowohl Code (Programm Teile) als auch formatierten Text (in Markdown) zu verarbeiten. Eine Jupyter Notebook Datei hat die Endung `.ipynb`. Vom Jupyter Notebook unterschieden werden muss die Arbeitsoberfläche in welcher die Jupyter Notebooks bearbeitet werden. Diese Oberfläche nennt sich JupyterLab und läuft in einem Webbrowser.

> **Das Jupyter Ökosystem**
>
> Die im Unterricht verwendeten Jupyter Notebooks sind Teil eines ganzen Jupyter Ökosystems. Der Name Jupyter setzt sich aus den drei Programmiersprachen **Ju**lia, **Pyt**hon und **R** zusammen, die in diesem Ökosystem eine zentrale Rolle spielen. Zum Jupyter Ökosystem gehören auch zahlreiche Erweiterungen und Tools, die die Arbeit mit Notebooks und Daten erleichtern.
> Der Unterricht beschränkt sich auf die Verwendung von Jupyter Notebooks mit der Programmiersprache Python sowie den Einsatz von JupyterLab als Arbeitsumgebung.

Damit dies alles funktioniert, braucht es ein paar weitere Vorbereitungsarbeiten.

1. Erstellen Sie im Ordner “Informatik” einen Unterordner mit dem Heutigen Datum als Namen. Formatieren Sie das Datum nach dem Schema “YYMMDD”, für den 1. August 2025 wäre das zum Beispiel “250801”.
2. Öffnen Sie den soeben erstellten Ordner.
3. Geben Sie die Tastenfolge `Ctrl + L` ein, um die Adresszeile des Dateimanagers zu aktivieren.
4. Überschreiben Sie den Inhalt der Adresszeile mit dem Text `cmd` und drücken Sie die Eingabetaste. Dadurch wird ein Terminal geöffnet, das direkt im aktuellen Ordner arbeitet.
5. Geben Sie im neu geöffneten Terminal den folgenden Befehl ein und drücken Sie die Eingabetaste:

   ```
   python -m venv .venv
   ```

   Dadurch wird eine sogenannte Python Virtual Environment erstellt (venv wegen **V**irtual **Env**ironment). Dieses Python Virtual Environment dient dazu, die eigenen Programmierprojekte unabhängig voneinander gestalten zu können.
6. Aktivieren Sie das Python Virtual Environment mit dem folgenden Befehl:

   ```
   .venv\Scripts\activate
   ```

   Ihr Terminal sieht nach dem Erstellen und Aktivieren der Python Virtual Environment ungefähr so aus:

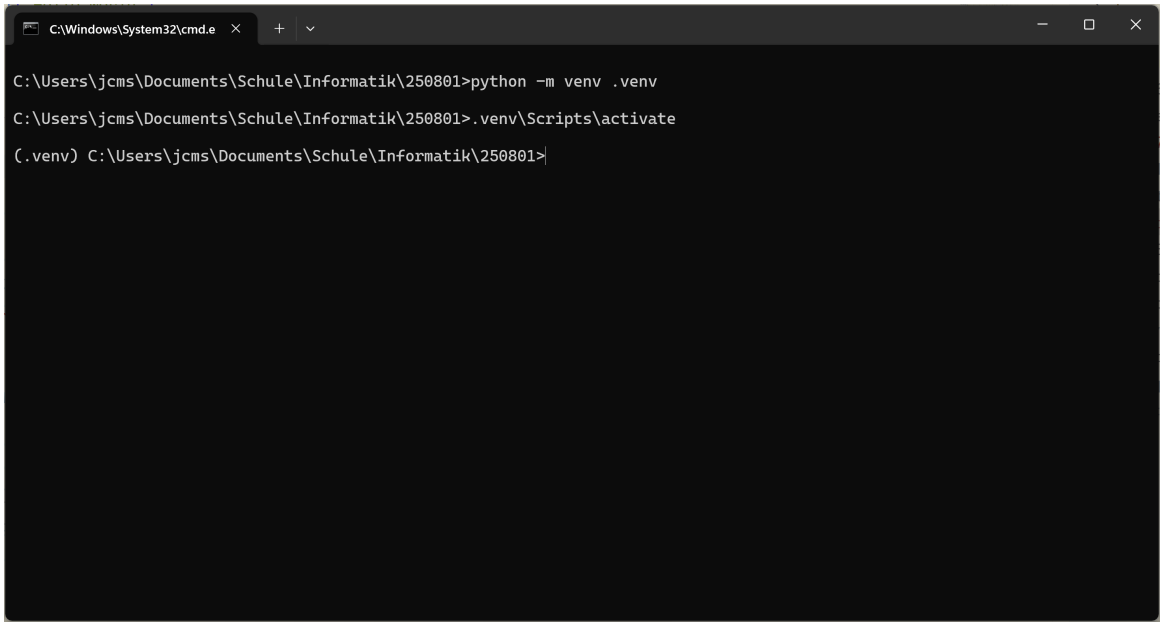


Abbildung 2.3: Aktivierte Python Virtual Environment

Das Wort in der Klammer am Anfang der Zeile zeigt den Namen der aktiven Python Virtual Environment an. Im vorliegenden Fall ist das .venv.

7. In der nun aktivierten Python Virtual Environment installieren Sie die benötigten Pakete mit dem folgenden Befehl:

```
pip install jupyter
```

Das dauert eine Weile.

Während der Installation werden die benötigten Pakete (Ergänzungen zur bestehenden Python Installation) heruntergeladen und in der Python Virtual Environment gespeichert. Das Terminal sieht dabei ungefähr so aus:

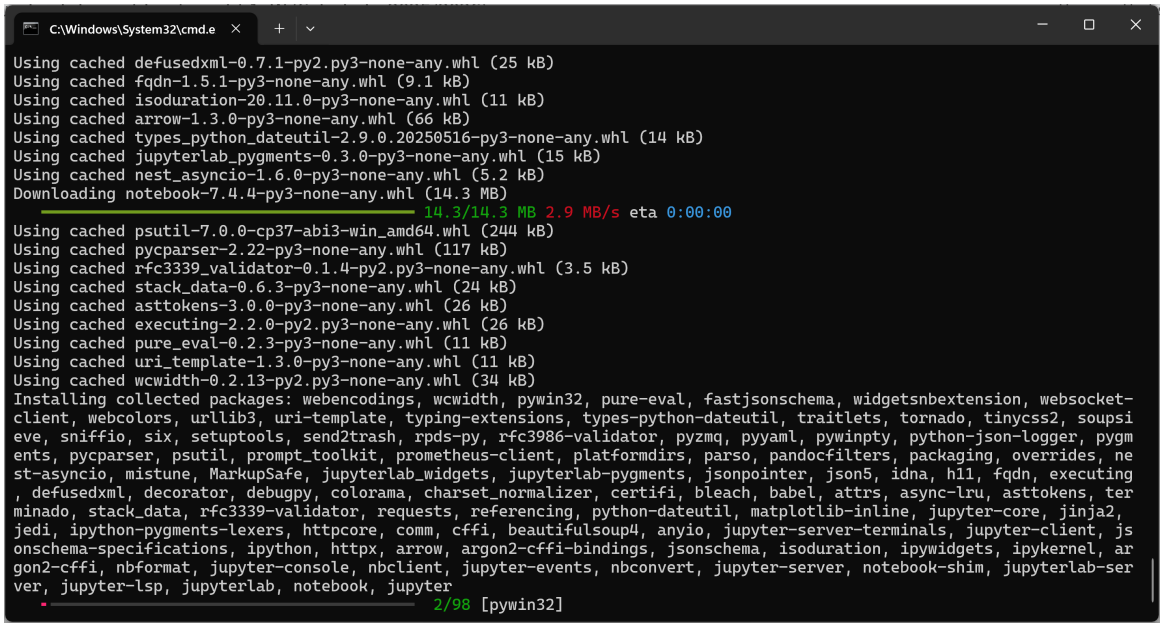


Abbildung 2.4: Terminal während der Jupyter Installation

Alle in einer Python Virtual Environment installierten Pakete sind innerhalb dieser Umgebung dauerhaft verfügbar und müssen daher für das gleiche Projekt kein zweites Mal installiert werden.

8. Starten Sie den Jupyter Server mit dem folgenden Befehl:

```
jupyter-lab
```

Dies startet den Jupyter Notebook Server und öffnet automatisch ein Browserfenster mit der Jupyter Notebook Oberfläche.

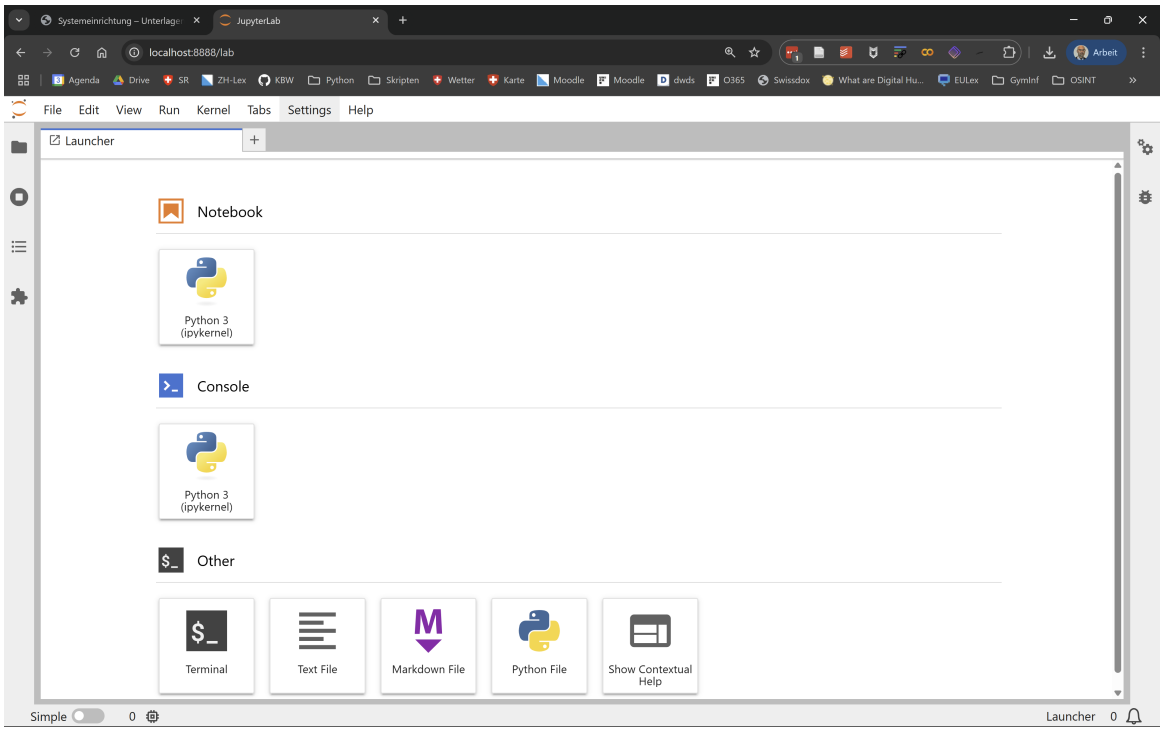


Abbildung 2.5: Startseite Jupyter Lab

9. Klicken Sie auf den Button “Python 3 (ipykernel)” unter dem Titel Notebook.

   Damit starten Sie ein neues Jupyter Notebook. Der Cursor blinkt in einer leeren Zelle. Bei dieser Zelle handelt es sich um eine sogenannte Code-Zelle. In einer Code-Zelle können Sie Python Code eingeben und ausführen.
   Überprüfen Sie das, indem Sie in der Zelle den Befehl `print("Hello World")` eingeben und anschliessend die Tastenfolge `Shift + Enter` drücken (alternativ können Sie auch auf den Button “Run” in der Werkzeugleiste klicken).

   Das Resultat sollte wie das folgende Bild aussehen.



Abbildung 2.6: Hello World in einem Jupyter Notebook

In einem Jupyter Notebook können Sie nicht nur Python Code ausführen, sondern auch Text (formatiert in Markdown) darstellen.
Für die Darstellung von Text müssen Sie die Zelle als Text-Zelle markieren. Dazu klicken Sie auf den Button “Code” in der Werkzeugleiste und wählen im Dropdown-Menü die Option “Markdown” aus.

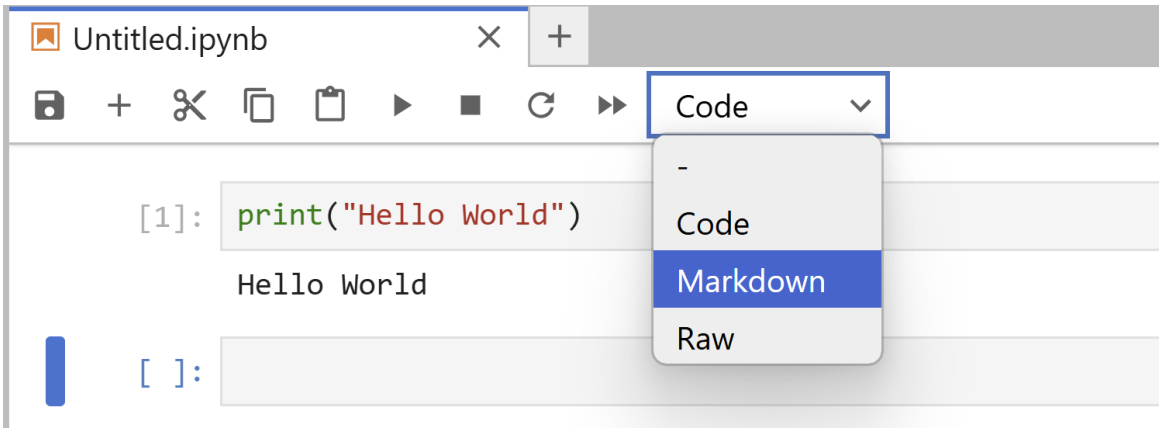


Abbildung 2.7: Umstellen der Zelle auf Markdown

Probieren Sie das aus. Schreiben Sie einen Titel und einen kurzen Text in die Zelle unterhalb der Code-Zelle mit `pringt("Hello World")`. Damit der Text in der Zelle formatiert angezeigt wird, müssen Sie die Zelle mit der Tastenfolge `Shift + Enter` ausführen (analog zum Ausführen von Code-Zellen).

Eine Zelle ist entweder eine Code-Zelle oder eine Text-Zelle. Für den Wechsel zwischen Code- und Text-Darstellung müssen Sie je eine neue Zelle anlegen. Das geht mit der Tastenfolge `Esc + B` (für "Below") oder `Esc + A` (für "Above") während Sie sich in einer Zelle befinden. Alternativ können Sie auch die Buttons "Insert Cell Below" oder "Insert Cell Above" aus den Werkzeugen der Zelle verwenden.

10. Das Jupyter Notebook ist eine Datei mit der Endung `.ipynb`. Neu erstellte Jupyter Notebooks erhalten den Namen "Untitled.ipynb". Um diesen Namen zu ändern, klicken Sie mit der rechten Maustatste auf den Titel "Untitled" in der oberen linken Ecke des Jupyter Notebooks und wählen Sie die Option "Rename" aus dem Kontextmenü. Anschliessend können Sie den Namen des Jupyter Notebooks eingeben.

2.5 Öffnen bestehender Jupyter Notebooks

Häufiger als das Erstellen eines neuen Jupyter Notebooks ist das Öffnen eines bereits bestehenden Jupyter Notebooks. Hier wird das entsprechende Vorgehen beschrieben.

1. Navigieren Sie in den Ordner in dem sich das Jupyter Notebook befindet.
2. Stellen Sie sicher, dass der Ordner über eine Python Virtual Environment mit installierten Jupyter Paketen verfügt.

   Öffnen Sie dazu im ausgewählten Ordner das Terminal (`ctrl + L` anschliessend `cmd` und Eingabetaste). Dann starten Sie die Python Virtual Environment und geben den Befehl `pip list` ein. Dieser Listet alle in der Python Virtual Environment installierten Pakete auf. Falls die Jupyter Pakete nicht aufgelistet werden, müssen Sie diese wie oben beschrieben installieren.

3. Starten Sie den Jupyter Server mit dem Befehl `jupyter-lab`.
4. Öffnen Sie das Dateiverzeichnis. Dazu müssen Sie auf dem linken Rand das Ordner-Symbol anklicken.
5. Wählen Sie das Jupyter Notebook aus, das Sie öffnen möchten. Mit einem Doppelklick auf das Jupyter Notebook wird dieses geöffnet.

   Wenn Sie das Dateiverzeichnis wieder schliessen möchten, klicken Sie auf das Ordner-Symbol auf der linken Seite erneut.

Diese Schritte funktionieren auch, wenn Sie ein Jupyter Notebook öffnen möchten, das Sie von jemand anderem erhalten haben. Sie müssen dieses dazu lediglich in den Ordner kopieren, in dem sich die Python Virtual Environment mit den installierten Jupyter Paketen befindet.

2.6 Häufige Fehlermeldungen

Problem / Fehlermeldung (Was Sie sehen)	Mögliche Ursache (Warum es passiert)	Lösung (Was Sie tun können)
Der Befehl `python` ist entweder falsch geschrieben oder konnte nicht gefunden werden.	Python wurde bei der Installation nicht zur PATH-Variable hinzugefügt. Der Computer weiss nicht, wo er `python.exe` finden soll.	1. Deinstallieren Sie Python über die Systemsteuerung. 2. Installieren Sie Python erneut. 3. Achten Sie diesmal unbedingt darauf, das Häkchen bei "Add Python to PATH" zu setzen.
Der Befehl `jupyter-lab` ist entweder falsch geschrieben oder konnte nicht gefunden werden.	Sie haben vergessen, die virtuelle Umgebung zu aktivieren. Der Befehl `jupyter-lab` existiert nur innerhalb der aktivierten Umgebung.	1. Überprüfen Sie, ob `(.venv)` am Anfang der Kommandozeile steht. 2. Falls nicht, führen Sie den Aktivierungsbefehl erneut aus: `.venv\Scripts\activate`.

Problem / Fehlermeldung (Was Sie sehen)	Mögliche Ursache (Warum es passiert)	Lösung (Was Sie tun können)
`ImportError: DLL load failed...` oder ähnliche Fehler unter Windows	Ein häufiges Problem mit der Installation von `pywin32`, einer wichtigen Windows-Bibliothek, die von Jupyter benötigt wird.	1. Stellen Sie sicher, dass Ihre virtuelle Umgebung aktiv ist. 2. Führen Sie den Befehl `pip install --upgrade pywin32` aus, um die Bibliothek zu reparieren.
Kernel Error oder der Status "Kernel starting, please wait..." ändert sich nicht	Die Verbindung zwischen der Browser-Oberfläche und dem Python-"Gehirn" (dem Kernel) ist gestört. Dies kann viele Ursachen haben.	1. Der einfachste erste Schritt: Klicken Sie im JupyterLab-Menü auf "Kernel" -> "Restart Kernel...". 2. Wenn das nicht hilft, schliessen Sie JupyterLab im Terminal (mit der Tastenkombination Strg + C) und starten Sie es mit `jupyter-lab` neu.
`Permission denied` (Zugriff verweigert) bei der Installation von Paketen	Sie versuchen, Pakete an einem systemweiten Ort zu installieren (z. B. in `C:\Program Files`), für den Sie keine Schreibrechte haben.	Dies ist genau das Problem, das virtuelle Umgebungen lösen! Stellen Sie sicher, dass Ihre `venv` aktiv ist (`(.venv)` muss sichtbar sein). Dadurch wird sichergestellt, dass alle Pakete lokal in Ihren Projektordner installiert werden, wo Sie die vollen Rechte haben.

Teil II

Einführung in Python

3 Problemlösung in der Informatik

3.1 Ausgangslage

Jede Disziplin hat ihre eigene Art, Probleme zu lösen. Das ist in der Informatik nicht anders. In der Informatik versucht man, grosse Probleme in kleinere Teilprobleme zu zerlegen. Das macht man so lange, bis die Teilprobleme so klein sind, dass sie einfach zu lösen sind.

Dies soll hier anhand von verschidenen Grafiken gezigt werden.

Damit in Python einfach mit Grafiken gearbeitet werden kann, wird das Paket `PyTamaro` verwendet. Dieses Paket wurde von der Università della Svizzera italiana (USI) extra für die Informatik-Ausbildung entwickelt.

Damit das Paket verwendet werden kann, muss es zuerst in der aktuellen Python Virtual Environment installiert werden.
Dazu öffnen Sie ein Terminal im Ordner, in dem sich dieses Jupyter Notebook befindet. Anschliessend starten Sie die Python Virtual Environment mit dem Befehl:

```
.venv\Scripts\activate
```

Danach können Sie `PyTamaro` mit dem Befehl

```
pip install pytamaro
```

installieren.

Um das Paket zu verwenden, muss es *importiert* werden. Die genauen Zusammenhänge müssen im Moment nicht bekannt sein. Wichtig ist lediglich, dass die folgende Zelle ausgeführt wird.

```
from pytamaro.de import (
    rechteck, kreis_sektor,
    blau, rot, weiss, schwarz,
    neben, ueber, ueberlagere,
    drehe, kombiniere,
    zeige_grafik, speichere_grafik,
)
```

3.2 Beispiel: Tricolore

Die Vorgehensweise wird anhand der Französischen Nationalflagge (Tricolore) gezigt.

Um die Zeichnung der Tricolore zu planen, wird die Grafik in ihre Einzelteile zerlegt.
Die Tricolore besteht aus drei gleich grossen Rechtecken in den Farben blau, rot und weiss.
Diese Rechtecke werden nebeneinander angeordnet.

Das bedeutet, dass die Länge und die Breite der Rechtecke definiert werden muss und basierend auf diesen Werten die drei Rechtecke gezeichnet werden. Anschliessend werden die drei Rechtecke nebeneinander angeordnet.

Der Befehl zum Zeichnen eines Rechtecks lautet

```
name = rechteck(länge, breite, farbe)
```

Bevor die Zeichnung tatsächlich erstellt wird, soll hier der Befehl im Detail erklärt werden:

- `name` ist der Name, unter dem das Rechteck gespeichert wird. Dieser Name kann später verwendet werden, um auf das Rechteck zuzugreifen.
- `rechteck` ist der Befehl, der ein Rechteck zeichnet. In der Klammer hinter dem Befehl werden die sogenannten *Argumente* angegeben. Diese steuern, wie das Rechteck aussieht.
- `länge` und `breite` sind die Argumente, die die Grösse des Rechtecks bestimmen. Diese Werte können beliebig gewählt werden.
- `farbe` ist das Argument, das die Farbe des Rechtecks bestimmt. Aufgrund der Eigenheiten von `PyTamaro` können ausschliesslich die Farben verwendet werden, welche importiert worden sind.

3.2.1 Rechtecke zeichnen

Als erstes wird hier gezeigt, wie das blaue Rechteck gezeichnetwird.

Damit das Resultat kontrolliert werden kann, wird die Grafik mit dem Befehl

```
zeige_grafik(name)
```

angezeigt.

```
bleu = rechteck(50, 100, blau)
zeige_grafik(bleu)
```



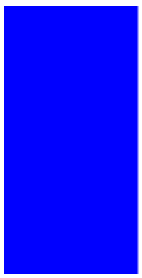
Nachdem das blaue Rechteck gezeichnet wurde, kann das weisse und das rote Rechteck analog gezeichnet und angzeigt werden.

```
# TODO: Rechtecke blanc und rouge zeichnen -> Schreiben Sie hier Ihren
# Code
blanc = rechteck(50, 100, weiss)
rouge = rechteck(50, 100, rot)
```

Als nächstes werden die drei Rechtecke nebeninander angeordnet. Dazu wird der Befehl `neben` verwendet. Dieser Befehl nimmt zwei Argumente entgegen: das erste Rechteck und das zweite Rechteck. Das erste Rechteck wird links vom zweiten Rechteck gezeichnet.

```
resultat = neben(linke grafik, rechte grafik)
```

```
zwei_drittel = neben(bleu, blanc)
zeige_grafik(zwei_drittel)
```



Analog können Sie nun das rote Rechteck rechts der zwei Drittel anordnen. Nennen Sie das Resultat `tricolore` und zeigen Sie es an.

```
# TODO: Trcolore zusammenfügen -> Schreiben Sie hier Ihren
# Code
```

3.3 Beispiel: Österreichische Flagge

Zeichnen Sie die Österreichische Flagge. Das Seitenverhältnis der Flagge ist 2:3.

Um Elemente übereinander anzuordnen, wird der Befehl `ueber` verwendet. Die Syntax dieses Befehls lautet:

```
resultat = ueber(obere grafik, untere grafik)
```

```
# TODO: Östereichische Flagge zeichnen -> Schreiben Sie hier Ihren
# Code
```

3.4 Beispiel: Schweizerfahne

Zeichnen Sie eine korrekt propotionierte Schweizerfahne. Die Dimensionen können Sie der folgenden Grafik entnehmen:

Verwenden Sie dazu die Befehle `rechteck`, `drehe` und `ueberlagere`.

```
# TODO: Schweizerfahne zeichnen -> Schreiben Sie hier Ihren
# Code
```

3.5 Beispiel: Tessiner Wappen

Als Referenz an die USI zeichnen Sie als letztes Beispiel das Tessiner Wappen.

Verwenden Sie dazu neben den bereits bekannten Befehlen zusätzlich die Befehle `ueber` und `kreis_sektor`.

```
# TODO: Tessiner Wappen zeichnen -> Schreiben Sie hier Ihren
# Code
```

3.6 Musterlösungen

3.6.1 Trikolore

```
bleu = rechteck(50, 100, blau)
blanc = rechteck(50, 100, weiss)
rouge = rechteck(50, 100, rot)

deux_tiers = neben(bleu, blanc)
tricolore = neben(deux_tiers, rouge)
```

```
7
8  zeige_grafik(tricolore)
```

3.6.2 Österreichische Flagge

```
1  roter_balken = rechteck(90, 20, rot)
2  weisser_balken = rechteck(90, 20, weiss)
3
4  oberer_teil = ueber(roter_balken, weisser_balken)
5  osterreichische_flagge = ueber(oberer_teil, roter_balken)
6
7  zeige_grafik(osterreichische_flagge)
```

3.6.3 Schweizerfahne

```
1  hintergrund = rechteck(320, 320, rot)
2  weisser_balken = rechteck(200, 60, weiss)
3  weisser_balken2 = drehe(90, weisser_balken)
4
5  kreuz = ueberlagere(weisser_balken, weisser_balken2)
6
7  schweizerfahne = ueberlagere(kreuz, hintergrund)
8
9  zeige_grafik(schweizerfahne)
```

3.6.4 Tessiner Wappen

```
1  roter_teil = rechteck(160, 240, rot)
2  blauer_teil = rechteck(160, 240, blau)
3  roter_sektor = kreis_sektor(160, 90, rot)
4  roter_sektor = drehe(180, roter_sektor)
5  blauer_sektor = kreis_sektor(160, 90, blau)
6  blauer_sektor = drehe(270, blauer_sektor)
7
8  ti_unten = neben(roter_sektor, blauer_sektor)
9  ti_oben = neben(roter_teil, blauer_teil)
10
11 tessin = ueber(ti_oben, ti_unten)
12
```

```
13 zeige_grafik(tessin)
```

4 Variabeln in Python

4.1 Vorbemerkungen: Python als Rechner

Python verfügt über eingebaute mathematische Fähigkeiten. Es kann die Grundrechenarten und kennt die Hierarchie der Operationen. Sie können das überprüfen, in dem Sie in der folgenden Zelle die Rechnung

$$2 + 3 \cdot 4$$

ausführen.

```
1  # hier können Sie die Rechnung ausführen
```

Die folgende Tabelle gibt einen Überblick über die direkt in Python verfügbaren mathematischen Funktionen:

Beschreibung	Befehl	Beispiel	Resultat
Addition	`+`	2 + 3	5
Subtraktion	`-`	3 - 2	1
Multiplikation	*	3 * 2	6
Division	`/`	3 / 2	1.5
Potenzen	**	3 ** 2	9
Wurzeln	`**(1/n)`	16 ** (1/2)	4.0
Ganzzahlige Division	`//`	7 // 2	3
Modulo	`%`	7 % 2	1

4.2 Variablen

In Python sind Variablen symbolische Namen für gespeicherte Daten. Variablen verweisen dabei auf den Speicherbereich im Computer, in welchem die entsprechenden Daten physikalisch abgelegt sind. Aus diesem Grund werden Variablen gelegentlich auch als Zeiger bezeichnet. Was genau für Daten in diesem Speicherbereich abgelegt werden, spielt keine Rolle und kann während der Ausführung eines Programmes auch ändern.

In Python werden Variablen Werte mit dem Gleichheitszeichen zugewiesen. Um der Variable x den Wert 2 zuzuweisen, ist die Eingabe `x = 2` erfoderlich. Die Variable muss links vom Gleichheitszeichen, der zuzuweisende Wert rechts davon stehen.

Überprüfen Sie dies, indem Sie in der folgenden Zelle der Variabel y den Wert 3 und der Variabel z den Wert 4 zuweisen. Anschliessend multiplizieren Sie die beiden Variablen miteinander.

```
# hier die Aufgabe einfüllen
```

Wenn Variablen neue Werte zugewiesen werden, wird die Referenz auf den Speicherbereich mit dem alten Wert gelöscht. Die Daten, welche ohne Verweis durch eine Variable im Speicher liegen, werden vom in Python eingebauten *Garbage Collector* im Hintergrund gelöscht und der so freigewordene Speicherplatz kann wieder verwendet werden.

Sie können überprüfen, dass Variablen neue Werte zugewiesen werden können, indem Sie in der untenstehenden Zelle die Variabeln y und z addieren. Sie erhalten dann das Resultat 7. Das heisst, den Variablen y und z sind immer noch die Werte 3 und 4 zugewiesen.

```
# addieren Sie hier y und z
```

Wenn Sie in der folgenden Zelle der Variabel y den Wert 5 zuweisen und anschliessend y und z addieren erhalten Sie als neues Resultat 9.

```
# weisen Sie hier y den neuen Wert zu
```

Variablen können auch Resultate von Berechnungen zugewiesen werden. Ausserdem können Variablen ganze Wörter als Namen haben. Dies ist gegenüber einzelnen Buchstaben vorzuziehen, weil dann aussagekräftige Namen gewählt werden können. Grundsätzich sind die Namen von Variabeln frei wählbar. Es gibt allerdings eine Reihe von reservierten Begriffen, welche in der Programmiersprache Python eine eigene Bedeutung haben. Unzulässig sind ausserdem Namen, die mit Ziffern beginnen.

Für die Darstellung von Namen für Variablen hat sich in Python eingebürgert, Variablen klein zu schreiben und Wörter durch Underlines zu trennen (*this_is_a_valid_variable*). Diese Darstellung nennt sich *Snake Case*. Zudem werden Variablen meist mit englischen Begriffen bezeichnet.

Weisen Sie in der nächsten Zelle der Variable `result` das Resultat der Rechnung $y + z$ zu und geben Sie das Resultat mit `print(result)` aus. `print()` ist eine Funktion, die Python zur Verfügung stellt. Was Funktionen sind, wird im nächsten Abschnitt erklärt.

```
# weisen Sie hier der Variable result das Resultat zu
```

4.3 Funktionen in Python

Python verfügt über viele bereits vordefinierte Funktionen. Die oben verwendete Funktion `print()` ist ein Beispiel dafür. Um zu demonstrieren, wie Funktionen in Python definiert

werden, zeige ich Ihnen als Beispiel eine Funktion, mit der zwei Zahlen addiert werden.

```
# Definition der Funktion
def get_sum(x, y):
    return x + y

# Aufruf der Funktion
result = get_sum(3,4)
# Ausgabe des Resultats des Funktionsaufrufs
print(result)
```

`def` ist das Schlüsselwort für die Definition einer Funktion. `get_sum` ist der von mir gewählte Name dieser Funktion. Für die Wahl des Namens einer Funktion gelten die gleichen Regeln, wie für Variablen. In den Klammern stehen die sogenannten Parameter, welche der Funktion übergeben werden, damit sie etwas damit macht. Mit dem Doppelpunkt wird die *Signatur* der Funktion abgeschlossen. Die *Signatur* zeigt idealerweise, *was* eine Funktion *womit* macht. Sie gibt aber keine Auskunft darüber, wie sie das macht.

Python gruppiert Befehle, die zusammengehören, durch die gleiche Tiefe der Einrückung. Eine Einrückung hat üblicherweise die Tiefe von vier Leerzeichen. Im Beispiel oben gibt es nur eine eingerückte Zeile, weil die Funktion nur aus einem Befehl besteht. Mit `return` gibt die Funktion das Resultat zurück.

Im Beispiel wird das Resultat der Berechnung, welche die Funktion ausführt der Variable `result` zugewiesen. Der Wert der Variable `result` wird mit `print(result)` ausgegeben.

Definieren Sie in der folgenden Zelle eine Funktion, mit der zwei Zahlen multipliziert werden.

```
# hier kommt Ihre Funktion hin
```

4.4 Datentypen

Als nächstes geht es um die Frage, auf welche Inhalte eine Variable zeigen kann.

Im Grundsatz kann eine Variable auf beliebige Inhalte verweisen.

Am einfachsten ist die Verwendung der grundlegenden Datenypen (basic data types), welche Python zur Verfügung stellt. Dies sind (mit ihren englischen Bezeichnungen):

- Integer (Ganzzahl)
- Floating-Point Number (Gleitkommazahl)
- Complex Number (komplexe Zahl)
- String (Zeichenkette)
- Boolean Type (Wahrheitswert)

Darüber hinaus ist es möglich, eigene Datentypen zu programmieren. Hier aber zuerst eine Beschreibung der grundlegenden Datentypen von Python.

4.4.1 Integer

Die Bezeichnung für Integer in Python ist ein kurzes `int`.

Anders als in anderen Progammiersprachen gibt es in Python theoretisch keine Beschränkung, wie gross ein Integer sein kann. Die einzige Grenze ist der Speicherplatz des auf dem der Integer gespeichert werden soll.

Wenn einer Variable ein grosser Integer zugewiesen wird, kann dieser zur besseren Lesbarkeit auch mit einem Underline als Tausendertrennzeichen geschrieben werden (`100_000`).

Um das Auszuprobieren, weisen Sie in der folgenden Zelle der Variable *a* den Wert von einer Million und der Variable *b* den Wert von einer Milliarde zu. Anschliessend addieren Sie *a* und *b* und weisen das Resultat der Variable *big_sum* zu. Zum Schluss geben Sie den Wert von *big_sum* mit der Funktion `print()` aus.

```
# hier können Sie Ihre Berechnung vornehmen
```

Eingegebene Zahlen werden automatisch als Dezimalzahlen interpretiert.

Integers können jedoch auch als Binär-, Oktal- oder Hexadezimalzahlen eingegeben werden. Die Eingabe erfordert dann allerdings ein Präfix, welches das Zahlensysten identifiziert. Die folgende Tabelle stellt die möglichen Präfixe zusammen.

Präfix	Bedeutung	Basis
`0b` (Null + Kleinbuchstabe `b`)	Binärzahl	2
`0B` (Null + Grossbuchstabe `B`)		2
`0o` (Null + Kleinbuchstabe `o`)	Oktalzahl	8
`0O` (Null + Grossbuchstabe `O`)		8
`0x` (Null + Kleinbuchstabe `x`)	Hexadezimalzahl	16
`0X` (Null + Grossbuchstabe `X`)		16

In der folgende Zelle finden Sie ein entsprechendes Beispiel.

Python kann Integer in verschiedenen Zahlensystemen darstellen. Um das Zahlensystem bei der Zuweisung zu spezifizieren, verwenden Sie die entsprechenden Präfixe. Die folgenden Beispiele zeigen die Zuweisung der Zahlen 10, 8, 255 und 16 in den Zahlensystemen Dezimal, Oktal, Hexadezimal und Binär:

```
dezimal      = 10
oktal        = 0o10
hexadezimal  = 0x10
```

```
4 binaer       = 0b10
```

Die Anzeige der Werte der entsprechenden Variablen erfolgt grundsätzlich im Dezimalsystem.

```
1 b = 0b101010
2 o = 0o52
3 x = 0x2a
4
5 print(b, o, x)
```

4.4.2 Gleitkommazahl

Die Bezeichnung für Gleitkommazahlen in Python ist `float`. Python interpretiert Zahlen mit einem Dezimalpunkt als Gleitkommazahlen. Optional können Zahlen mit `e` oder `E` in "wissenschaftlicher" Schreibweise eigegeben werden (1000 = 1e3 bzw. 1e-3 = 0.001).

Weisen Sie in der folgenden Zelle den Variablen `million` und `billionth` die passenden Werte in wissenschaftlicher Schreibweise zu.

```
1 # hier die Werte den beiden Variablen zuweisen
```

4.4.3 Komplexe Zahlen

Python kann auch mit komplexen Zahlen umgehen. Der Abschnitt zu diesem Thema kann wieder aufgegriffen werden, wenn Sie in Mathe die komplexen Zahlen besprochen haben.

4.4.4 String

Zeichenketten (String) werden von Python als `str` bezeichnet.

Zeichenketten sind beliebige Zeichenfolgen. Damit Python Zeichenketten als solche erkennt, müssen sie durch die Verwendung von einfachen oder doppelten Anführungs- und Schlusszeichen als solche gekennzeichnet werden.

`"Ich bin eine Zeichenkette."` oder `'Ich bin auch eine Zeichenkette.'`

Wenn man innerhalb einer Zeichenkette Anführungszeichen braucht, müssen die eingrenzenden Anführungszeichen von der "anderen Sorte" sein (`"It's cool learning Python!"` oder `'Der Lehrer sagt: "Es ist cool Python zu lernen."'`). Eine andere Möglichkeit reservierte Zeichen zu verwenden ist der Gebrauch eines "escape"-Zeichens. In Python ist das der "backslash" (\). Die beiden Beispielsätze von vorher hätten entsprechend auch folgendermassen geschrieben werden können:

`'It\'s cool learning Python!'` bzw. `"Der Lehrer sagt: \"Es ist cool Python zu lernen.\""`

Die Länge von Zeichenketten wird lediglich durch die Speicherkapazität des verwendeten Systems begrenzt. Zeichenketten können nicht nur sehr lang, sondern auch leer sein (`''`).

Zeichenketten können, wie alle Datentypen, Variablen zugewiesen werden. Dies zeigt das folgende Beispiel.

```
# Zuweisung eines Strings zu einer Variabel
standard_greeting = "Hello World"

# Ausgabe der Variabel
print(standard_greeting)
```

4.4.5 Boolean Type

Wahrheitswerte werden in Python als `bool` bezeichnet. Wahrheitswerte können entweder "wahr" oder "falsch" sein.

```
wahr      = True
falsch    = False
```

Die Ausgabe findet sich in der folgenden Zelle.

```
wahr      = True
falsch    = False
print(wahr, falsch)
```

Welche Warheitswerte sich bei Vergleichen ergeben, kann mit den folgenden Operatoren überprüft werden:

```
==   # Gleichheit
!=   # Ungleichheit
>    # Größer als
<    # Kleiner als
>=   # Größer oder gleich
<=   # Kleiner oder gleich
```

```
# Testen Sie hier den Wahrheitswert von Vergleichen mit Zahlen
```

Wahrheitswerte werden zur Steuerung von Programmflüssen verwendet. Mit einem Wahrheitswert kann zum Beispiel gesteuert werden, wie oft ein Programmteil wiederholt werden soll.

4.5 Funktionen mit Type-Hints

Zum Abschluss komme ich noch einmal auf die Definition von Funktionen zurück. In Python können Variablen - anders als zum Beispiel in Java - beliebige Datentypen zugewiesen werden. Wenn Variablen im Verlauf eines Programms mehrfach verwendet werden, können ihnen auch unterschiedliche Datentypen zugewiesen werden. Dies ist allerdings schlechter Programmierstil.

Aus diesem Grund ist es sinnvoll, bei der Definition einer Funktion zu deklarieren, welche Datentypen die Parameter haben und welcher Datentyp der Rückgabewert hat. Dies soll mit dem Beispiel der Funktion `get_quotient` verdeutlicht werden.

```
1  def get_quotient(x : int, y : int) -> float:
2      return x / y
```

Hier wird angegeben, dass die Parameter x und y vom Datentyp `int` sein sollen. Der Datentyp des Rückgabewertes wird hinter `->` geschrieben. Im Beispiel ist der Rückgabewert vom Typ `float`. Das ist so, weil die Funktion zum Beispiel $3/4 = 3.5$ rechnet.

Aber Achtung: die Funktion arbeitet auch dann korrekt, wenn ein anderer als der deklarierte Datentyp übergeben wird. Voraussetzung ist lediglich, dass der Datentyp mit den verwendeten Operationen kompatibel ist. Die "Type-Hints" dienen lediglich der besseren Nachvollziehbarkeit, was die Funktion macht.

4.6 Ausgewählte Musterlösungen

4.6.1 Funktion zur Multiplikation zweier Zahlen

```
1  def get_product(a, b):
2      return a * b
3
4  product = get_product(3, 4)
5  print(product)
```

4.6.2 Gleitkommazahlen in wissenschaftlicher Schreibweise

```
1  million = 1e6
2  billonth = 1e-9
3  print(million, billonth)
```

```
1000000.0 1e-09
```

5 Wiederholungen in Python (For-Loops)

Eine Stärke von Computerprogrammen ist die wiederholte Ausführung von Anweisungen. Viele Programmiersprachen stellen dafür ein Konstrukt mit dem Namen 'For-Schleife' zur Verfügung. Eine 'For-Schleife' funktioniert unabhängig von einer konkreten Programmiersprache folgendermassen:

```
FÜR variable VON startwert BIS endwert [MIT schrittweite]
    Anweisungen
ENDE FÜR
```

Übersetzt nach Python sieht das so aus:

```
1  for i in range(n):
2      do...
```

`startwert BIS endwert [MIT schrittweite]` wird dabei durch `range(n)` ausgedrückt. Dabei ist `n` der Endwert. Gezählt wird bis zum aber ohne den Endwert. Startwert und Schrittweite haben Vorgabewerte. Der Vorgabewert für den Start ist `0`, derjenige der Schrittweite `1`. Weil `range()` diese vorgegebenen Werte hat, müssen diese nicht explizit angegeben werden. Wenn der Startwert abweichend vom Vorgabewert festgelegt werden soll, kann dieser explizit angegeben werden. Der Aufruf von `range()` sieht dann so aus:

```
1  range(startwert, endwert)
```

Falls eine von `1` abweichende Schrittweite festgelegt werden soll lautet der Aufruf

```
1  range(startwert, endwert, schrittweite)
```

In diesem Fall müssen neben dem Endwert sowohl der Startwert und die Schrittweite angegeben werden. Andernfalls kann nicht zwischen den einzelnen Angaben zu Endwert, Startwert und Schrittweite unterschieden werden.

5.1 Anwendungsübung zu Wiederholungen in Python

Im Folgenden bauen Sie eine Blume mit Python for-loops und PyTamaro.

In der folgenden Zelle werden zuerst alle von PyTamaro zur Verfügung gestellten Funktionen importiert.

```
1  from pytamaro.de import *
```

Das Ziel ist es, eine Blume wie die abgebildete zu zeichnen.

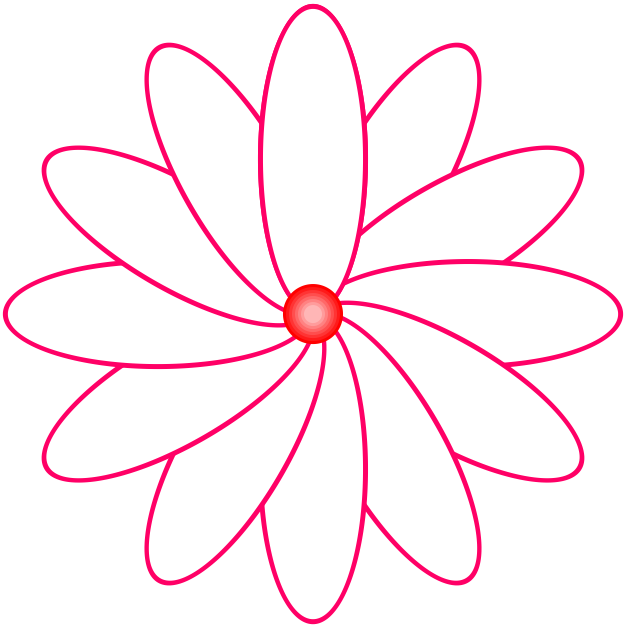


Abbildung 5.1: Blume

Die wichtigsten Funktionen von PyTamaro für diese Aufgabe sind `fixiere()` und `kombiniere()`.

Die Funktion `fixiere()` legt für eine Grafik einen frei gewählten Ankerpunkt fest. Dazu muss man wissen, dass jede in PyTamaro gezichnete Grafik von einer sogenannten "Bounding Box" umgeben ist. Diese Bounding Box ist ein Rechteck, das die Grafik vollständig umschliesst. Standardmässig liegt der Ankerpunkt einer Grafik in der Mitte der Bounding Box. Mit `fixiere()` kann man den Ankerpunkt jedoch an eine andere Stelle der Bounding Box verschieben. PyTamaro stellt die folgenden Positionen für Ankerpunkte zur Verfügung:

- `mitte`: Mitte der Bounding Box (Standard)
- `mitte_links`: Mitte der linken Seite der Bounding Box
- `mitte_rechts`: Mitte der rechten Seite der Bounding Box
- `oben_links`: Obere linke Ecke der Bounding Box
- `oben_mitte`: Mitte der oberen Seite der Bounding Box
- `oben_rechts`: Obere rechte Ecke der Bounding Box
- `unten_links`: Untere linke Ecke der Bounding Box
- `unten_mitte`: Mitte der unteren Seite der Bounding Box
- `unten_rechts`: Obere rechte Ecke der Bounding Box

Der Ankerpunkt wird mit dem Befehl `fixiere(position, grafik)` gesetzt.

Der Ankerpunkt ist wichtig für die Funktion `drehe()`, die eine Grafik um einen bestimmten Winkel dreht. Die Drehung erfolgt immer um den Ankerpunkt.

Die Funktion `kombiniere(vordere Grafik, hintere Grafik)` fügt zwei Grafiken zusammen. Die erste gegebene Grafik liegt im Vordergrund und die zweite im Hintergrund. Die Grafiken werden so ausgerichtet, dass ihre Ankerpunkte übereinanderliegen.

5.1.1 Schritt 1: Zerlege das Bild in seine Einzelteile

Die Blume besteht im wesentlichen aus zwei Formen:

1. Blütenblätter in Form von Ellipsen sowie
2. einer Scheibe in der Form eines Kreises in der Mitte.

Um die Blume zu zeichnen, müssen diese beiden Formen zuerst einzeln erstellt werden.

```
bluetenblatt = ellipse(50, 150, blau)
# die Farbe blau wurde gewhält, damit das Blatt vor dem Hintergrund
# sichtbar ist

zeige_grafik(bluetenblatt)
```



```
scheibe = ellipse(30, 30, rot)
zeige_grafik(scheibe)
```



5.1.2 Schritt 2: Positioniere die Blütenblätter

Die Blütenblätter sind rund um die Mitte der Blume angeordnet. Da es zwölf Blütenblätter sind, beträgt der Winkel zwischen zwei Blütenblättern $\frac{360°}{12} = 30°$. Die Drehung der Blütenblätter erfolgt um den Ankerpunkt `unten_mitte` der Blütenblätter.
Die Blütenblätter müssen also zuerst fixiert und dann gedreht werden.

```
bluetenblatt_fixiert = fixiere(unten_mitte, bluetenblatt)
bluetenblatt_30 = drehe(30, bluetenblatt_fixiert)
zeige_grafik(bluetenblatt_30)
```



5.1.3 Schritt 3: Zeichnen aller erforderlichen Blütenblätter

Die Blume hat zwölf Blütenblätter. Diese können von Hand jedes einzelne erstellt werden. Das dritte Blütenblatt ist um 60° gedreht und wird entsprechend mit

```
bluetenblatt_60 = drehe(60, bluetenblatt_fixiert)
```

erstllt. Das kann man für alle zwölf Blütenblätter machen bis man beim zwölften Blütenblatt angekommen ist, das um 330° gedreht ist.

```
bluetenblatt_330 = drehe(330, bluetenblatt_fixiert)
```

Das ist aber sehr mühsam und fehleranfällig. Viel einfacher ist es, eine Schlaufe (`for`-loop) zu verwenden, die die Blütenblätter basierend auf der Grundform automatisch erstellt.

```
for i in range(12):
    winkel = i * 30
    bluetenblatt_gedreht = drehe(winkel, bluetenblatt_fixiert)

zeige_grafik(bluetenblatt_gedreht)
```



Diese Schlaufe durchläuft die Zahlen von 0 bis 11 (also 12 Zahlen) und berechnet für jede Zahl den entsprechenden Drehwinkel. Dann wird das Blütenblatt um diesen Winkel gedreht. Allerdings wird das gedrehte Blütenblatt in jeder Iteration der Schlaufe in der gleichen Variable `bluetenblatt_gedreht` gespeichert. Am Ende der Schlaufe enthält diese Variable nur das letzte gedrehte Blütenblatt (das um 330° gedrehte Blütenblatt).

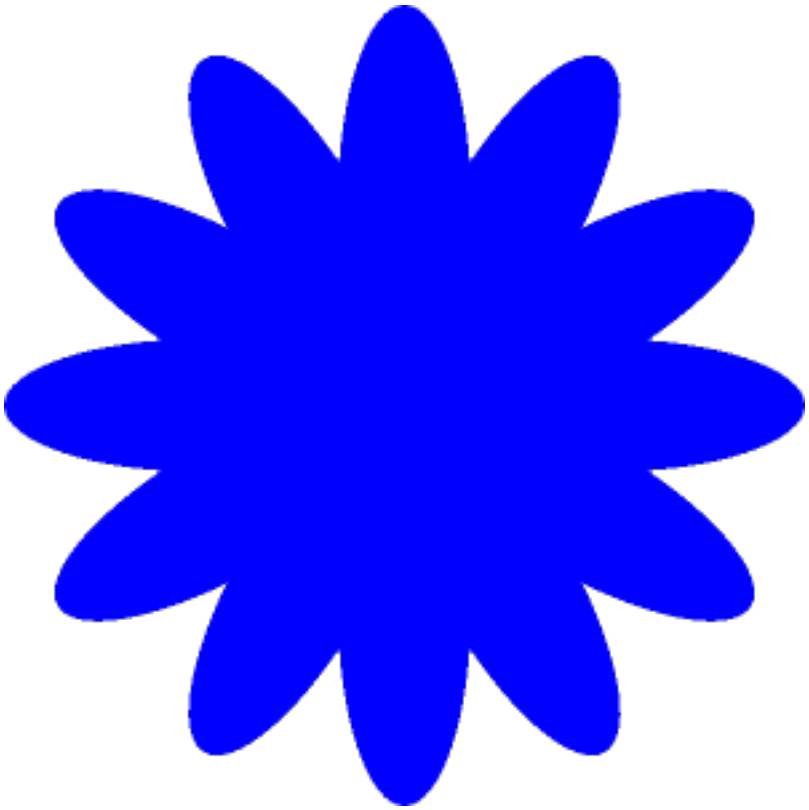
5.1.4 Schritt 4: Kombiniere alle Blütenblätter

Um das unerwünschte Verhalten zu vermeiden, dass am Ende der Schlaufe nur das letzte gedrehte Blütenblatt in der Variable `bluetenblatt_gedreht` enthalten ist, müssen die gedrehten Blütenblätter in jeder Iteration der gewünschten Gesamtgrafik hinzugefügt werden. Dazu wird die Funktion `kombiniere()` verwendet.

```
blume = bluetenblatt_fixiert

for i in range(12):
    winkel = i * 30
    bluetenblatt_gedreht = drehe(winkel, bluetenblatt_fixiert)
    blume = kombiniere(blume, bluetenblatt_gedreht)

zeige_grafik(blume)
```



Damit sind die Blütenblätter in der gewünschten Position und Anzahl zusammengefügt. Damit Sie allerdings aussehen, wie die Blume in der Vorlage, müssen die einzelnen Blütenblätter weiss eingefärbt und mit einem rosa Rand versehen werden.

Rosa ist keine vordefinierte Farbe in PyTamaro. Sie können jedoch eine beliebige Farbe mit der Funktion `rgb_farbe(rot, gruen, blau)` erstellen. Dabei sind `rot`, `gruen` und `blau` Zahlenwerte von 0 bis 255, die den Anteil der jeweiligen Farbe an der Gesamtfarbe angeben. Damit nicht alle Kombinationen durchprobiert werden müssen, können Sie mit einem Online-Tool wie RGB Color Picker die gewünschte Farbe auswählen und die entsprechenden Werte für rot, grün und blau ablesen.

```
weisses_blatt = ellipse(50, 150, weiss)
rosa_rand = ellipse(55, 155, rgb_farbe(255, 0, 102))

bluetenblatt_mit_rand = ueberlagere(weisses_blatt, rosa_rand)

zeige_grafik(bluetenblatt_mit_rand)
```



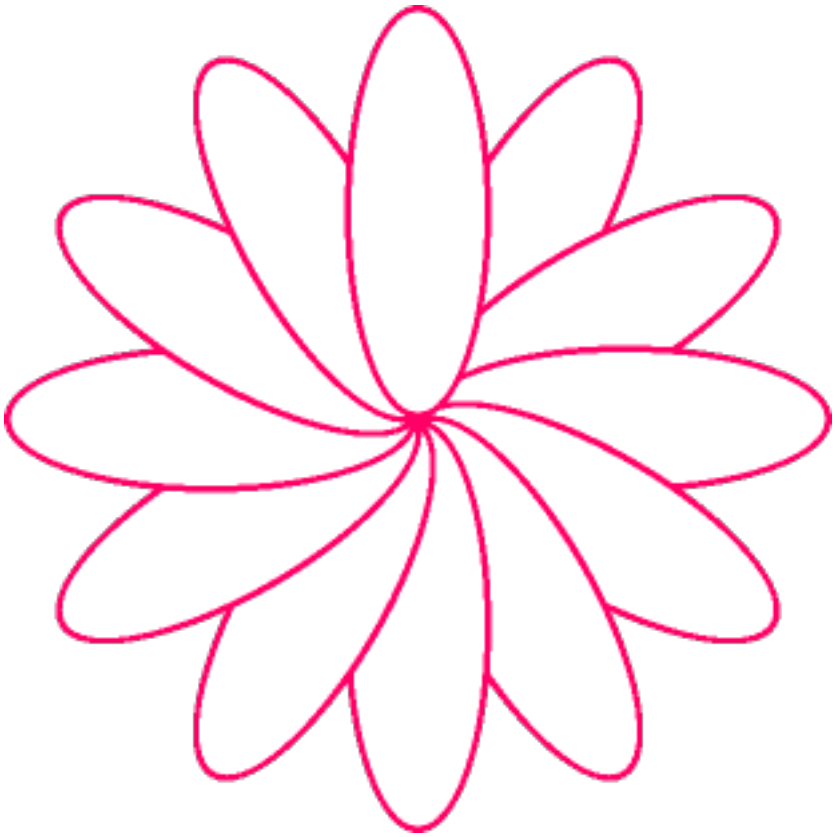
Anschliessend können die Blütenblätter mit Rand in der vorher geschriebenen Schlaufe zur Gesamtgrafik zusammengefügt werden.

```
bluetenblatt_mit_rand_fixiert = fixiere(unten_mitte,
↪ bluetenblatt_mit_rand)

blume_weiss = bluetenblatt_mit_rand_fixiert

for i in range(12):
    winkel = i * 30
    bluetenblatt_gedreht = drehe(winkel, bluetenblatt_mit_rand_fixiert)
    blume_weiss = kombiniere(blume_weiss, bluetenblatt_gedreht)

zeige_grafik(blume_weiss)
```



5.1.5 Schritt 5: Scheibe im Zentrum der Blume

Bei genauer Betrachtung der Blume fällt auf, dass die Scheibe in der Mitte der Blume nicht einfach einfarbig gelb ist, sondern gegen die Mitte hin einen Farbverlauf aufweist. Diesen Farbverlauf wird erzeugt, indem mehrere Kreise mit kleiner werdendem Radius und abnehmender Farbintensität übereinandergelegt werden.

```
scheibe = ellipse(30, 30, rgb_farbe(255, 0, 0))

for i in range(1, 8):
    tmp = ellipse(30 - 3*i, 30 - 3*i, rgb_farbe(255, i*26, i*26))
    scheibe = ueberlagere(tmp, scheibe)

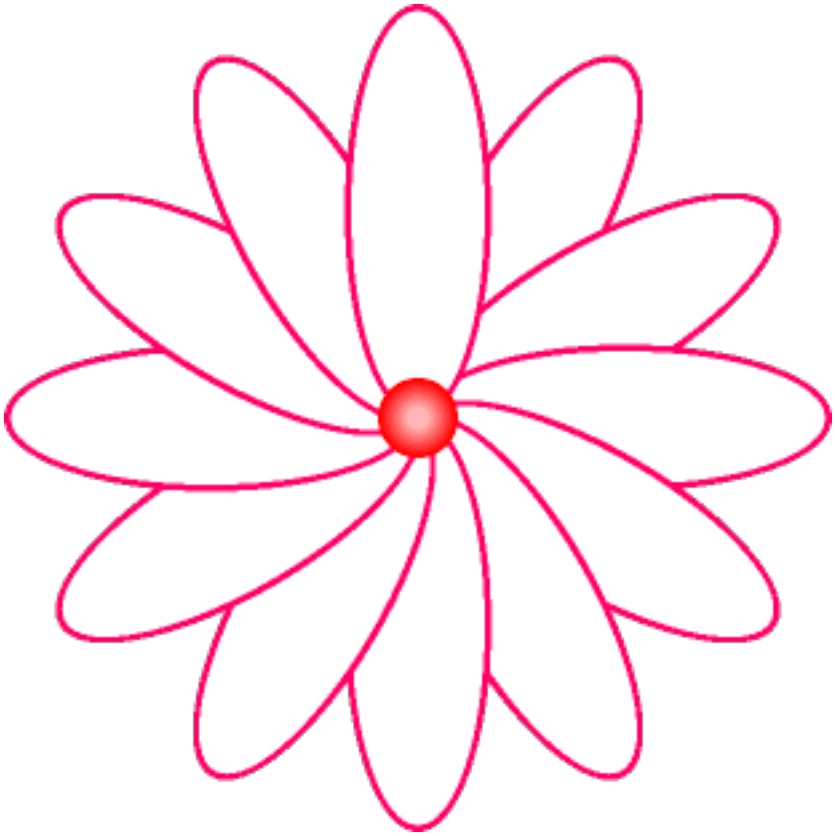
zeige_grafik(scheibe)
```



5.1.6 Schritt 6: Kombination der Elemente zur vollständigen Blume

Zum Schluss wird die Scheibe auf die Blütenblätter gelegt und Blume so vervollständigt.

```
blume_komplett = ueberlagere(scheibe, blume_weiss)
zeige_grafik(blume_komplett)
```



6 Programmverzweigungen (Bedingungen)

Es gibt Situationen, in denen es wünschenswert ist, dass ein Teil eines Programms nur ausgeführt wird, wenn eine bestimmte Voraussetzung erfüllt ist.
Als Beispiel dafür mag die Blume von vergangener Woche dienen. Wenn eine Blume gezeichnet werden soll, welche abwechslungsweise dunkelblaue und hellblaue Blütenblätter hat.

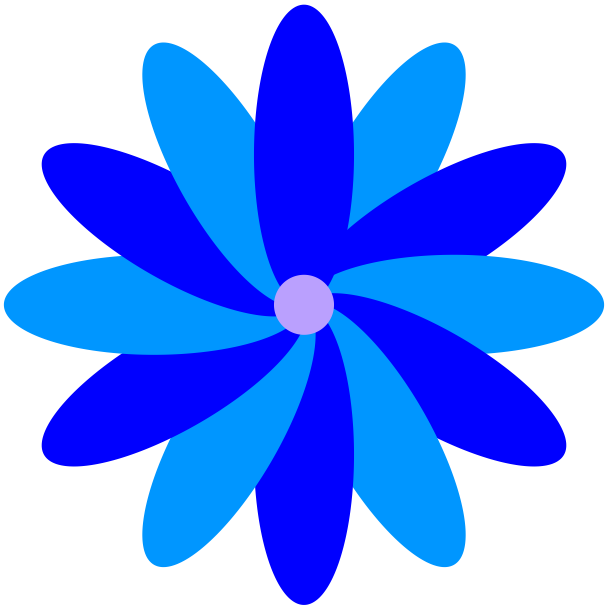


Abbildung 6.1: Beispielblume

Diese Blume wurde ebenfalls mit einer `for`-Schlaufe gezeichnet. Allerdings wurde abwechslungsweise ein dunkelblaues und ein hellblaues Blütenblatt verwendet.
Abwechslungsweise kann auch als wenn eine gerade Zahl kommt bzw. wenn eine ungerade Zahl kommt verstanden werden. Kommen beim Zählen doch abwechslungsweise gerade und ungerade Zahlen. Übersetzt in die `for i in range(12)`-Schlaufe heisst das, immer wenn der Wert in der Variabel `i` eine gerade Zahl ist dann soll ein dunkelblaues Blüentblatt verwendet werden und immer wenn der Wert der Variabel `i` eine ungerade Zahl ist, soll ein hellblaues Blütenblatt verwendet werden.

In Python verwendet man dazu die folgende Syntax:

```
1 if BEDINGUNG:
2     ANWEISUNG(en)
```

```
else:
    ANWEISUNG(en)
```

Das heisst, immer wenn die `BEDINGUNG` erfüllt ist (wenn sie `wahr`) ist, werden die eingerückten Anweisungen ausgeführt. In allen anderen Fällen werden die nach `else:` eingerückten Anweisungen ausgeführt.

Für die Zeichnung der Blume bedeutet das also, dass immer wenn `i` eine gerade Zahl ist, soll ein dunkelblaues Blütenblatt verwendet werden. Eine gerade Zahl ist dabei jede Zahl, die ohne Rest durch zwei Teilbar ist.
Ob dies der Fall ist, kann mit dem Modulo Operator (`%`) getestet werden (vgl. Abschnitt Vorbemerkungen: Python als Rechner.) Eine Zahl ist ohne Rest durch zwei Teilbar, wenn gilt

$$i \; mod \; 2 = 0$$

Übersetzt nach Pyhton schreibt sich das als

```
if i % 2 == 0:
```

Für die Prüfung, ob das Resultat 0 ist, wird dabei `==` verwendet. Dies rührt daher, dass das einfache gleichheitszeichen bereits als Operator für die Zuweisung eines Wertes zu einer Variabel besetzt war.

Der Vollständige Code für die Abwechslungsweise Verwendung eines dunkelblauen bzw. eines hellblauen Blütenblattes sieht folgendermassen aus:

```
if i % 2 == 0:
    blume = kombiniere(blume, bluetenblatt_dunkel)
else:
    blume = kombiniere(blume, bluetenblatt_hell)
```

Falls mehr als zwei Fälle unterschieden werden müssen, lautet die Python Syntax

```
if BEDINGUNG:
    ANWEISUNG(en)
elif BEDINGUNG:
    ANWEISUNG(en)
else:
    ANWEISUNG(en)
```

Dabei können beliebig viele `elif` Bedingungen wiederholt werden.

Als Bedingung kann nicht nur eine Gleichheit überprüft werden. Es ist auch möglich grösser `>`, grösser oder gleich `>=`, kleiner `<` und kleiner oder gleich `<=` zu prüfen.

6.1 Anwendungsübung zu Bedingungen in Python

6.1.1 Aufgabenstellung

Zeichnen Sie die folgende Blume.

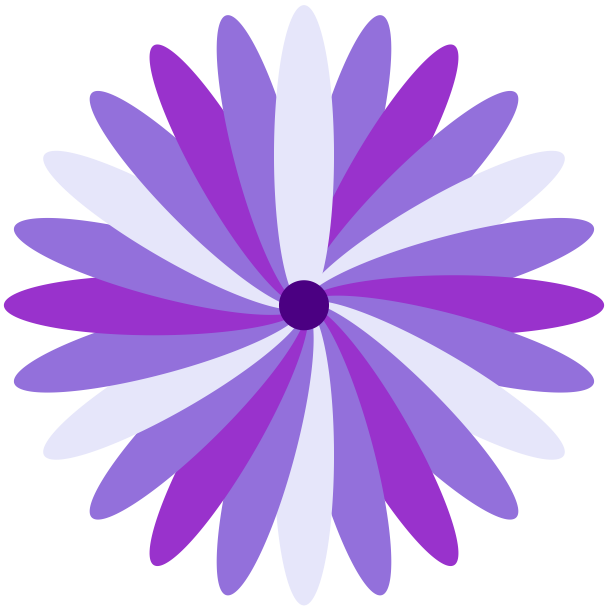


Abbildung 6.2: Blumenvorlage

Die erfoderlichen Importe und Farbdefinitionen finden Sie in der folgenden Code Zelle.

```
from pytamaro.de import *

lavender = rgb_farbe(230, 230, 250)
mediumpurple = rgb_farbe(147, 112, 219)
darkorchid = rgb_farbe(153, 50, 204)
indigo = rgb_farbe(75, 0, 130)
```

```
# Hier ist Platz für Ihre Blume
# Hinweis: die Verschiebung einer Farbe um eine Postion kann erreicht
# werden, wenn i % 4 nicht 0 sondern eine Zahl ergibt. Bei % 4 können
# dies die Zahlen 1, 2 oder 3 sein.
```

6.1.2 Musterlösung

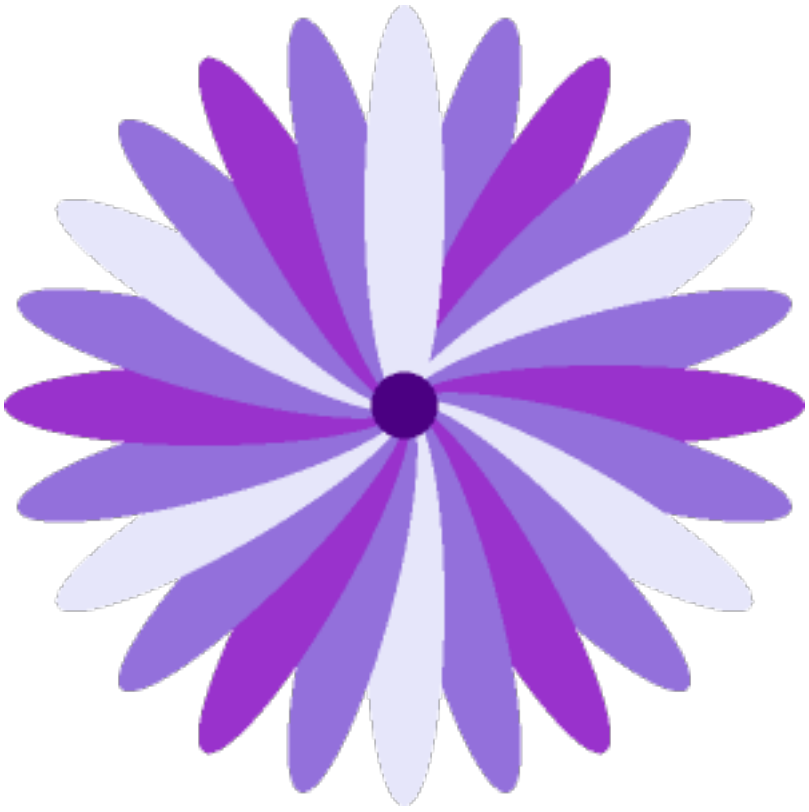
```
bluetenblatt_l = ellipse(30, 150, lavender)
bluetenblatt_m = ellipse(30, 150, mediumpurple)
bluetenblatt_d = ellipse(30, 150, darkorchid)
scheibe_i = ellipse(25, 25, indigo)

bluetenblatt_m_fix = fixiere(unten_mitte, bluetenblatt_m)
bluetenblatt_l_fix = fixiere(unten_mitte, bluetenblatt_l)
bluetenblatt_d_fix = fixiere(unten_mitte, bluetenblatt_d)

blume3 = scheibe_i
for i in range(24):

    if i % 4 == 0:
        bl_temp = drehe(i * 15, bluetenblatt_l_fix)
        blume3 = kombiniere(blume3, bl_temp)
    elif i % 4 == 2:
        bd_temp = drehe(i * 15, bluetenblatt_d_fix)
        blume3 = kombiniere(blume3, bd_temp)
    else:
        bm_temp = drehe(i * 15, bluetenblatt_m_fix)
        blume3 = kombiniere(blume3, bm_temp)

zeige_grafik(blume3)
```



7 Programmieren einfacher Funktionen

7.1 Kreisfläche

Die Fläche eines Kreises berechnet sich nach der Formel

$$A = \pi r^2$$

wobei A die Fläche π die Kreiszahl - hier mit zwei Nachkommastellen - 3.14 und r den Radius des Kreises darstellt.

Aufgabe: Implementieren Sie eine Funktion `kreisflaeche()` welche ein Argument `r` für Radius entgegennimmt und die Fläche eines Kreises zurückgibt.

```
1  def kreisflaeche():
2      # TODO: implementieren Sie die Funktion
```

7.2 Quotient

Ein Quotient kann unterschiedlich dargestellt werden. Eine Möglichkeit ist die Darstellung als Dezimalzahl.

Aufgabe: Implementieren Sie eine Funktion `quotient()` welche zwei Dezimalzahlen als Argumente entgegennimmt und den Quotienten als Dezimalzahl zurückgibt. Sehen Sie für den Sonderfall der Division durch Null eine Lösung vor.

```
1  def quotient():
2      # TODO: implementieren Sie die Funktion
```

7.3 Fakultät

Die Fakultät ist jene Funktion, welche das Produkt der natürlichen Zahlen bis n berechnet und wird als $n!$ geschrieben. Es gilt also

$$n! = 1 \cdot 2 \cdot 3 \cdot ... \cdot n$$

Wobei zu beachten ist, dass $0! = 1$ gilt.

Aufgabe: Implementieren Sie eine Funktion `fakultaet` welche als Argument $n \in \mathbb{N}$ entgegennimmt.

```
def fakultaet():
    # TODO: implementieren Sie die Funktion
```

7.4 Quadratische Gleichungen (Mitternachtsformel)

Quadratische Gleichungen der Form $ax^2 + bx + c = 0$ können mit der Formel

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

gelöst werden.
Für die Implementierung ergibt sich daraus allerdings das Problem, dass die Formel zwei Resultate ergibt und eine Python Funktion nur ein Objekt zurückgeben kann.

Als Workaround können die beiden Resultate in ein Tupel gepackt werden. Ein Tupel ist eine unveränderliche Zusammenstellung mehrer Werte. In Python werden sie in Klammern dargestellt. Die Zuweisung eines Tupels zu einer Variabel sieht dementsprechend folgendermassen aus:

```
t = (wert_1, wert_2, ..., wert_n)
```

Die detaillierten eigenschaften von Tupeln können hier ausser Acht gelassen werden. Es reicht x_1 und x_2 einem Tupel `resultat` zuzuweisen und `resultat` zurückzugeben.

Aufgabe: Implementieren Sie eine Funktion `mitternachtsformel`, welche die Argumente a, b und c entgegennimt und das Tupel x_1 und x_2 zurückgibt.

```
def mitternachtsformel():
    # TODO: implementieren Sie die Funktion
```

7.5 Musterlösungen

7.5.1 Kreisfläche

```
def kreisflaeche(r: float) -> float:
    """
    Berechnet die Fläche eines Kreises anhand des gegebenen Radius.

```

```
    Parameter
    ----------
    r : float
        Der Radius des Kreises.

    Rückgabewert
    ------------
    float
        Die Fläche des Kreises.
    """
    PI = 3.14  # Näherungswert für die Kreiszahl Pi
    a = PI * r ** 2  # Formel: Fläche = Pi * r^2
    return a
```

7.5.2 Quotient

```
def quotient(z: float, n: float) -> float:
    """
    Berechnet den Quotienten zweier Zahlen.

    Gibt eine Fehlermeldung zurück, falls der Nenner null ist.

    Parameter
    ----------
    z : float
        Der Zähler der Division.
    n : float
        Der Nenner der Division.

    Rückgabewert
    ------------
    float oder str
        Der berechnete Quotient oder eine Fehlermeldung als
↪   Zeichenkette,
        falls eine Division durch Null versucht wurde.
    """

    # Fehlermeldung für Division durch Null
    fehlermeldung = "Eine Division durch Null ist unzulässig."
    if n == 0:
        return fehlermeldung  # Fehlerfall: Division durch Null
    else:
```

```
        q = z / n  # Berechnung des Quotienten
        return q
```

7.5.3 Fakultät

```
def fakultaet(n: int) -> int:
    """
    Berechnet die Fakultät einer nicht-negativen ganzen Zahl n.

    Die Fakultät n! ist das Produkt aller positiven ganzen Zahlen
    von 1 bis n. Für n < 0 ist die Fakultät nicht definiert.

    Parameter:
        n (int): Die Zahl, deren Fakultät berechnet werden soll.

    Rückgabewert:
        int: Die Fakultät von n, oder eine Fehlermeldung als String bei
    ↪ n < 0.
    """
    if n < 0:
        # Fakultät ist für negative Zahlen nicht definiert
        return f'Die Fakultät für {n} ist nicht definiert.'
    elif n == 0:
        # Per Definition ist 0! = 1
        return 1
    else:
        prod = 1  # Initialisiere das Produkt mit 1
        for i in range(1, n + 1):
            prod *= i  # Multipliziere prod mit dem aktuellen Wert von i
        return prod  # Gib das berechnete Produkt zurück
```

7.5.4 Mitternachtsformel

```
def mitternachtsformel(a: float, b: float, c: float) -> tuple:
    """
    Löst eine quadratische Gleichung der Form ax² + bx + c = 0
    mit der Mitternachtsformel.

    Parameter
    ----------
    a : float
```

```
        Der Koeffizient vor x² (muss ungleich 0 sein).
    b : float
        Der Koeffizient vor x.
    c : float
        Der konstante Term.

    Rückgabewert
    ------------
    tuple oder str
        Ein Tupel mit den beiden Lösungen (x1, x2) oder eine Fehlermeldung
        als Zeichenkette, falls die Gleichung keine reellen Lösungen besitzt.
    """
    # Berechnung der Diskriminante: d = b² - 4ac
    d = b ** 2 - 4 * a * c
    if d < 0:
        # Keine reellen Lösungen vorhanden
        return "Diese Gleichung hat keine Lösung."
    else:
        # Berechnung der beiden Lösungen mit der Mitternachtsformel:
        # x1 = (-b + d ** (1/2)) / (2a)
        # x2 = (-b -  d ** (1/2)) / (2a)
        x1 = (-b + d ** (1/2)) / (2 * a)
        x2 = (-b - d ** (1/2)) / (2 * a)
        return (x1, x2)
```

8 Prüfungsvorbereitung

8.1 Belegung von Variabeln

Das folgende Code Snippet demonstriert die Belegung von Variabeln. Insbesondere soll es auch zeigen, was geschieht, wenn die Variabeln in Operationen einbezogen werden.

Überlegen Sie sich bevor Sie das Snippet ausführen zuerst Anweisung für Anweisung, welche Werte in den Variabeln a, b und c nach Ausführung der jeweiligen Anweisung gespeichert sind. Notieren Sie sich die Werte auf ein Blatt Papier. Anschliessend führen Sie die Zelle zur Kontrolle Ihrer Überlegungen aus.

```
# Anfangswerte zuweisen
a = 5
b = 10
c = a
print(f"Nach Initialisierung:, a = {a}, b = {b}, c = {c}")

# Wert von a ändern
a = a + 3
print(f"Nach a = a + 3: a = {a},  b = {b},  c = {c}")

# Wert von b ändern
b = a * c
print(f"Nach b = a * c: a = {a},  b = {b},  c = {c}")

# Wert von c ändern
c = b - a
print(f"Nach c = b - a: a = {a},  b = {b},  c = {c}")
```

8.2 Funktionen

Die Definiton einer neuen Funktion erweitert den Umfang der in Python zur Verfügung stehenden Befehle. Entsprechend ist es möglich, einmal definierte Funktionen in neuen Funktionen zu verwenden.

Unten finden Sie ein einfaches Beispiel für diese Vorgehensweise.

```python
def erfolgsberechnung(ertrag: float, aufwand: float) -> float:
    """Berechnet den wirtschaftlichen Erfolg als Differenz von Ertrag
    und Aufwand.

    Args:
        ertrag: Der gesamte erzielte Ertrag.
        aufwand: Der gesamte angefallene Aufwand.

    Returns:
        Der berechnete Erfolg (Ertrag minus Aufwand).
    """
    erfolg = ertrag - aufwand
    return erfolg


def renditeberechnung(ertrag: float, aufwand: float,
                      anfangskapital: float,
                      schlusskapital: float) -> float:
    """Berechnet die Rendite in Prozent basierend auf Erfolg
    und durchschnittlichem Kapital.

    Die Funktion nutzt die 'erfolgsberechnung', um den Erfolg
    zu ermitteln, und setzt diesen ins Verhältnis zum durchschnittlich
    gebundenen Kapital.

    Args:
        ertrag: Der gesamte erzielte Ertrag.
        aufwand: Der gesamte angefallene Aufwand.
        anfangskapital: Das Kapital zu Beginn der Periode.
        schlusskapital: Das Kapital am Ende der Periode.

    Returns:
        Die berechnete Rendite, ausgedrückt in Prozent (%).
    """
    erfolg = erfolgsberechnung(ertrag, aufwand)
    durchschnittskapital = (anfangskapital + schlusskapital) / 2
    rendite = erfolg / durchschnittskapital * 100
    return rendite
```

8.2.1 Berechnung des Volumens eines Zylinders

Die folgende Grafik zeigt einen Zylinder.

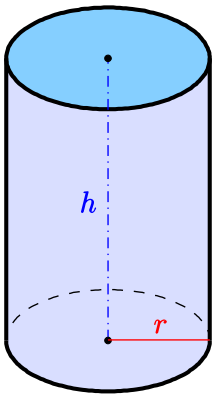


Abbildung 8.1: Zylinder

Die Formel für die Berechnung des Volumens lautet

$$v = \pi \cdot r^2 \cdot h$$

Zerlegen Sie dazu die Berechnung in zwei Teile. Definieren Sie zuerst eine Funkiton für die Berechnung der Bodenfläche und verwenden Sie diese anschliessend in einer Funktion zur Berechnung des Volumens.

Für die Kreiszahl π können Sie das Modul `math` importieren. Dies stellt Ihnen mit dem Befehl `math.pi` π in einer grossen genauigkeit zur Verfügung.

```
1  # TODO: implementieren Sie hier Ihre Funktionen
```

8.2.2 Berechnung der kinetischen Energie eines Körpers

Die kinetische Energie eines Körpers berechnet sich nach der Formel

$$E_{kin} = \frac{1}{2} \cdot m \cdot v^2$$

wobei m für die Masse des Körpers und v für dessen Geschwindigkeit steht. Implementieren Sie eine Funktion für die Berechnung der kinetischen Energie eines Körpers. Beachten Sie dabei, dass experimentell lediglich Daten über die Masse des Körpers, die zurückgelegte Strecke des Körpers sowie die dafür erforderliche Zeit zugänglich sind.

```
1  # TODO: implementieren Sie hier Ihre Funktionen
```

8.3 Musterlösungen

8.3.1 Berechnung des Volumnens eines Zylinders

```python
import math

def kreisflaeche(r : float) -> float:
    area = math.pi * r ** 2
    return area

def zylindervolumen(r : float, h : float) -> float:
    bodenflaeche = kreisflaeche(r)
    volumen = bodenflaeche * h
    return volumen
```

8.3.2 Berechnung der kinetischen Energie eines Körpers

```python
def geschwindigkeit(s: float, t: float) -> float:
    """Berechnet die Geschwindigkeit v aus zurückgelegter Strecke und
    Zeit.

    Args:
        s: Die zurückgelegte Strecke (in Metern).
        t: Die dafür benötigte Zeit (in Sekunden).

    Returns:
        Die berechnete Geschwindigkeit v (in m/s).
    """
    v = s / t
    return v


def kinetische_energie(m: float, s: float, t: float) -> float:
    """Berechnet die kinetische Energie eines Körpers basierend auf
    experimentellen Daten.

    Die Funktion berechnet zuerst die Geschwindigkeit v aus s und t
    mithilfe der Funktion 'geschwindigkeit' und wendet dann die Formel
    E_kin = 0.5 * m * v^2 an.

    Args:
        m: Die Masse des Körpers (in Kilogramm).
```

```
26          s: Die zurückgelegte Strecke (in Metern).
27          t: Die dafür benötigte Zeit (in Sekunden).
28
29      Returns:
30          Die berechnete kinetische Energie E_kin (in Joule).
31      """
32      v = geschwindigkeit(s, t)
33      e_kin = (m * v ** 2) / 2
34      return e_kin
```

9 Datenstrukturen: Listen

9.1 Daten in Listen speichern

Bisher hatten Sie Datentypen wie `int`, `float` `str` und `bool` angeschaut. Wir möchten nun mehrere Werte in einer Liste speichern.

Ich möchte zum Beispiel alle Wochentagekürzel in einer Liste speichern. Um eine Liste im Code zu speichern, muss man die Liste mit `[` beginnen und mit `]` enden. Die Elemente werden mit einem Komma `,` getrennt.

```
wochentage = ["Mo", "Di", "Mi", "Do", "Fr", "Sa", "So"]
print(wochentage)
```

9.1.1 Aufgabe: Monatsliste erstellen

Machen Sie eine Liste mit den 12 Monaten und printen Sie die Monate.

```
"""Aufgabe: Monatsliste erstellen"""

monate = # TODO
print(monate)
```

9.2 Index

Die Werte werden durchnummeriert. Die dazugehörige Nummer eines Werts nennen wir den *Index*.

Das erste Element hat den Index 0!

Um ein Element aus der Liste auszugeben können wir nach dem Variablennamen die eckigen Klammen und den Index angeben.

`wochentage[0]` ist `"Mo"`. `wochentage[1]` ist `"Di"`.

9.2.1 Aufgabe: Index lesen

Lesen Sie den Codeblock und schrieben Sie hier auf, was Sie als Output erwarten, *bevor* Sie den Codeblock ausführen!

Ich erwarte, dass das Programm folgendes ausgibt:

```
TODO: Diese Zeile ersetzen.
```

```
lieblingstag = wochentage[3]
print(f"Mein Lieblingstag ist {lieblingstag}.")
```

9.3 Überschreiben von Elementen

Wir betrachten in diesem Kapitel eine Liste mit Zahlen (`int`). Wir speichern für jeden Monat die Länge des Monats. Im Falle eines Schaltjahrs möchten wir den Februar überschreiben mit 29. Studieren Sie das Beispiel:

```
monatslaenge = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
schaltjahr = True
if schaltjahr:
    monatslaenge[1] = 29 # Der Februar hat den Index 2.
print(monatslaenge)
```

9.4 Schleifen

Listen eignen sich hervorragend um eine for-Schleife zu bauen. Betrachten Sie das Beispiel, welches den Index als `i` und den Wochentagsname als `wochentag` ausgibt. Die Liste `wochentage` möchten wir auf keinen Fall überschreiben. Also verwenden wir für die *Laufvaribale* andere Variablennamen. In diesem Fall nehmen wir `i` für den Index und `wochentag` (Singular) für den Name des Tages.

```
for i, wochentag in enumerate(wochentage):
    print(f"{i}: {wochentag}")
```

Falls Sie den Index nicht benötigen, können Sie die Schleife mit `for wochentag in wochentage:` machen.

9.4.1 Aufgabe: Monatsschleife komplettieren

Erstellen Sie eine Schleife, welche folgendes Ausgibt:

```
Der Januar hat 31 Tage.
Der Februar hat 29 Tage.
Der März hat 31 Tage.
Der April hat 30 Tage.
Der Mai hat 31 Tage.
Der Juni hat 30 Tage.
Der Juli hat 31 Tage.
Der August hat 31 Tage.
Der September hat 30 Tage.
Der Oktober hat 31 Tage.
Der November hat 30 Tage.
Der Dezember hat 31 Tage.
```

```
"""Aufgabe: Monatsschleife komplettieren"""

for ... m_name in ...:
    laenge = ... i ...
    print(f"Der {m_name} hat {laenge} Tage.")
```

9.5 Länge

Mit der Funktion `len(obj)` kann man die Länge der Liste `obj` abfragen.

```
"""Beispiel für len(obj)"""
print(f"Die Woche hat {len(wochentage)} Tage.")
```

9.6 Ergänzen und entfernen

Mit der Methode `append` kann ein Element am Ende der Liste hinzufügen und mit `pop` wieder entfernen.

```
tasks = ["einkaufen", "spazieren gehen", "staubsaugen"]
tasks.append("essen")
print(f"Neu ist auch 'essen' auf der Taskliste: {tasks}")

tasks.pop(1)
```

```
print(f"Welcher Task ist nun erledigt? {tasks}")

tasks.pop()
print("Wird kein Index angegeben, so wird das letzte Element"
      f"in der Liste entfernt: {tasks}")
```

9.7 Aufgabe: Fibonacci

Die Fibonacci Reihe lautet 1, 1, 2, 3, 5, 8, 13, 8+13, …

Die Summe der letzten zwei Zahlen ergeben die nächste Zahl.

Ergänzen Sie den Code, damit die Folge länger wird. Editieren Sie Stellen mit drei Punkten.

```
"""Aufgabe: Fibonacci"""

fibonacci: list = [1, 1, 2, 3, 5, 8, 13] # Diese Zeile nicht verändern.
for _ in range(4):
    n = ... # Die Länge von der Folge
    fibonacci ... (fibonacci ... + fibonacci ...)
    print(fibonacci)
```

9.8 Aufgabe: zeichne Chomp

Wie am Anfang der Lektion mündlich besprochen kodieren wir eine Situation von Chomp.

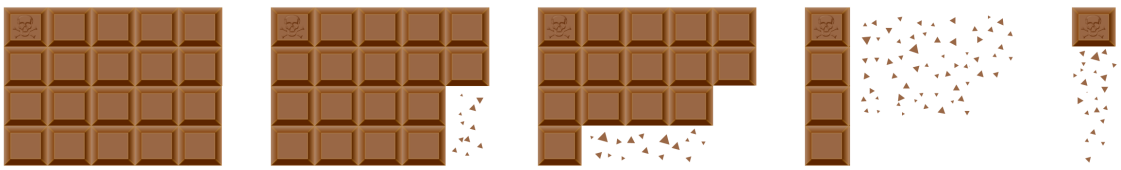


Abbildung 9.1: Chomp_gameplay

Die dritte Situation beschreiben wir als Liste `[4, 3, 3, 3, 2]`.

Schreiben Sie eine Funktion `zeichne_chomp(chomp: list) -> None` welche die Situation zeichnet.

```
"""Aufgabe: zeichne Chomp"""
from pytamaro.de import *
```

```

def zeichne_chomp(chomp: list) -> None:
    zelle = kombiniere(
        rechteck(68, 38, rgb_farbe(153, 103, 69)),
        rechteck(70, 40, rgb_farbe(51, 34, 23)),
    )
    leere_zelle = rechteck(70, 40, weiss)
    tafel = leere_grafik()
    maximum = # TODO # Groesse der erste Spalte
    for ... in chomp:
        spalte = leere_grafik()
        ...
        ...
        tafel = neben(tafel, spalte)
    zeige_grafik(tafel)
```

```
# Test für die Aufgabe
chomp_test: list = [4, 3, 3, 3, 2]
zeichne_chomp(chomp_test)
```

9.9 Aufgabe: spiele Chomp

Schreiben Sie eine Funktion `chomp_spielzug(chomp: list, spielzug:list) -> list` welche aus der ersten Situation die neue Situation berechnet. Der Spielzug ist eine Liste von der Form `[x, y]`.

Test: `chomp_spielzug([4, 3, 3, 3, 2], spielzug=[3, 2])` ergibt `[4, 3, 1, 1, 1]`

Bonusidee: Überprüfen Sie, ob der Spielzug überhaupt erlaubt ist.

```
"""Aufgabe: programmiere den Spielzug in Chomp"""

# TODO: Programmieren Sie es selber!


# Hier können Sie den Code testen:
assert chomp_spielzug([4, 3, 3, 3, 2], spielzug=[3, 2]) == [4, 3, 1, 1,
 ↪  1]
```

9.10 Übersicht

Sie wissen, wie Sie

- eine Liste erstellen.
- Elemente abfragen.
- Elemente ändern.
- mit einer Schleife durch die Liste gehen.
- die Länge einer Liste bestimmen.
- Elemente hinzufügen oder entfernen.

9.11 Zusatzstoff

Weitere Infos und Features von Listen auf https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range

Stichwörter: Slice, Spezialfall str, insert, in, join

10 Datenstrukturen: Dictionary

Dictionaries (Wörterbücher oder Maps) können mehrere Werte in einem Objekt speichern, wie es Listen auch können. Die Werte haben aber keinen Index sondern einen Schlüssel.

Ein *Dictionary* in Python ist vergleichbar mit einem echten Wörterbuch. Stellen Sie sich vor, Sie suchen die Telefonnummer (Wert) einer Person (Schlüssel). In einem Python-Dictionary könnten Sie den Namen der Person als Schlüssel und ihre Telefonnummer als Wert speichern.

```
"""Beispiel von einem dict"""
telefonbuch = {
    "Feuerwehr": 118,
    "Polizei": 117,
    "Ambulanz": 144,
    "Rega": 1414,
    "Beratung + Hilfe 147": 147,
    "allgemeiner Notruf": 112,
}
```

10.1 Operationen für Dictionaries

- Erstellen eines Dictionaries: `dict_name: dict = {}`

  Erstellt ein neues, leeres Dictionary mit dem Variablennamen *dict_name*.

- Elemente aktualisieren: `dict_name['Schlüssel'] = 'Neuer Wert'`

  Aktualisiert den Wert eines bestehenden Schlüssels oder fügt einen neuen Schlüssel mit dem angegebenen Wert hinzu, wenn er nicht existiert.

- Elemente aufrufen: `var: str = dict_name['Schlüssel']`

  `var` ist ein str mit dem Wert `'Neuer Wert'`. Man kann den Wert auch direkt zum printen oder anderes brauchen.

- Elemente löschen: `dict_name.pop('Schlüssel')`

  Entfernt das Element mit dem angegebenen Schlüssel aus dem Dictionary.

- Durch das Dictionary durchgehen: `for key, value in dict_name.items()`

  Dieser Prozess heisst *Iterieren*. Wir kennen ihn schon von der Liste oder Range. Iterieren heisst die Liste von Anfang bis Ende durchgehen. Jedes Mal wird in die Variablen `key`, `vlaue` den Schlüssel, respektive den Wert des Eintrags zugewiesen. Sie können den Variablen auch andere Namen geben.

10.1.1 Aufgabe: Vervollständigen Sie das dict

```python
"""Aufgabe: dict vervollständigen"""

voci = {
    "und": "and",
    "Auto": ... ,
    "Katze": ... ,
    "Tag": ... ,
    "Ende": ... ,
    "Familie": ... ,
    "Zuhause": ... ,
    "Name": ... ,
    "Menschen": ... ,
    "lesen": ... ,
    "Schule": ... ,
    "sprechen": ... ,
}

# Test
print(voci["Schule"])
```

10.1.2 Aufgabe: Schreiben Sie ein dict

Schreiben Sie ein `dict` für Ihre Klasse, sodass der Nachname dem Schlüssel/key und der Vorname dem Wert/value entspricht.

```python
"""Aufgabe: schreiben Sie ein dict"""
klasse = ...
```

10.1.3 Aufgabe: Schleife

Iterieren Sie durch die ganze Liste, sodass alle aus der Klasse mit `"Hallo [Vorname] [Nachname]"` begrüsst werden.

```
"""Aufgabe: Schleife"""
for nachname, vorname in ... :
    print(f"Hallo {vorname} {nachname}")
```

10.1.4 Aufgabe: Selektion

Geben Sie nur die Nachnamen aus, welche ein `e` im Nachnamen haben.

Tipp: Verwenden Sie `if 'e' in nachname:`

```
"""Aufgabe: Selektion"""

...
```

10.1.5 Aufgabe: Telefonliste

Erstellen Sie eine Liste `threedigtslist`, die nur die Namen der Notfallorganisationen aus dem dict `telefonbuch` welche genau 3 Ziffern beinhalten.

```
"""Aufgabe: Telefonliste"""

...


# Test
assert type(threedigtslist) == list
assert len(threedigtslist) == 5
for soultion in ['Feuerwehr', 'Polizei', 'Ambulanz', 'Beratung + Hilfe
    ↪ 147', 'allgemeiner Notruf']:
    assert soultion in threedigtslist
```

11 Beurteilung von Algorithmen

Die Effizienz eines Algorithmus kann grundsätzlich nach seinem Rechenaufwand oder Speicherbedarf beurteilt werden. Man spricht in diesem Zusammenhang auch von Zeit- bzw. Platzkomplexität.

Wir haben uns mit dem Zählen von Vergleichen nur mit der Rechenaufwand, also mit der Zeitkomplexität beschäftigt. Die Berechnungen werden sehr stark vereinfacht.

11.1 Bubblesort

Wenn wir mit dem Bubblesort n Objekte sortieren, haben wir $n-1$ Vergleiche im ersten Durchgang. Vereinfacht sagen wir einfach n Vergleiche. Im schlimsten Fall, müssen ist das kleinste Element am falschen Ende. Bei jedem Durchgang verschieben wir das Element nur um einen Platz. Dass heisst wir müssen etwa n Durchgänge machen. Die Laufzeit vom Bubblesort beträgt somit $n \cdot n = n^2$.

11.2 Mergesort

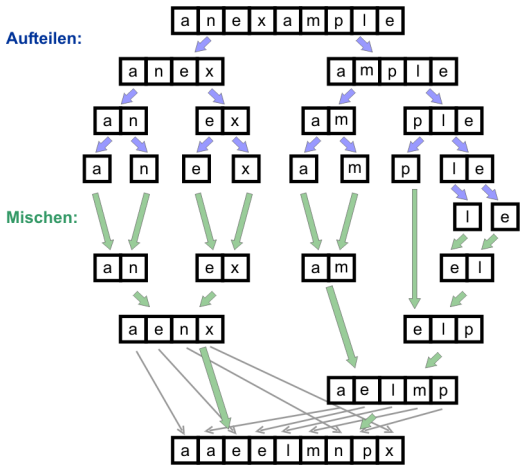


Abbildung 11.1: Mergesort_example

Bevor wir uns die Laufzeit von Mergesort anschauen, müssen wir quantifizieren, wie oft wir die Listen aufteilen müssen. Haben wir 2 Elemente, separieren wir 1 Mal. Haben wir 4 Elemente separieren wir 2 mal.

len	Teilungen
2	1
4	2
8	3
16	4
32	5
64	6
1024	10

Diese Funktion, welche der Länge zur Anzahl Teilungen zuordnet, nennt man *Logarithmus zur Basis 2*. Man notiert sie wie folgt: $\log_2(n)$

Auf jeder der $\log_2(n)$ Ebene machen wir etwa n Vergleiche. Die Laufzeit vom Mergesort beträgt somit $n \cdot \log_2(n)$.

12 Divide and Conquer

Das Prinzip *divide and conquer* (teile und herrsche) wurde am Beispiel von Mergesort gezeigt, aber nicht explizit erwähnt. Dieses Prinzip basiert auf der Idee, das kleine Probleme einfacher zu lösen sind als grosse. Deshalb versucht man, das zu lösende Problem in Teilprobleme aufzuteilen und diese, kleineren Probleme, zu lösen um anschliessend die Teillösungen zu einer Lösung für das Ursprungsproblem zusammenzusetzen.

Diese Idee wird oft durch *Rekursion* umgesetzt. Der Begriff Rekursion kann am besten am Beispiel einer Kindergeschichte erklärt werden:

> Es isch e mal en Maa gsi, de hät en hole Zah gha. I dem Zah häts es Truckli gha und i däm Truckli häts es Briefli gha. I dem Briefli isch gstande, es isch e mal en Maa gsi, de hät en hole Zah gha. I dem Zah häts es Truckli gha und id däm Truckli häts es Briefli gha…

Rekursive Funktionen sind entsprechend Funktionen, die sich selber aufrufen. Mergesort rief sich selber auf, um die Liste zu sortieren.

12.1 Fibonacci-Folge

Die Fibonacci-Folge - benannt nach dem Italienischen Mathematiker Leonardo Fibonacci - ist eine Zahlenfolge, in welcher die nächste Zahl die Summe der beiden vorangegangenen Zahlen bildet. In Zahlen sieht das folgendermassen aus:

$$1, 1, 2, 3, 5, 8, 13, ...$$

Die Fibonacci-Folge wird rekursiv deifiniert:

$$f_n = f_{n-1} + f_{n-2}, \text{ sofern } n \geq 3$$

Sie haben die Fibonacci-Folge bei den Listen bereits programmiert. Obwohl die Programmiung als Liste *viel* effizienter ist, machen wir hier eine Übung, Fibonacci als rekursive Funktion zu programmieren:

```
1  """Beispiel: fibonacci rekursiv definiert mit Printstatement"""
2  def fibonacci(n: int) -> int:
3      print(f"Es wird fibonacci({n}) ausgeführt.")
4      if n < 2:
```

```
5           return 1
6       return fibonacci(n-1) + fibonacci(n-2)
```

12.1.1 Aufgabe: Fibonacci-Folge

1. Lesen Sie die Funktion `fibonacci`. Denken Sie im Kopf durch, was passiert, wenn Sie `fibonacci(0)` oder `fibonacci(1)` ausführen. Schreiben Sie Ihre Vermutung auf und führen Sie erst dann die Funktion aus. Vergleichen Sie Ihre Vermutung mit dem Resultat.
2. Denken Sie im Kopf durch, was passiert, wenn Sie `fibonacci(2)` ausführen. Schreiben Sie Ihre Vermutung auf und führen Sie erst dann die Funktion aus. Vergleichen Sie Ihre Vermutung mit dem Resultat.
3. Bonus: Denken Sie im Kopf durch, was passiert, wenn Sie `fibonacci(3)` ausführen. Schreiben Sie Ihre Vermutung auf und führen Sie erst dann die Funktion aus. Vergleichen Sie Ihre Vermutung mit dem Resultat.

12.2 Endlose Rekursion

Wenn wir bei der Funktion `fibonacci` die Zeilen

```
1       if n < 2:
2           return 1
```

weglassen würden, so würde die Funktion endlos sich selber aufrufen, wie im Beispiel der Kindergeschichte.

Meistens wird dieses Problem mit Basisfälle gelöst.

12.3 Beispiel: Anstossen

Gestern assen wir mit 6 Personen zu Abend. Vorgestern mit 140 Personen. Wie oft schlagen die Gläser zusammen wenn alle miteinander anstossen?

Die Aufgabe können wir einfach reduzieren. Wenn ich die 6. Person am Tisch bin, stosse ich mit 5 Personen an. Die 5 Personen müssen untereinander noch anstossen. Das heisst $a_n = a_{n-1} + (n-1)$.

Für die Rekursive Funktion, brauchen wir somit nur noch die Basisfälle. In unserem Beispiel ist es: Wenn nur 0, 1 oder 2 Personen da sind, wie oft wird angestossen?

Personen	Anstossen
0	0

Personen	Anstossen
1	0
2	1

```python
"""Aufgabe: finden Sie 2 Fehler"""
def anstossen(n: int) -> int:
    ## Basisfälle
    if n <= 1:
        return 0
    if n == 2:
        return 2

    return anstossen(n-1) + (n + 1)
```

```python
"""Test"""
for i in range(10):
    print(f"{i} Personen stossen {anstossen(i)} Mal an.")

i = 140
print(f"{i} Personen stossen {anstossen(i)} Mal an.")
```

12.3.1 Bonus: Die Gausssche Summenformel

Der Deutsche Mathematiker Carl Friedich Gauss soll nach der Legende seinen Mathematiklehrer bei einer Strafaufgabe überlistet haben. Gemäss dieser Legende soll der Lehrer Gauss (und seinen Klassenkameraden) den Auftrag gegen haben, die Zahlen von 1 bis 100 zusammenzuzählen.

Gauss soll nach kurzer Zeit mit der Lösung zum Lehrer gegangen sein. Gauss' Lösung basierte auf folgender Formel:

$$1 + 2 + 3 + ... + (n-1) + n = \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Diese Formel kann direkt als Funktion implementiert werden. Das entsprechende Beispiel finden sie in der folgenden Zelle.

```python
def gauss_direct(n : int) -> int:
    return int((n*(n+1))/2)

print(f'Die Summe der Zahlen von 1 '
```

```
5         f'bis 100 ist {gauss_direct(100)}.')
```

Alternativ kann die Formel aber auch rekursiv implementiert werden. Entsprechend stellt sich die Frage, was wäre ein kleineres, aber grundsätzlich gleichartiges Problem?
Im vorliegenden Fall wäre das einfachste gleichartige Problem, die Summe aus einer Liste von Zahlen mit der Länge 1 und dem Wert 1 zu bilden. Dann ist die Summe auch 1. Als Formel könnte man das so schreiben:

$$\sum_{k=1}^{n} k = 1, \text{ sofern } n = 1$$

Daraus ergibt sich die Frage, wie man von $n = 100$ zu $n = 1$ kommt.

Für alle Fälle, in denen $n > 1$ ist, gilt

$$\sum_{k=1}^{n} k = n + \sum_{k=1}^{n-1} k$$

In dieser Formel kann nun immer wieder $n - 1$ eingesetzt werden. Das ist die Rekursion.

Zusammengefasst kann dies folgendermassen dargestellt werden:

$$\sum_{k=1}^{n} = \begin{cases} 1, & n = 1 \quad \text{Basisfall} \\ n + \sum_{k=1}^{n-1} k, & \forall n > 1 \quad \text{Rekursionsfall} \end{cases}$$

Der Basisfall ist einerseits der einfachste Fall und andererseits auch der letzte Fall, der bearbeitet werden muss.
Beides ist wichtig. Dass der Basisfall der einfachste Fall ist, hilft das Problem zu lösen. Dass der Basisfall der Lezte Fall ist, der bearbeitet werden muss, ermöglicht es, die Rekursion zu beenden.

Im Rekursionsfall steht "$\forall n > 1$". Das $\forall$ Zeichen ist der sogenannte Allquantor und bedeutet hier "**für alle** n grösser als 1.

Das eine Problemlösung rekursiv implementiert wird, bedeutet, dass die Funktion sich selber aufruft.
In der folgenden Zelle wird die gausssche Summenformel gemäss obiger Darstellung rekursiv implementiert.

```
1  """Bonusaufgabe: Gauss Rekursiv"""
2  def gauss_recursive(n : int) -> int:
3      # TODO: rekursive Implementation des kleinen Gauss
4      pass
5
6  print(f'Die Summe der Zahlen von 1 '
```

```
7         f'bis 100 ist {gauss_recursive(100)}.')
```

12.4 Fakultät

Mit Fakultät ($n!$) wird in der Mathematik jene Funktion bezeichnet mit der alle natürlichen Zahlen $\leq n$ miteinander multipliziert werden.

$$n! = 1 \cdot 2 \cdot ... \cdot (n-1) \cdot n = \prod_{k=1}^{n} k$$

Als Besonderheit muss mann wissen, dass $0! = 1$ gilt.

In Python kann dies mit einer `for` Schleife berechnet werden. In der folgenden Zelle wird eine entsprechende Funktion definiert.

```
1  def factorial_loop(n : int) -> int:
2      """result wird gleich 1 gesetzt,
3      weil 1 das neutrale Element der
4      Multiplikation ist """
5      result = 1
6      for i in range(1, n+1):
7          result *= i
8
9      return result
10
11 print(f'Das Resultat von 5! ist '
12       f'{factorial_loop(5)}.')
```

Die Funktion kann allerdings auch rekursiv definiert werden. Die Definition sieht dann folgendermassen aus:

$$n! = \begin{cases} 1, & n = 0 \vee n = 1 \quad \text{Basisfall} \\ n \times (n-1)!, & \forall n > 1 \quad \text{Rekursionsfall} \end{cases}$$

Entsprechend kann eine Funktion zur Berechnung von $n!$ auch rekursiv implementiert werden.

```
1  """Aufgabe: Fakultät"""
2  def factorial_recursive(n : int) -> int:
3      # TODO: rekursive Implementation der Fakultätsfunktion
4      pass
5
```

```
print(f'Das Resultat von 5! ist '
      f'{factorial_recursive(5)}.')
```

12.5 Wörter umdrehen

Gesucht ist eine rekursive Funktion, welche ein Wort rückwärts schreibt (bsp. Kantonsschule nach eluhcssnotnaK).

```
"""Aufgabe: umdrehen"""
def reverse(s : str) -> str:
    if len(s) == 0 or len(s) == 1:
        return ...
    else:
        head = s[0]
        tail = s[1:]
        return ...
```

```
print(reverse('Kantonsschule'))
```

12.6 Bonus: Anzahl Möglichkeiten zu Zahlen

Ich Verkaufe eine Heft für 6 CHF. Wie viele Möglichkeiten gibt es 5 CHF in Münzen (wir betrachten nur die Münzen 1, 2, 5 CHF).

- $5 + 1$
- $2 + 2 + 2$
- $2 + 2 + 1 + 1$
- $2 + 1 + 1 + 1 + 1$

Es gibt 4 Möglichkeiten.

Um die Frage zu beantworten, kann man die Frage auch aufteilen. Die grösste mögliche Münze ist 5. Also gibt es die Möglichkeiten mit einem 5 Fränkler zu bezahlen (1 Möglichkeit) oder ohne keinem 5 Fränkler zu bezahlen (3 Möglichkeiten). Diese Anzahl Möglichkeiten zusammen addiert ergibt die Möglichkeiten. Die Funktion benötigt 2 Argumente: Den zu bezahlenden Betrag und die grösste erlaubte Münze.

```
"""
Bonusaufgabe
```

```
Studieren Sie die Funktion und füllen Sie die Lücken mit den
↪ Auslassungspunkten.
"""
MUENZEN = [1000, 200, 100, 50, 20, 10, 5, 2, 1]
def anz_zahlmöglichkeiten(
    value: int,
    largest_index: int=0,
    ) -> int:
    """Rekursive Funktion"""
    if value < 0:
        return 0
    if value == 0:
        return 1
    if largest_index + 1 == len(MUENZEN):
        # Pay with smales coins
        return 1
    # Pay with largest Coin
    ret = ...

    # Pay without larges coin
    ret += ...
    return ret

print(anz_zahlmöglichkeiten(5))
```

```
print(anz_zahlmöglichkeiten(500))  # 6295434
# print(anz_zahlmöglichkeiten(700))  # 40208370
# print(anz_zahlmöglichkeiten(1000)) # 321335887
```

```
"""
Bonusaufgabe
Printen Sie die Möglichkeiten!

[5]
[2, 2, 1]
[2, 1, 1, 1]
[1, 1, 1, 1, 1]
"""
MUENZEN = [1000, 200, 100, 50, 20, 10, 5, 2, 1]
def print_zahlmöglichkeiten(...)
    ...

print_zahlmöglichkeiten(5)
```

13 Prüfungsvorbereitung 2

13.1 Lernziele

Ich erwarte, dass Sie in der Lage sind

- eine Jupyter Arbeitsumgebung zu starten (auswendig);
- Python Listen zu erstellen und zu verändern;
- Werte aus Python Listen direkt oder mit Hilfe von `for` Schleifen zu verarbeiten;
- Python Dictionaries zu erstellen und zu verändern;
- Wert aus Python Dictionaries direkt oder mit Hilfe von `for` Schleifen zu verarbeiten sowie
- rekursive Funktionen zu implementieren.

Die Inhalte der letzten Prüfung werden als Grundlagen vorausgesetzt. Die Prüfung findet mit Papier und Bleistift statt. Als Hilfsmittel dürfen Sie eine handschriftliche Zusammenfassung im Umfang einer Seite A4 mitbringen. Die Zusammenfassung darf alles ausser die Schritte zum Start einer Jupyter Arbeitsumgebung enthalten.

13.2 Listen

13.2.1 Listen erstellen

Das Fähnlein Fieselschweif besteht aus Donald Ducks Neffen Tick, Trick und Track. Erstellen Sie eine Liste `fieselschweif` mit den drei Mitgliedern.

```
1  # TODO: Liste erstellen
```

13.2.2 Listen bearbeiten

Ergänzen Sie die vorher erstellte Liste 'fieselschweif' um den Namen *Dagobert*.

```
1  # TODO: fieselschweif ergänzen
```

Dagobert gefällt es nicht im Fähnlein Fieselschweif. Entfernen Sie Ihn wieder und lassen Sie Ihren Code den Text *Dagobert hat das Fähnlein Fieselschweif wieder verlassen.* ausgeben. Dagobert soll dabei als Variabel `quitter` in den Code eingefügt werden.

```
# TODO: implementieren Sie die obige Vorgabe
```

13.2.3 Über Listen iterieren

Gegeben sei eine Liste `numbers`. Die Liste beinhaltet die natürlichen Zahlen von 1 bis 10 (`numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`). Erstellen Sie mit Hilfe einer `for` schleife basierend auf der Liste `numbers` eine neue Liste `triplet` welche aus den Elementen der Dreierreihe des kleinen Einmaleins besteht.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# TODO: implementieren Sie die obige Vorgabe
```

13.2.4 Python Dictionary erstellen

Erstellen Sie ein Python Dictionary `mk`, welches die Geschäftsadresse von Jacques Mock Schindler enthält. Die ersten zwei Einträge erhalten Sie als Vorlage.

```
mk = {
    'name': 'Mock Schindler',
    'first_name': 'Jacques',
    # TODO: kompletieren Sie das Dictionary
}
```

13.2.5 Über ein Python Dictionary iterieren

Geben Sie den Inhalt des Dictionary `mk` mit Hilfe einer `for` Schleife aus. Dabei soll die Ausgabe jedes Eintrags auf einer neuen Zeile erfolgen.

```
# TODO: implementieren Sie obige Vorgabe
```

13.2.6 Rekursion

Ein Palindrom ist ein Wort, das in beide Richtungen gelesen werden kann. Ein Beispiel für ein Palindrom ist *Sugus*. Schreiben Sie eine rekursive Funktion `palindrom_tester`, welche testet, ob eine gegebene Zeichenfolge ein Palindrom ist.

Hinweis: Die einzelnen Zeichen eines Strings können wie die Elemente einer Liste mit Hilfe eines Index abgerufen werden. Das erste Zeichen eines Strings hat dabei den Index 0.

Damit Sie einfacher prüfen können, ob Ihre Funktion korrekt arbeitet, übergeben Sie ihr ausschliesslich Strings in Kleinbuchstaben.

```
# TODO: implementieren Sie die obige Vorgabe
```

13.3 Musterlösungen

13.3.1 Listen erstellen

Das Fähnlein Fieselschweif besteht aus Donald Ducks Neffen Tick, Trick und Track. Erstellen Sie eine Liste `fieselschweif` mit den drei Mitgliedern.

```
fieselschweif = ['Tick', 'Trick', 'Track']
```

13.3.2 Listen bearbeiten

Ergänzen Sie die vorher erstellte Liste 'fieselschweif' um den Namen *Dagobert*.

```
fieselschweif.append('Dagobert')
```

Dagobert gefällt es nicht im Fähnlein Fieselschweif. Entfernen Sie Ihn wieder und lassen Sie Ihren Code den Text *Dagobert hat das Fähnlein Fieselschweif wieder verlassen.* ausgeben. Dagobert soll dabei als Variabel `quitter` in den Code eingefügt werden.

```
quitter = fieselschweif.pop()
print(f'{quitter} hat das Fähnlein Fieslschweif wieder verlassen.')
```

13.3.3 Über Listen iterieren

Gegeben sei eine Liste `numbers`. Die Liste beinhaltet die natürlichen Zahlen von 1 bis 10 (`numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`). Erstellen Sie mit Hilfe einer `for` schleife basierend auf der Liste `numbers` eine neue Liste `triplet` welche aus den Elementen der Dreierreihe des kleinen Einmaleins besteht.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

triplet = []

for number in numbers:
```

```
        triplet.append(number * 3)
```

13.3.4 Python Dictionary erstellen

Erstellen Sie ein Python Dictionary `mk`, welches die Geschäftsadresse von Jacques Mock Schindler enthält. Die ersten zwei Einträge erhalten Sie als Vorlage.

```
mk = {
    'name': 'Mock Schindler',
    'first_name': 'Jacques',
    'street': 'Rosenstrasse 1',
    'postcode': 8400,
    'town': 'Winterthur',
}
```

13.3.5 Über ein Python Dictionary iterieren

Geben Sie den Inhalt des Dictionary `mk` mit Hilfe einer `for` Schleife aus. Dabei soll die Ausgabe jedes Eintrags auf einer neuen Zeile erfolgen.

```
for key, value in mk.items():
    print(f'{key}: {value}')
```

13.3.6 Rekursion

Ein Palindrom ist ein Wort, das in beide Richtungen gelesen werden kann. Ein Beispiel für ein Palindrom ist *Sugus*. Schreiben Sie eine rekursive Funktion `palindrom_tester`, welche testet, ob eine gegebene Zeichenfolge ein Palindrom ist.

Hinweis: Die einzelnen Zeichen eines Strings können wie die Elemente einer Liste mit Hilfe eines Index abgerufen werden. Das erste Zeichen eines Strings hat dabei den Index 0.

Damit Sie einfacher prüfen können, ob Ihre Funktion korrekt arbeitet, übergeben Sie ihr ausschliesslich Strings in Kleinbuchstaben.

```
def palindrom_tester(text):
    """
    Prüft rekursiv, ob ein String ein Palindrom ist
    (vorwärts und rückwärts gelesen gleich).
    Beispiele: "Lagerregal", "Anna", "Reittier"
    """
```

```
    # Vorverarbeitung: Kleinbuchstaben für einfacheren Vergleich
    # (für den Fall, dass Sie sich nicht an den Tipp gehalten haben)
    text = text.lower()

    # Rekursionsanker 1: Leerer String oder Einzelbuchstabe
    # ist immer ein Palindrom.
    if len(text) <= 1:
        return True

    # Rekursionsanker 2: Wenn erster und letzter Buchstabe
    # nicht übereinstimmen,
    # kann es kein Palindrom sein. Abbruch mit False.
    if text[0] != text[-1]:
        return False

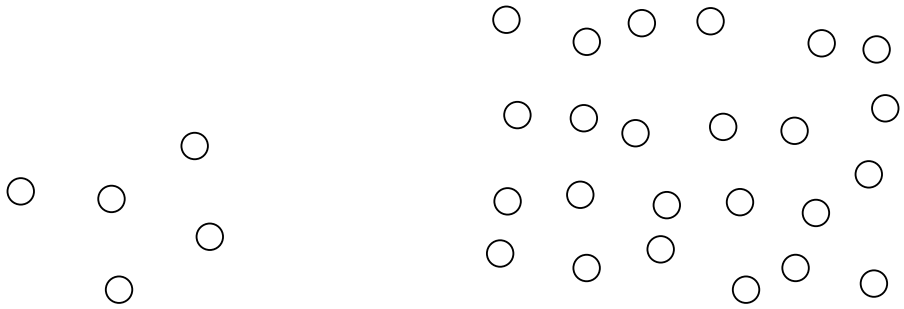
    # Rekursionsschritt:
    # Wenn die Ränder passen, prüfe den Teilstring ohne den
    # ersten und letzten Buchstaben.
    return palindrom_tester(text[1:-1])
```

Teil III

Data Representation

14 Number Representation

Counting has a long history, as does the representation of numbers. At the beginning of numerical recording, there was a one-to-one representation. This means that each each object to be counted was represented by a symbol. This approach is not particularly efficient. Therefore, early in human history people started developing more compact representations of numbers.



(a) Five (b) Twenty-four

Abbildung 14.1: Objects to count

14.1 One-to-one Number Representation

In a one-to-one number representations the objects in Abbildung 14.1a could be represented by 𝍸 the objects in Abbildung 14.1b by 𝍸 𝍸 𝍸 𝍸 𝍷 respectively.

14.2 Symbolic Number Representation

While the Romans were not the pioneers of representing numerical values through symbols, their system remains the most prominent historical example. Consequently, the Roman method for denoting the quantity of objects is illustrated in figures Abbildung 14.1a (**V**) and Abbildung 14.1b (**XXIV**).

Roman numerals are generally intuitive up to the number twelve; however, representing larger values increases complexity significantly. In principle, each symbol within a Roman numeral possesses a fixed value. An exception occurs only when a symbol of lesser value precedes one of greater value, indicating subtraction.

Furthermore, Roman numerals require considerable space. The current year becomes *MMXXVI* an increase in space consumption by 50% compared to the decimal *2026*.

14.3 Number Representation and Positional Notation

14.3.1 Decimal Representation of Numbers

Unlike Roman numerals, decimal numerals possess both a **face value** and a **positional value**. For instance, the quantity of objects shown in Abbildung 14.1a is represented by the digit *5*, signifying *five ones*. In Abbildung 14.1b, the quantity becomes *24*, representing *two tens plus four ones*.

When expressed as a formal calculation:

$$5 = 5 \times 10^0 \quad \text{and} \quad 24 = 2 \times 10^1 + 4 \times 10^0$$

This distinction demonstrates the mechanics of a **positional numeral system**:

- The **face value** (the digit itself) acts as a coefficient.
- The **positional value** (determined by the digit's position) corresponds to a power of the base.

This differentiation between face and positional values enables a significantly more compact representation of numbers compared to additive symbolic systems such as Roman numerals.

14.3.2 Binary Representation of Numbers

A well-known joke in computer science circles goes:

> There are 10 kinds of people:
>
> - those who understand binary numbers, and
> - those who don't.

Understanding binary numbers is essential to appreciating this joke.

While decimal numbers are represented in formal calculations as products of single digits and powers of the base 10, binary numbers use base 2. The quantities shown in Abbildung 14.1a and Abbildung 14.1b, when represented as binary numbers, are 101_2 and 11000_2, respectively.

Expressed as formal calculations:

$$5_{10} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 101_2$$

and

$$24_{10} = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 11000_2$$

One position in a binary number is called a *bit* (short for *binary digit*). Eight bits compose a *byte*, which is the smallest addressable unit of memory in most computer systems. For example, the ASCII code for the letter 'A' is stored in one byte: 01000001_2.

14.3.3 Hexadecimal Numbers

Hexadecimal numbers use base 16_{10}. Therefore, the quantities shown in Abbildung 14.1a and Abbildung 14.1b become 5_{16} and 18_{16}, respectively.

Because the Arabic numerals used in the decimal system provide only ten digits (0–9), additional symbols are needed to represent hexadecimal numbers. The symbols chosen are the first six uppercase letters of the Latin alphabet. Thus, hexadecimal numbers are composed of the sixteen symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

These numbers expressed as a formal calculation:

$$5_{10} = 5 \times 16^0 = 5_{16}$$

$$24_{10} = 1 \times 16^1 + 8 \times 16^0 = 18_{16}$$

Each hexadecimal digit represents exactly four bits (also called a nibble). Hexadecimal numbers are widely used in computer science as the follwing examples demonstrate:

- Memory addresses: `0x7FFFFFFF`
- RGB color codes: `#FF5733` (red: FF, green: 57, blue: 33)
- Binary-to-hex conversion: Each hex digit represents exactly four bits (e.g., $\mathrm{F}_{16} = 1111_2$).

14.4 Number Conversion

The following sections describe the algorithms for the conversion of numbers from one representation to the other.

14.4.1 Decimal to Binary Numbers

First, an empty string is initialized to represent the binary number. In a loop, the number n to be converted is processed using the modulo 2 operation. The result of this operation is prepended as the most significant bit (to the far left) of the existing binary string. Subsequently, the number n is halved using integer division, and the result of this division becomes the new value of n. This process is repeated until n reaches zero.

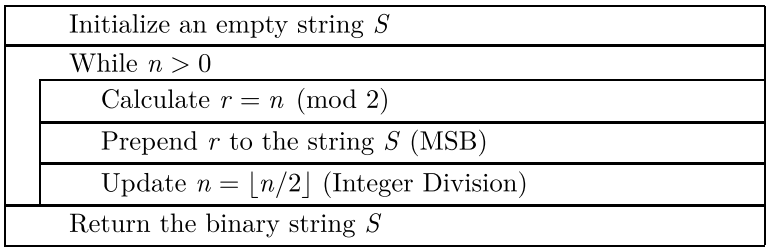


Abbildung 14.2: Nassi-Shneiderman diagram describing the algorithm (MSB = Most Significant Bit).

Task: Please implement the algorithm as described.

```python
def decimal_to_binary(n: int) -> str:
    # TODO: implement the aforementioned algorithm
    pass
```

14.4.2 Binary to Decimal Conversion

To convert binary numbers to decimal numbers, the formal calculation described above must be expressed algorithmically.

1. Initialize a counter e to zero.
2. Initialize a result variable r to zero.
3. Extract the rightmost digit d from the binary number.
4. Multiply d by 2^e and add the result to r.
5. Increment the counter e by one.
6. Repeat steps 3–5 for each remaining digit, moving from right to left, until all digits have been processed.

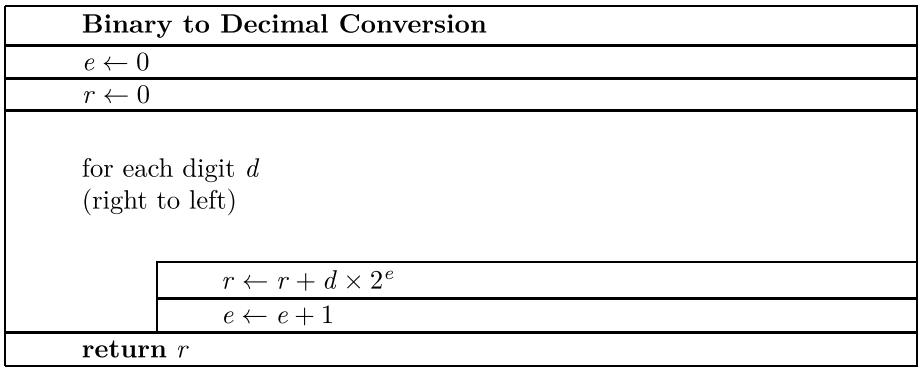


Abbildung 14.3: Nassi-Shneiderman diagram for the algorithm to convert binary numbers to decimal ones.

Task: Please implement the algorithm as described.

```
def binary_to_decimal(b: str) -> int:
    # TODO: implement the aforementioned algorithm
    pass
```

14.4.3 Decimal to Hexadecimal Conversion

To convert decimal numbers to hexadecimal numbers, the algorithm can be based on the binary conversion algorithm described earlier. However, hexadecimal representation requires additional symbols beyond 0–9. A lookup table maps decimal values to their hexadecimal equivalents, as shown in Tabelle 14.1.

Tabelle 14.1: Conversion table: decimal to hexadecimal.

Dec	Hex	Dec	Hex
0	0	8	8
1	1	9	9
2	2	10	A
3	3	11	B
4	4	12	C
5	5	13	D
6	6	14	E
7	7	15	F

Using this table, the conversion algorithm proceeds as follows:

1. Initialize an empty string to represent the hexadecimal number.
2. While $n > 0$:
    a. Compute $d = n \bmod 16$.
    b. Look up the hexadecimal digit corresponding to d in Tabelle 14.1.
    c. Prepend this digit to the hexadecimal string.
    d. Update n using integer division: $n \leftarrow \lfloor n/16 \rfloor$.
3. Return the hexadecimal string.

Decimal to Hexadecimal Conversion	
hex_string ← “” (empty string)	
$n > 0$	$d \leftarrow n \bmod 16$
	digit ← lookup(d) in table
	hex_string ← digit + hex_string
	$n \leftarrow \lfloor n/16 \rfloor$
return hex_string	

Abbildung 14.4: Nassi-Shneiderman diagram of the algorithm for the decimal to hexadecimal conversion.

Task: Please implement the algorithm as described.

```python
def decimal_to_hexadecimal(d: int) -> str:
    # TODO: implement the aforementioned algorithm
    pass
```

14.4.4 Hexadecimal to Decimal Conversion

As with the conversion from decimal to hexadecimal, the reverse conversion requires only a small adaptation of the binary-to- decimal algorithm.

The formal calculation must account for the hexadecimal digit mapping shown in Tabelle 14.1.

1. Initialize a counter e to zero.
2. Initialize a result variable r to zero.
3. Extract the rightmost digit d from the hexadecimal number.
4. Convert the digit d to its decimal equivalent using Tabelle 14.1.
5. Multiply the decimal value by 16^e and add the result to r.
6. Increment the counter e by one.
7. Repeat steps 3–6 for each remaining digit, moving from right to left, until all digits have been processed.

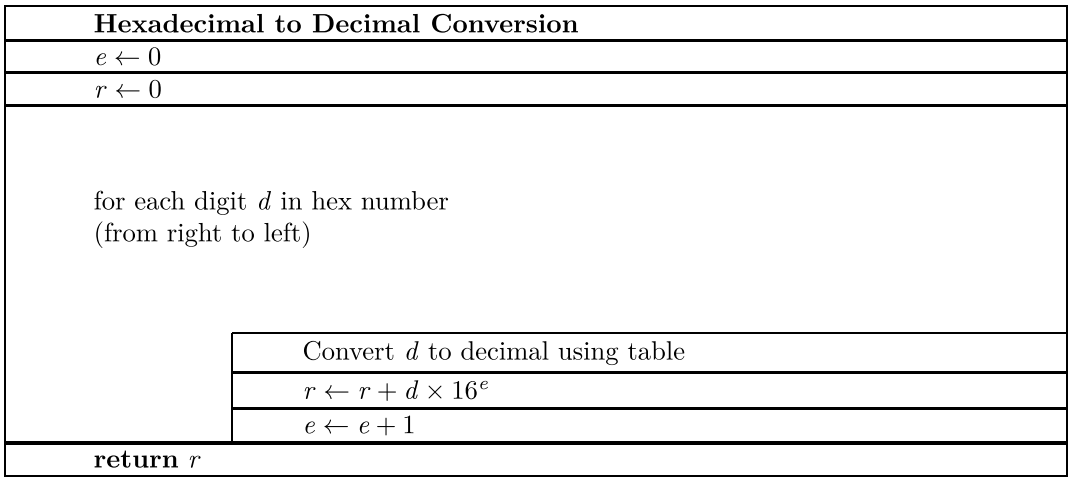


Abbildung 14.5: Nassi-Shneiderman diagram of the algorithm for the hexadecimal to decimal conversion.

Task: Please implement the algorithm as described.

```python
def hexadecimal_to_decimal(h: str) -> int:
    # TODO: implement the aforementioned algorithm
    pass
```

14.4.5 Decimal to Roman Numeral Conversion

To convert decimal numbers to Roman numerals, a greedy algorithm is employed that repeatedly subtracts the largest possible Roman value from the number.

Roman numerals use seven basic symbols and six subtractive combinations, as shown in Tabelle 14.2.

Tabelle 14.2: Roman numeral values in descending order.

Decimal	Roman	Type
1000	M	Basic
900	CM	Subtractive
500	D	Basic
400	CD	Subtractive
100	C	Basic
90	XC	Subtractive
50	L	Basic
40	XL	Subtractive
10	X	Basic
9	IX	Subtractive

Decimal	Roman	Type
5	V	Basic
4	IV	Subtractive
1	I	Basic

Using this table, the conversion algorithm proceeds as follows:

1. Initialize an empty string to represent the Roman numeral.
2. Initialize an index i to point to the first entry in Tabelle 14.2 (largest value).
3. While the decimal number $n > 0$:
   a. Let v be the decimal value at index i.
   b. Let s be the Roman symbol(s) at index i.
   c. If $n \geq v$:
      - Append symbol s to the Roman numeral string.
      - Subtract v from n: $n \leftarrow n - v$.
   d. Otherwise:
      - Increment index i to move to the next smaller value.
4. Return the Roman numeral string.

Example: Convert 24_{10} to Roman numerals.

Step	n	i	v	s	Action	Result
1	24	0	1000	M	Skip	“ ”
2	24	1	900	CM	Skip	“ ”
3	24	2	500	D	Skip	“ ”
4	24	3	400	CD	Skip	“ ”
5	24	4	100	C	Skip	“ ”
6	24	5	90	XC	Skip	“ ”
7	24	6	50	L	Skip	“ ”
8	24	7	40	XL	Skip	“ ”
9	24	8	10	X	Append	“X”
10	14	8	10	X	Append	“XX”
11	4	8	10	X	Skip	“XX”
12	4	9	9	IX	Skip	“XX”
13	4	10	5	V	Skip	“XX”
14	4	11	4	IV	Append	“XXIV”
15	0	—	—	—	Done	“XXIV”

Result: $24_{10} = \text{XXIV}$

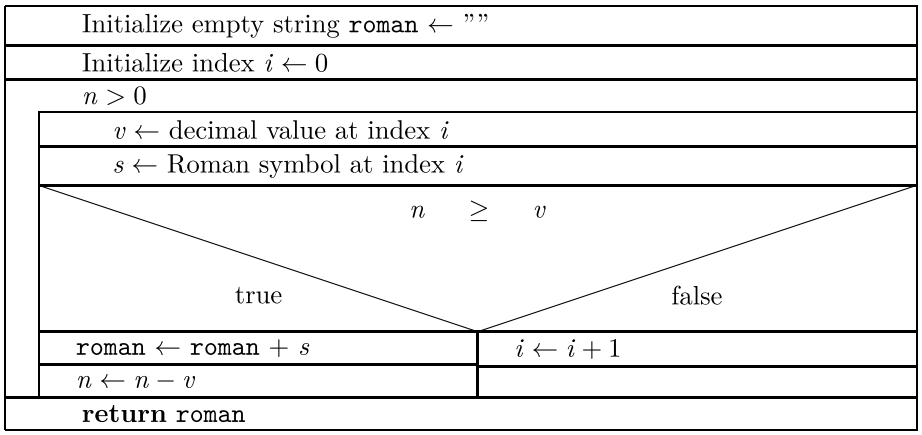


Abbildung 14.6: Nassi-Shneiderman diagram of the algorithm for decimal to Roman numeral conversion.

Task: Please implement the algorithm as described.

```
1  def decimal_to_roman(d: int) -> str:
2      # TODO: implement the aforementioned algorithm
3      pass
```

15 Image Representation in Computers

Images are composed of shapes and colours. We have observed this through our work with the PyTamaro library. However, this experience does not answer the question of how shapes and colours are represented in computer systems. While humans perceive colours visually, computers must encode them as numerical data.

Let us begin with the colour representation.

15.1 Colour Representation

To display a colour on a computer screen two aspects must be considered:

1. The colour model, and
2. the colour encoding.

15.1.1 The Colour Model

At a theoretical level, three distinct colour models exist: light-based additive colour models (RGB), pigment-based subtractive colour models (CMYK), and prerceive oriented models (HSL/HSV).

For computer screens, the RGB model is predominant whilst for printers we use the CMYK model. The HSL/HSV model is used in image processing software. Because this is a computer science class, we will focus on the RGB model. The abbreviation RGB stands for red, green, and blue. If the beams of a red, a green, and a blue torch are directed at one spot, the colour of that spot will be white where the three beams overlap, as illustrated in Abbildung 15.1.

IIn computer screens, the torches are replaced by a grid of red, green, and blue LEDs. A group of three LEDs (one red, one green, one blue) forms a single pixel on the screen. Colours are created by varying the intensity of light emitted by each LED in the pixel group.

A typical computer screen LED can emit light at 256 different intensity levels, ranging from 0 (off) to 255 (maximum brightness). Since each pixel contains three LEDs, and each LED has 256 possible intensity levels, the total number of displayable colours is:

$$256 \times 256 \times 256 = 256^3 = 16{,}777{,}216$$

This is called **8-bit colour depth** (or **24-bit true colour**) because each of the three colour channels (red, green, blue) uses 8 bits to represent its 256 intensity levels. The term “24-bit” comes from the total: 8 bits $\times$ 3 channels = 24 bits per pixel.

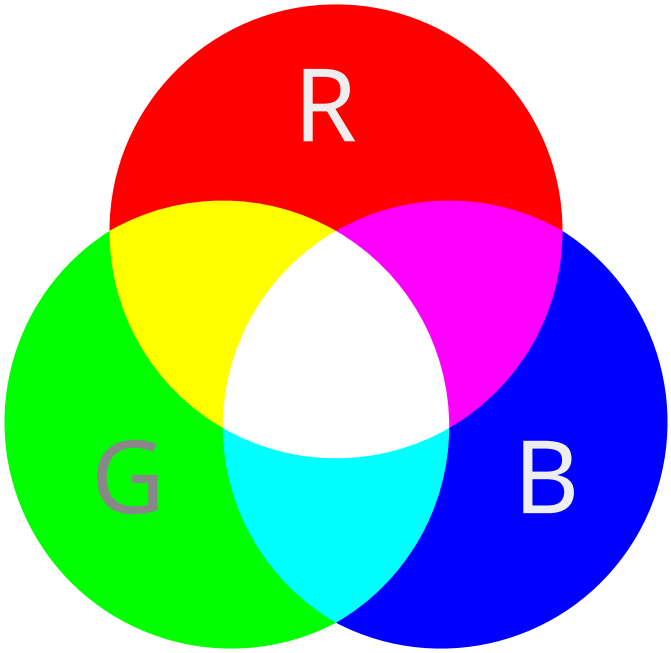


Abbildung 15.1: Additive RGB colour mixing

15.1.2 Colour Encoding

The above explanation leads to the question of how the numerical colour representation is encoded in computer systems.

RGB colour values can be represented in two common notations:

1. **Decimal RGB notation:** Three integers ranging from 0 to 255. For example, RGB(10, 255, 100) represents a vivid green.
2. **Hexadecimal notation:** A six-digit hexadecimal number preceded by `#`. For example, `#0AFF64` represents the same vivid green.

In hexadecimal notation, each pair of hexadecimal digits represents one colour channel. The first two digits represent the red channel, the next two the green channel, and the last two the blue channel:

- `#0AFF64` breaks down as:
  - `0A` = 10 (Red)
  - `FF` = 255 (Green)
  - `64` = 100 (Blue)

This hexadecimal notation is preferred in web development and graphic design because it is compact and aligns with how colours are stored in computer memory (8 bits = 2 hexadecimal digits per channel).

15.2 Colour Table

```
from pytamaro import *

# TODO: Create an 8 × 8 RGB colour grid with PyTamaro
#
# Requirements:
# - Create a grid of coloured rectangles (8 rows, 8 columns)
# - Vary the red and green values across the grid
# - Keep blue at 0 for simplicity
# - Each cell should be 40×40 pixels
#
# Hints:
# - Use rgb_color(r, g, b) to create colours
# - Use rectangle(width, height, color) to create rectangles
# - Use beside() to place rectangles horizontally
# - Use above() to stack rows vertically
# - Calculate RGB values using: value = row * step or col * step
#   where step = 255 // 7 (to get 8 evenly-spaced values from 0 to 255)
```

16 The Concept of an Algorithm

In everyday life we constantly follow step-by-step procedures: cooking recipes, assembly instructions, or tying shoe laces. In computer science we formalise this idea with the concept of an **algorithm**.

16.1 Definition

An **algorithm** is a finite sequence of well-defined, unambiguous instructions that, given some input, produces an output in a finite number of steps.

16.2 Properties of an Algorithm

For a set of instructions to qualify as an algorithm, it must satisfy the following properties:

Property	Meaning
Finiteness	The algorithm terminates after a finite number of steps.
Determinism	Each step has exactly one well-defined successor.
Definiteness	Every instruction is precise and unambiguous.
Input	The algorithm receives zero or more inputs.
Output	The algorithm produces at least one result.
Effectiveness	Every step can actually be carried out (is computable).

16.3 Example: Tying Shoe Laces

An everyday example of an algorithm is tying shoe laces.

- **Input:** An untied shoe with two loose laces.
- **Output:** A tied shoe with a secure bow.
- **Finiteness:** The process ends once the bow is formed.
- **Definiteness:** Each step (cross laces, form loop, wrap around, pull through) can be described precisely.

- **Determinism:** Given the same starting position, the same steps always produce the same knot.
- **Effectiveness:** Every step involves a physical action that can be performed.

When you wrote instructions for your partner to tie a shoe, you experienced first-hand how important *definiteness* is: any vague step ("make a knot") leads to confusion.

16.4 From Everyday Algorithms to Mathematical Algorithms

Algorithms are not limited to physical tasks. Many important algorithms operate on numbers. One of the oldest known algorithms is the **Euclidean algorithm** for computing the **Greatest Common Divisor (GCD)** of two integers.

The GCD of two integers is the largest positive integer that divides both numbers without a remainder.

Examples: - gcd(12, 8) = 4 (because 4 divides both 12 and 8) - gcd(48, 18) = 6 (because 6 divides both 48 and 18) - gcd(56, 15) = 1 (the numbers share no common factor other than 1)

16.5 The Euclidean Algorithm

The Euclidean algorithm computes the GCD by repeatedly replacing the larger number with the *remainder* of dividing the two numbers. The process continues until the remainder is zero. At that point, the other number is the GCD.

16.5.1 The Algorithm in Natural Language

1. Given two positive integers a and b.
2. Compute the remainder $r = a \bmod b$.
3. Replace a with b and b with r.
4. If $b = 0$, then a is the GCD. **Stop.**
5. Otherwise, go back to step 2.

16.5.2 Checking the Algorithm Properties

- **Input:** Two positive integers a and b.
- **Output:** Their greatest common divisor.
- **Finiteness:** The remainder strictly decreases with each step, so the algorithm always terminates.
- **Determinism:** Each step is uniquely determined (compute remainder, swap values).
- **Definiteness:** "Divide a by b, take the remainder" is unambiguous.
- **Effectiveness:** Integer division and remainder are basic computable operations.

16.5.3 Worked Example: gcd(48, 18)

Step	a	b	b * q + r	r
1	48	18	18 * 2 + 12	12
2	18	12	12 * 1 + 6	6
3	12	6	6 * 2 + 0	0

After step 3, $b = 0$, so the GCD is $a = \mathbf{6}$.

16.5.4 Worked Example: gcd(56, 15)

Step	a	b	b * q + r	r
1	56	15	15 * 3 + 11	11
2	15	11	11 * 1 + 4	4
3	11	4	4 * 2 + 3	3
4	4	3	3 * 1 + 1	1
5	3	1	1 * 3 + 0	0

After step 5, $b = 0$, so the GCD is $a = \mathbf{1}$. The two numbers are *coprime* (they share no common factor other than 1).

16.6 Implementing the Euclidean Algorithm in Python

Now it is your turn to translate the algorithm into Python.

Recall the algorithm:

1. Compute the remainder $r = a \bmod b$. (In Python: `a % b`)
2. Replace a with b and b with r.
3. If $b = 0$, return a (the GCD).
4. Otherwise, repeat from step 1.

Hint: You will need a `while` loop that runs as long as `b` is not zero.

Task: Complete the function below so that it returns the GCD of two positive integers.

```
def gcd(a: int, b: int) -> int:
    # The Euclidean Algorithm
    #
    # Use a while loop that continues as long as b is not 0.
    # In each iteration:
```

```
    #   1. Compute the remainder of a divided by b  (hint: use the %
    ↪ operator)
    #   2. Set a to the old value of b
    #   3. Set b to the remainder
    # When b becomes 0, a holds the GCD — return it.

    pass  # TODO: replace this line with your implementation
```

16.6.1 Testing Your Implementation

Run the cell below to check whether your `gcd` function produces the correct results.

```
# Test cases — all should print True
print(gcd(48, 18) == 6)
print(gcd(56, 15) == 1)
print(gcd(12, 8) == 4)
print(gcd(100, 75) == 25)
print(gcd(17, 13) == 1)
```

16.7 Ways to Describe an Algorithm

In this notebook we described the Euclidean algorithm in three different ways:

Notation	Example	Pros	Cons
Natural language	"Divide a by b and take the remainder …"	Easy to understand	Can be ambiguous
Table (trace)	Step-by-step table with values of a, b, r	Shows execution clearly	Only for specific inputs
Programming language	`while b != 0: …`	Executable	Tied to one language

17 Algorithm Representation: Pseudocode & Struktogramme

Last week we described the GCD algorithm in three ways: natural language, a trace table, and Python code.

Today we add two more notations:

- **Pseudocode** — precise, structured, language-independent text
- **Struktogramme** (Nassi–Shneiderman diagrams) — visual block diagrams

By the end of this notebook you will be able to translate freely between Python and pseudocode.

17.1 What Is Pseudocode?

Pseudocode is a way of describing an algorithm using structured, human-readable language that *resembles* a programming language but is not tied to any specific one.

Advantage	Explanation
More precise than natural language	Avoids ambiguity
More readable than real code	No syntax details (colons, brackets, imports)
Language-independent	Anyone who knows *any* language can read it
Widely used	Textbooks, exams, professional documentation

17.2 Pseudocode Conventions

There is no single "official" pseudocode standard, but the conventions below are widely used and will be our standard for this course.

Element	Python	Pseudocode
Assignment	`x = 5`	`x ← 5`
Input	`x = int(input())`	`READ x`
Output	`print(x)`	`PRINT x`
Condition	`if x > 0:`	`IF x > 0 THEN`
Else	`else:`	`ELSE`

Element	Python	Pseudocode
End block	*(indentation)*	`END IF`
While loop	`while x > 0:`	`WHILE x > 0 DO`
End loop	*(indentation)*	`END WHILE`
For loop	`for i in range(n):`	`FOR i ← 0 TO n-1 DO`
End for	*(indentation)*	`END FOR`
Function	`def gcd(a, b):`	`FUNCTION gcd(a, b)`
Return	`return a`	`RETURN a`
Not equal	`!=`	`≠`
Modulo	`%`	`MOD`

Key idea: Python uses *indentation* to show structure. Pseudocode uses *keywords* (`THEN`, `DO`, `END IF`, `END WHILE`) to show where blocks begin and end.

17.3 Worked Example: GCD in Pseudocode

Here is the GCD function you wrote last week in Python:

```python
def gcd(a, b):
    while b != 0:
        r = a % b
        a = b
        b = r
    return a
```

And here is the same algorithm in pseudocode:

```
FUNCTION gcd(a, b)
    WHILE b ≠ 0 DO
        r ← a MOD b
        a ← b
        b ← r
    END WHILE
    RETURN a
END FUNCTION
```

Notice the correspondences: - `def` → `FUNCTION` … `END FUNCTION` - `while` → `WHILE` … `DO` … `END WHILE` - `=` (assignment) → `←` - `%` → `MOD` - `!=` → `≠`

17.4 Exercise 1: Python → Pseudocode

Read the Python function below carefully. Then **write the equivalent pseudocode on paper** (or in a new markdown cell).

Use the conventions table above as a reference.

```python
def max_of_three(a, b, c):
    if a >= b and a >= c:
        return a
    elif b >= a and b >= c:
        return b
    else:
        return c
```

Click to reveal the solution

```
FUNCTION max_of_three(a, b, c)
    IF a ≥ b AND a ≥ c THEN
        RETURN a
    ELSE IF b ≥ a AND b ≥ c THEN
        RETURN b
    ELSE
        RETURN c
    END IF
END FUNCTION
```

17.5 Exercise 2: Pseudocode → Python

The pseudocode below describes a **linear search** algorithm. It searches through a list for a given value and returns the index of the first occurrence, or `-1` if the value is not found.

```
FUNCTION linear_search(lst, target)
    FOR i ← 0 TO length(lst) - 1 DO
        IF lst[i] = target THEN
            RETURN i
        END IF
    END FOR
    RETURN -1
END FUNCTION
```

Task: Implement this algorithm in Python in the cell below.

```python
def linear_search(lst: list, target) -> int:
    pass  # TODO: implement the pseudocode above
```

```python
# Test cases — all should print True
print(linear_search([10, 20, 30, 40, 50], 30) == 2)
print(linear_search([10, 20, 30, 40, 50], 10) == 0)
print(linear_search([10, 20, 30, 40, 50], 50) == 4)
print(linear_search([10, 20, 30, 40, 50], 99) == -1)
print(linear_search([], 5) == -1)
```

17.6 Exercise 3: Pseudocode → Python

The pseudocode below prints every integer from 1 to n. For multiples of 3 it prints `"Fizz"` instead of the number, for multiples of 5 it prints `"Buzz"`, and for multiples of both 3 and 5 it prints `"FizzBuzz"`.

```
FUNCTION fizzbuzz(n)
    FOR i ← 1 TO n DO
        IF i MOD 3 = 0 AND i MOD 5 = 0 THEN
            PRINT "FizzBuzz"
        ELSE IF i MOD 3 = 0 THEN
            PRINT "Fizz"
        ELSE IF i MOD 5 = 0 THEN
            PRINT "Buzz"
        ELSE
            PRINT i
        END IF
    END FOR
END FUNCTION
```

Task: Implement this algorithm in Python.

```python
def fizzbuzz(n: int):
    pass  # TODO: implement the pseudocode above
```

```python
# Run this cell to test — expected output:
# 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz,
# 11, Fizz, 13, 14, FizzBuzz
```

```
fizzbuzz(15)
```

17.7 Structure Diagrams (Struktogramme)

A **Struktogramm** (Nassi–Shneiderman diagram) is a visual way of representing an algorithm using nested rectangles. Each rectangle represents one instruction or control structure.

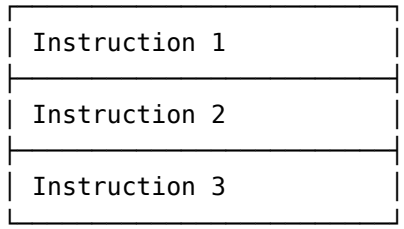
Advantage	Explanation
Visual	Structure is visible at a glance
No arrows	Forces structured programming (no "goto")
Widely used	Common in German-speaking CS education and exams

17.7.1 The Three Building Blocks

Every Struktogramm is built from just three elements:

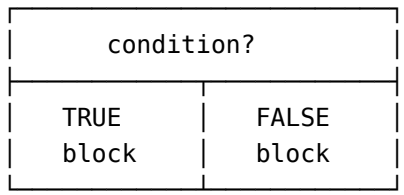
17.7.1.1 1. Sequence

Instructions are executed one after another, top to bottom.



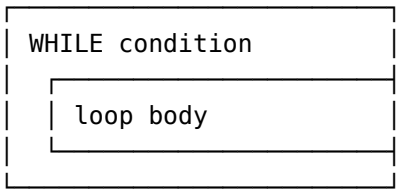
17.7.1.2 2. Selection (IF / ELSE)

A condition splits execution into two branches.

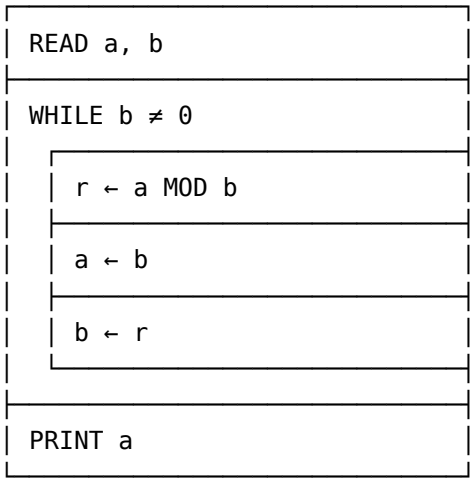


17.7.1.3 3. Iteration (WHILE loop)

A block is repeated as long as a condition holds.



17.7.2 Worked Example: GCD Struktogramm



Reading the diagram: - Top to bottom = execution order (sequence) - The indented box = the loop body (iteration) - `READ` at the top = input, `PRINT` at the bottom = output

17.7.3 Drawing Task (on Paper)

Now draw Struktogramme on paper for the algorithms you translated above:

1. **`max_of_three(a, b, c)`** — uses sequence and selection.
2. **`linear_search(lst, target)`** — uses sequence, selection, and iteration.
3. Sum of all integers from 1 to n.

Compare your diagrams with a partner when you're done.

17.8 Summary: Comparing Notations

Notation	Precise?	Executable?	Visual?	Language-independent?
Natural language				
Pseudocode				
Struktogramm				
Python				

No single notation is “best” — each has strengths. Good computer scientists can switch between notations fluently.

18 Sorting Algorithms

18.1 The Sorting Problem

Input: A sequence of n comparable elements (e.g. numbers, strings).

Output: The same elements rearranged in non-descending order.

Example: - Input: `[4, 2, 5, 1, 3]` - Output: `[1, 2, 3, 4, 5]`

Sorting is one of the most fundamental operations in computer science. There are many different algorithms that solve this problem — you just invented one (or more!) with your playing cards.

18.2 Pseudocode Conventions (Quick Reference)

Recall from last week:

Element	Python	Pseudocode
Assignment	`x = 5`	`x ← 5`
Condition	`if x > 0:`	`IF x > 0 THEN`
Else	`else:`	`ELSE`
End block	*(indentation)*	`END IF`
While loop	`while x > 0:`	`WHILE x > 0 DO`
For loop	`for i in range(n):`	`FOR i ← 0 TO n-1 DO`
End for	*(indentation)*	`END FOR`
Function	`def f(a, b):`	`FUNCTION f(a, b)`
Return	`return a`	`RETURN a`

New for today — SWAP:

Python	Pseudocode
`a, b = b, a`	`SWAP a AND b`
`cards[i], cards[j] = cards[j], cards[i]`	`SWAP cards[i] AND cards[j]`

Python lets you swap two variables in one line: `a, b = b, a`. This works because Python evaluates the right side *first*, then assigns both values simultaneously.

18.3 Helper: Visualising Each Step

The function below prints the current state of the list and highlights which positions were just compared. Use it inside your sorting function to trace what is happening.

```python
def show_step(cards: list, compared: tuple = None, label: str = ""):
    """Print the list, highlighting the compared positions."""
    parts = []
    for idx, val in enumerate(cards):
        if compared and idx in compared:
            parts.append(f"[{val}]")
        else:
            parts.append(f" {val} ")
    line = "  ".join(parts)
    if label:
        print(f"{label:>12s}:  {line}")
    else:
        print(f"              {line}")
```

```python
# Example usage
demo = [4, 2, 5, 1, 3]
show_step(demo, label="start")
show_step(demo, compared=(0, 1), label="compare 0,1")
```

18.4 Your Sorting Algorithm

You invented a sorting strategy with the playing cards. Now implement it in Python!

Task:

1. If you haven't already, write pseudocode for your strategy on the worksheet.
2. Translate your pseudocode into the Python function below.
3. Use `show_step()` to visualise what your algorithm does.
4. Test it — does it produce a sorted list for every input?

Don't worry about efficiency or elegance yet. The goal is to get a *working* implementation of *your* strategy.

```python
def my_sort(cards: list):
    n = len(cards)
    show_step(cards, label="start")

    # TODO: implement your sorting strategy here
    # Use show_step(cards, compared=(i, j), label=...) to trace.

    pass


# Test
test = [4, 2, 5, 1, 3]
print("Before:", test)
my_sort(test)
print("After: ", test)
```

```python
# Once you're happy with your implementation, test it properly:
test_cases = [
    [4, 2, 5, 1, 3],
    [5, 4, 3, 2, 1],
    [1, 2, 3, 4, 5],
    [3, 3, 1, 2, 2],
    [1],
]
for tc in test_cases:
    original = tc.copy()
    my_sort(tc)
    status = "✓" if tc == sorted(original) else "✗"
    print(f"  {status}  {original} → {tc}")
```

18.5 Named Sorting Algorithms

Now that you have a working sorting function, let's look at two well-known sorting algorithms. You may recognise your own strategy in one of them — or discover a completely different approach.

18.6 Selection Sort

Core idea: Repeatedly find the *minimum* of the unsorted portion and swap it into the next position.

18.6.1 Pseudocode

```
FUNCTION selection_sort(cards, n)
    FOR i ← 0 TO n - 2 DO
        min_pos ← i
        FOR j ← i + 1 TO n - 1 DO
            IF cards[j] < cards[min_pos] THEN
                min_pos ← j
            END IF
        END FOR
        SWAP cards[i] AND cards[min_pos]
    END FOR
END FUNCTION
```

18.6.2 Walkthrough (5 cards: 4, 2, 5, 1, 3)

Pass	Unsorted portion	Minimum	Swap with pos	Result
1	4, 2, 5, 1, 3	1 (pos 3)	pos 0	**1**, 2, 5, 4, 3
2	2, 5, 4, 3	2 (pos 1)	pos 1 (no move)	1, **2**, 5, 4, 3
3	5, 4, 3	3 (pos 4)	pos 2	1, 2, **3**, 4, 5
4	4, 5	4 (pos 3)	pos 3 (no move)	1, 2, 3, **4**, 5

18.6.3 Exercise: Implement Selection Sort

Translate the pseudocode above into Python. Use `show_step()` to print the state after each pass.

```python
def selection_sort(cards: list) -> list:
    n = len(cards)
    # TODO: implement Selection Sort here
    # Hint: use the pseudocode as your guide.
    # Use show_step(cards, compared=(i, j), label=...) to trace.
    pass


# Test
```

```
test = [4, 2, 5, 1, 3]
print("Before:", test)
selection_sort(test)
print("After: ", test)
```

Click to reveal the solution

```
def selection_sort(cards: list) -> list:
    n = len(cards)
    for i in range(n - 1):
        min_pos = i
        for j in range(i + 1, n):
            if cards[j] < cards[min_pos]:
                min_pos = j
        cards[i], cards[min_pos] = cards[min_pos], cards[i]
        show_step(cards, label=f"pass {i+1}")
```

18.7 Bubble Sort

Core idea: Walk through the list from left to right, compare adjacent elements, and swap them if they are in the wrong order. Repeat until the list is sorted. The largest unsorted element "bubbles up" to the end in each pass.

18.7.1 Pseudocode

```
FUNCTION bubble_sort(cards, n)
    FOR i ← 0 TO n - 2 DO
        FOR j ← 0 TO n - 2 - i DO
            IF cards[j] > cards[j + 1] THEN
                SWAP cards[j] AND cards[j + 1]
            END IF
        END FOR
    END FOR
END FUNCTION
```

18.7.2 Walkthrough (5 cards: 4, 2, 5, 1, 3)

Pass 1: (compare positions 0–1, 1–2, 2–3, 3–4)

Compare	Before	Swap?	After
pos 0 & 1	**4 2** 5 1 3	Yes	**2 4** 5 1 3
pos 1 & 2	2 **4 5** 1 3	No	2 **4 5** 1 3
pos 2 & 3	2 4 **5 1** 3	Yes	2 4 **1 5** 3
pos 3 & 4	2 4 1 **5 3**	Yes	2 4 1 **3 5**

→ After pass 1: `2, 4, 1, 3, 5` — the 5 has bubbled to the end.

Passes 2–3 continue until the list is fully sorted: `1, 2, 3, 4, 5`.

18.7.3 Exercise: Implement Bubble Sort

Translate the pseudocode above into Python. Use `show_step()` to print the state after each pass.

```
def bubble_sort(cards: list) -> list:
    n = len(cards)
    # TODO: implement Bubble Sort here



# Test
test = [4, 2, 5, 1, 3]
print("Before:", test)
bubble_sort(test)
print("After: ", test)
```

Click to reveal the solution

```
def bubble_sort(cards: list) -> list:
    n = len(cards)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if cards[j] > cards[j + 1]:
                cards[j], cards[j + 1] = cards[j + 1], cards[j]
        show_step(cards, label=f"pass {i+1}")
```

18.8 Testing Your Implementations

Run the cell below to test whichever sorting function you implemented. Then test `selection_sort` and `bubble_sort` after you've implemented them.

```
import random


def test_sort(sort_func, uses_bounds=False):
    """Test a sorting function on several inputs."""
    test_cases = [
        [4, 2, 5, 1, 3],
        [1, 2, 3, 4, 5],         # already sorted
        [5, 4, 3, 2, 1],         # reverse sorted
        [1],                      # single element
        [3, 3, 1, 2, 2],         # duplicates
        random.sample(range(1, 11), 10),  # random 10 elements
    ]
    all_passed = True
    for i, case in enumerate(test_cases):
        original = case.copy()
        expected = sorted(case)
        if uses_bounds:
            sort_func(case, 0, len(case) - 1)
        else:
            sort_func(case)
        if case == expected:
            print(f"Test {i+1}: PASS  {original} → {case}")
        else:
            print(f"Test {i+1}: FAIL  {original} → {case}  (expected
            ↪  {expected})")
            all_passed = False
    if all_passed:
        print("\n✓ All tests passed!")
    else:
        print("\n✗ Some tests failed — check your code.")
```

```
# Test your own sorting function:
test_sort(my_sort)
```

```
# Test Selection Sort:
test_sort(selection_sort)
```

```
# Test Bubble Sort:
test_sort(bubble_sort)
```

18.9 Counting Comparisons

How many comparisons does each algorithm need? The functions below are instrumented versions that count every comparison. Run them on the same input to compare.

```
def selection_sort_count(cards: list) -> int:
    """Selection Sort that returns the number of comparisons."""
    n = len(cards)
    comparisons = 0
    for i in range(n - 1):
        min_pos = i
        for j in range(i + 1, n):
            comparisons += 1
            if cards[j] < cards[min_pos]:
                min_pos = j
        cards[i], cards[min_pos] = cards[min_pos], cards[i]
    return comparisons


def bubble_sort_count(cards: list) -> int:
    """Bubble Sort that returns the number of comparisons."""
    n = len(cards)
    comparisons = 0
    for i in range(n - 1):
        for j in range(n - 1 - i):
            comparisons += 1
            if cards[j] > cards[j + 1]:
                cards[j], cards[j + 1] = cards[j + 1], cards[j]
    return comparisons
```

```
# Compare on the same input
original = [6, 3, 8, 2, 7, 1]

cards_a = original.copy()
c_sel = selection_sort_count(cards_a)
print(f"Selection Sort: {c_sel} comparisons → {cards_a}")
```

```
7
8  cards_b = original.copy()
9  c_bub = bubble_sort_count(cards_b)
10 print(f"Bubble Sort:    {c_bub} comparisons → {cards_b}")
```

19 Algorithms and Computational Complexity

> **Arrays: The Basic Data Structure**
>
> An array is a basic data structure. It stores a fixed number of elements of the same data type under a single variable name. Sorting these elements is one of the most fundamental tasks in computer science.

19.1 Selection Sort

Selection Sort works as follows:

1. Go through the array to find the lowest value.
2. Move the lowest value to the front of the unsorted part of the array.
3. Repeat this process for the unsorted part as many times as there are values in the array.

In order to evaluate the time complexity of Selection Sort, it is necessary to **analyse** how the number of comparisons increases as a function of the number of elements being sorted. Due to the way Selection Sort operates, approximately $\frac{1}{2}n^2$ comparisons are required for n elements. This means the number of operations increases **quadratically** relative to the input size.

Since we **categorise** an algorithm's efficiency according to its **order of magnitude**, we refer to this as having quadratic time complexity. In this process, constant factors (like the $\frac{1}{2}$) and lower-order terms are disregarded, as the primary concern when evaluating an algorithm is how the time requirement scales with extremely large datasets.

19.2 Big O Notation

To formally capture this focus on the fundamental rate of growth, computer scientists use a specific shorthand known as **Big O notation**.

The 'O' stands for 'Order of', indicating the **complexity class** of an algorithm. Instead of meticulously counting every single operation, Big O notation describes the upper bound of the growth rate. For instance, stating that Selection Sort has a time complexity of $O(n^2)$ tells us that if the input size doubles, the time required will roughly quadruple, regardless of the computer's speed or any minor mathematical constants.

19.3 Merge Sort: A More Efficient Approach

To illustrate why choosing the right complexity class matters, let's compare Selection Sort with **Merge Sort**. While Selection Sort works by repeatedly searching for the minimum, Merge Sort uses a **"divide and conquer"** strategy. It recursively splits the array into two halves until each part contains only one element, and then merges these parts back together in the correct order.

19.3.1 The Merge Step: Combining with Efficiency

The actual efficiency of Merge Sort lies in the **merge step**:

1. Imagine you have two smaller piles of cards, both already sorted from lowest to highest.
2. You only ever need to compare the two "top" cards of each pile.
3. You pick the smaller one, place it into the new, larger array, and move to the next card in that pile.

Because this step only requires a single pass through the elements, merging two sorted lists of total size n takes **linear time**, or $O(n)$. Since the "Divide" phase splits the problem into $\log n$ levels and an $O(n)$ merge operation is performed at each level, the total time complexity becomes:

$$n \cdot \log n = O(n \log n)$$

19.4 Comparison of Growth Rates

The "log-linear" growth of Merge Sort is significantly more efficient than the "quadratic" growth of Selection Sort as the input size n increases.

Tabelle 19.1: Comparison of time complexity between Selection Sort and Merge Sort

Number of Elements(n)	Selection Sort($O(n^2)$)	Merge Sort($O(n \log n)$)	Efficiency(Ratio)
10	100 operations	~33 operations	3x faster
1,000	1,000,000	~10,000	100x faster
1,000,000	1,000,000,000,000	~20,000,000	**50,000x faster**

For a small list, the difference is negligible. However, for large datasets, Selection Sort becomes unusable, while Merge Sort remains efficient. This is the power of understanding computational complexity: it allows us to predict how an algorithm will perform when data scales up.

Teil IV

Functions

20 Functions: Revision and Consolidation

20.1 Function: A Definition

A function is a self-contained block of code that encapsulates specific logic. By using parameters and return values, it acts as a 'black box' that processes inputs into outputs.

Let's break this definition down into its core components to understand how a function actually works:

1. "Self-contained block"

   A function is like a "mini-program" within the main program. It has its own name and its own scope. This means that variables defined inside a function usually don't interfere with the rest of the code. It promotes modular design, making the code organised and easier to read and to debug.

2. "Encapsulates specific logic"

   Encapsulation means "wrapping things up." There is no need to know how the function actually works to use it. It is sufficient to know what parameters it takes and what values it returns. This is the first step toward abstraction.

3. "Parameters" (The Inputs)

   Parameters are the "knobs" or "settings" of a function. They allow for passing information into the function. Without parameters, a function would do exactly the same thing every time. With parameters, it becomes flexible and can handle different inputs each time it is called.

4. "Return Values" (The Outputs)

   A function doesn't just "do" something; it often "gives" something back. The `return` statement is the exit point of the function, handing a result back to the part of the program that called it.

5. The "Black Box" Concept

   This is a key concept in computer science. The user provides an input, the "black box" (the function) performs its processing logic, and it produces an output. The user of a function only cares about the interface (Inputs/Outputs), not the implementation.

20.2 Practical Example: The VAT Calculator

In this example, we calculate the net price from a given gross price. In Swiss retail, prices are usually quoted as *gross* — i.e., including VAT. The *net price* is what remains after the VAT portion is removed. Since the standard VAT rate in Switzerland is 8.1 %, we use this as a default parameter. This makes the function easier to use for standard cases while remaining flexible for others.

```python
def calculate_net_price(gross_price: float, vat_rate: float = 8.1) ->
    float:
    """
    Calculates the net price from a given gross price.

    Args:
        gross_price (float): The total price including VAT.
        vat_rate (float): The VAT percentage to be removed.
                          Defaults to 8.1 (Swiss standard rate).

    Returns:
        float: The calculated net price, rounded to two decimal places.
    """
    # gross = net * (1 + VAT/100)  ->  net = gross / (1 + VAT/100)
    gross_factor = 1 + (vat_rate / 100)
    net_price = gross_price / gross_factor

    return round(net_price, 2)

# --- Using the "Black Box" ---

# Case A: Standard rate (8.1% is used automatically)
standard_total = calculate_net_price(100.0)
print(f"Standard Net Price: {standard_total} CHF")

# Case B: Reduced rate for everyday goods (2.6%)
reduced_total = calculate_net_price(100.0, 2.6)
print(f"Reduced Net Price: {reduced_total} CHF")
```

New Elements Explained

- **Type Hints** (`: float` and `-> float`): These document for the programmer (and the IDE) that the function expects decimal numbers and will return a decimal number. They are like labels on the "knobs" of our black box. *Note: Python does not enforce Type Hints at runtime — they serve documentation and tooling, not validation.*
- **The Docstring** (`""" ... """`): This is the manual for the black box. It explains what the function does, what the arguments (Args) represent, and what the function Returns. In professional IDEs, this text pops up when you hover over the function name. The format used here is the *Google style* for docstrings.
- **Default Parameter** (`vat_rate: float = 8.1`): By assigning a value in the signature, we make this input optional. If the user doesn't provide a rate, Python automatically uses 8.1.

How this illustrates our five points:

Modularity & Scope: The variables `gross_factor` and `net_price` only exist inside the function. Your main program stays "clean" and doesn't get cluttered with temporary calculation variables.

Encapsulation: The mathematical formula (dividing by the factor rather than just subtracting a percentage) is hidden. The user doesn't need to be a maths expert; they just need to call the function.

Default Parameters: By setting `vat_rate=8.1`, we've made the function smarter. It assumes the most common case but allows the user to override it for special cases like hotels or groceries.

The Interface: The "interface" here is simply `calculate_net_price(gross_price, vat_rate)`. As long as you know this signature, you can use the tool without ever looking at the code inside the function again.

20.3 Coding Exercise: The BMI Calculator

20.3.1 Objective

Apply your knowledge of functions, encapsulation, and documentation by creating a tool to calculate the Body Mass Index (BMI).

20.3.2 The Task

Write a Python function named `calculate_bmi` that takes a person's weight and height as inputs and returns their BMI. Remember the "Black Box" principle: the user should only need to provide the numbers; your function handles the maths.

20.3.3 The Formula

The BMI is calculated by dividing a person's weight in kilograms by the square of their height in metres:

$$BMI = \frac{\text{weight (kg)}}{\text{height (m)}^2}$$

20.3.4 Requirements

- **Function Signature:** Use Type Hints to specify that both inputs and the return value are floats.
- **Docstring:** Include a complete Docstring at the beginning of the function. Use the Google style (with the sections `Args:` and `Returns:`) as shown in the previous example. Write the prose in British English spelling (e.g., "metres", "organised").
- **Encapsulation:** The internal calculation should be performed within the function, and the result should be returned rounded to two decimal places.
- **Testing:** Call your function at the end of your script to calculate the BMI for a person who weighs 80 kg and is 1.85 m tall.

20.3.5 Exercise Template:

```
# Define the signature: which parameters with which Type Hints?
# What is the return type?
def calculate_bmi(...) -> ...:
    """
    [Write a brief description of the function here]

    Args:
        [List the arguments here]

    Returns:
        [Describe the return value here]
    """
    # 1. Implementation: Calculate the BMI using the formula

```

```
    # 2. Return Value: Return the result rounded to 2 decimal places
    pass

# --- Test your "Black Box" ---
# 1. Call the function with 80.0 and 1.85
# 2. Print the result with an appropriate message
```

```
def calculate_bmi(weight: float, height: float) -> float:
    """
    Calculates the body mass index of a person from the person's weight
    and height.

    Args:
        weight (float): The person's weight in kilogram.
        height (float): The person's height in metres.
    Returns:
        float: The person's BMI, rounded to two decimal places.
    """
    # 1. logic -> implementing the bmi formula
    bmi = weight / (height ** 2)

    # 2. formatting -> round the return value to 2 decimals
    return round(bmi, 2)

print(calculate_bmi(80.0, 1.85))
```

20.3.6 Success Criteria

- The function returns `23.37` for a weight of 80 kg and a height of 1.85 m.
- The Docstring clearly explains what the parameters represent.
- The calculation logic is entirely contained within the function (Encapsulation).
- The function uses the `return` statement correctly rather than just printing the result inside the function.

20.3.7 Optional Extension: A Classifier Function

The BMI is a rough indicator and has well-known limitations (it ignores muscle mass and body composition). As a follow-up exercise, write a second function `classify_bmi(bmi: float) -> str` that returns a category according to the WHO scale (e.g. *underweight*, *normal*, *overweight*, *obese*).

This is the natural bridge to the next topic — conditional statements (`if` / `elif` / `else`) — and it also lets you practice **function composition**:

```
category = classify_bmi(calculate_bmi(80.0, 1.85))
```

20.4 Coding Exercise: The Cuboid Space Diagonal

20.4.1 Objective

Create a geometry tool that calculates the space diagonal (the longest internal line) of a cuboid. This exercise will help you practice handling multiple parameters and using Python's mathematical libraries.

20.4.2 The Task

Write a Python function named `calculate_space_diagonal` that accepts the three edge lengths of a cuboid and returns the length of the space diagonal.

20.4.3 The Formula

To find the space diagonal (d) of a cuboid with edge lengths a, b, and c, we use the 3D version of Pythagoras' Theorem:

$$d = \sqrt{a^2 + b^2 + c^2}$$

20.4.4 Requirements

- **Module Import:** You will need the `sqrt` function from Python's `math` module. After `import math`, the call `math.sqrt(x)` calculates the square root of a `float`.
- **Function Signature:** Use Type Hints to ensure all three inputs and the return value are floats.
- **Docstring:** Write a professional Docstring in the Google style (sections `Args:` and `Returns:`), using British English spelling (e.g., "metres", "calculated").
- **Encapsulation:** The user should not have to worry about the square root or the squaring of numbers; the "Black Box" handles the maths internally.
- **Testing:** Test your function with a cuboid that has the dimensions $a = 5.0$, $b = 10.0$, and $c = 2.0$.

Tip: You can square a number with `x ** 2` (or `x * x`). The square root requires `math.sqrt(x)` after `import math`.

```
import math  # this works analogously to the import of PyTamaro

```

```
def calculate_space_diagonal(...) -> ...:
    # TODO: implement the function according to the given requirements
    pass

# --- Test your "Black Box" ---
# Call the function with edge lengths 5.0, 10.0, and 2.0
# Expected result: approx. 11.36
```

```
import math

def calculate_space_diagonal(a: float, b: float, c: float) -> float:
    """
    Calculates the space diagonal of a cuboid from the edge lengths.

    Args:
        a (float): Length of the first edge.
        b (float): Length of the second edge.
        c (float): Length of the third edge.

        The function works independently of the actual units as long as
        all edges are provided in the same unit.

    Returns:
        float: The calculated space diagonal of a cuboid in the same
        unit as the input.
    """

    d = math.sqrt(a**2 + b**2 + c**2)
    return d

calculate_space_diagonal(5.0, 10.0, 2.0)
```

21 Functions II: Calculating Triangles

21.1 Functions within functions

Python functions can not only process Python statements, but they can also call user-defined functions. The implementation of Heron's formula will serve as an example.

21.1.1 Calculating a Triangle's Area Using Heron's Formula

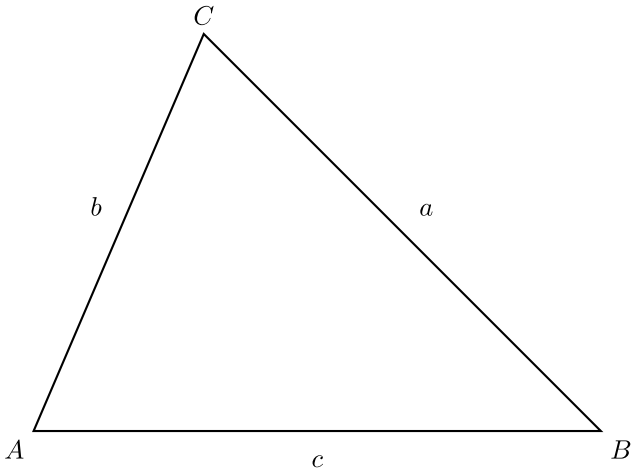


Abbildung 21.1: General Triangle.

To calculate the area T of a triangle, we use Heron's formula

$$T = \sqrt{s(s-a)(s-b)(s-c)}$$

where s denotes the *semiperimeter*. The semiperimeter is calculated by adding the lengths of the triangle's sides and dividing the sum by two.

$$s = \frac{1}{2} \cdot (a + b + c)$$

Coding Task

Implement a function `area()` to calculate the area of a triangle from its sides. Use a subfunction `semiperimeter()` to simplify the logic.

```python
def semiperimeter(a: float, b: float, c: float) -> float:
    """Calculates the semiperimeter of a triangle.

    Args:
        a: Side a of the triangle.
        b: Side b of the triangle.
        c: Side c of the triangle.

    Returns:
        s: The semiperimeter of the triangle.
    """
    return (a + b + c) / 2

def area(a: float, b: float, c: float) -> float:
    """Calculates the area of a triangle.

    The triangle's area is calculated from its sides using Heron's
    formula.

    Args:
        a: Side a of the triangle.
        b: Side b of the triangle.
        c: Side c of the triangle.

    Returns:
        T: The Area of the triangle."""
    s = semiperimeter(a, b, c)
    T = (s*(s-a)*(s-b)*(s-c))**(1/2)
    return T

print(area(3, 4, 5))
# should print 6.0 to the command line
```

```
6.0
```

21.1.2 The Altitude of a Triangle

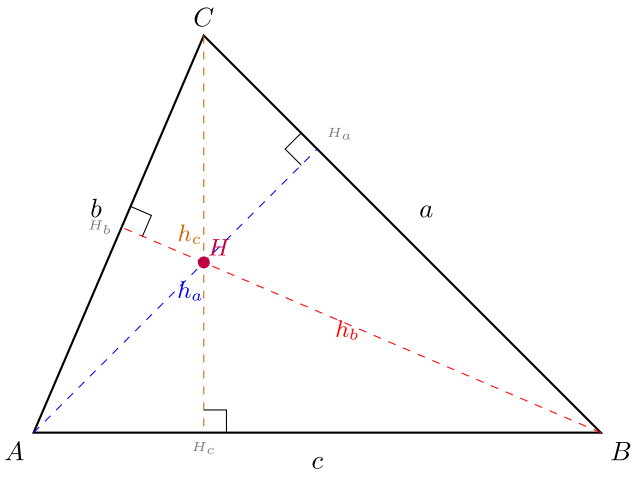


Abbildung 21.2: Altitude of a Triangle.

The altitude h of a triangle is calculated by the formula

$$h_a = \frac{2}{a}\sqrt{s(s-a)(s-b)(s-c)}$$

where s denotes the *semiperimeter*. To calculate the altitude h_b and h_c,switch the respective variables in the formula accordingly.

Coding Task

Implement a function `altitude()` to calculate the altitude of a triangle from its sides. Use as many subfunctions as possible.

```
1  def altitude(a: float, b: float, c: float, base: str = 'c') -> float:
2      """Calculates the altitude of a triangle.
3
4      The altitude is calculated to the base c by default. The base can
5      be overridden if needed.
6
7      Args:
8          a: Side a of the triangle.
9          b: Side b of the triangle.
10         c: Side c of the triangle.
11         base: The base of the altitude. The default value is 'c'
12
13     Returns:
```

```
14          h: The altitude of the triangle."""
15      T = area(a, b, c)
16
17      if base == 'c':
18          h = (2*T)/c
19      elif base == 'a':
20          h = (2*T)/a
21      else:
22          h = (2*T)/b
23
24      return h
25
26  print(altitude(3, 4, 5, 'a'))
```

Default Values for Function Arguments

If you have to draw the altitude of a triangle, you most likely would draw it from the horizontal side as a base line. Therefore, the base line will frequently be side c of your triangle. To account for this probability, Python offers a default value option for function arguments, as demonstrated in the `altitude()` function above.
The syntax requires that arguments with a default value must be placed at the end of the argument list. The default value is assigned using the = sign. This default value can be overridden either implicitly as a positional argument by providing a full list of arguments (e.g., `altitude(3, 4, 5, 'a')`), or explicitly as a keyword argument by directly assigning a new value to the parameter (e.g., `altitude(3, 4, 5, base='a')`).

21.1.3 Circumcircle of a Triangle

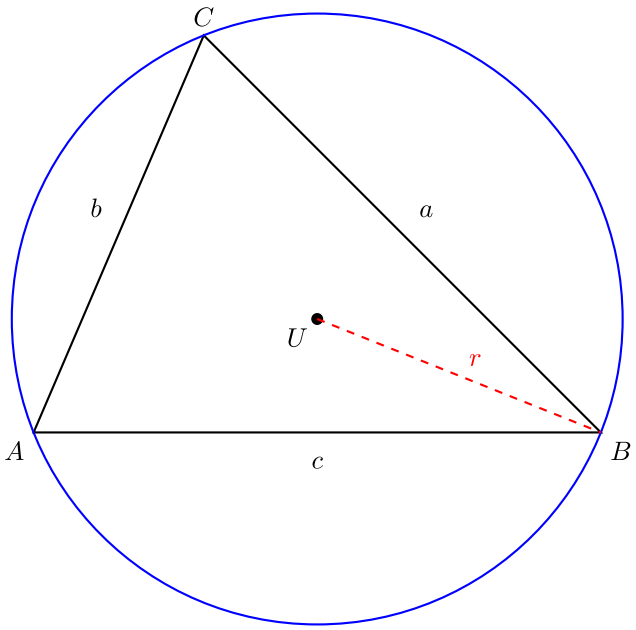


Abbildung 21.3: The Circumcircle of a Triangle.

The radius r of the circumcircle of a triangle is calculated by the formula

$$r = \frac{abc}{4T}$$

where T denotes the area of the triangle.

Coding Task

Implement a function `circumcircle()` to calculate the circumcircle radius of a triangle from its sides. Use as many subfunctions as possible.

```
def circumcircle(a: float, b: float, c: float) -> float:
    """Calculates the circumcircle radius of a triangle.

    Args:
        a: Side a of the triangle.
        b: Side b of the triangle.
        c: Side c of the triangle.

    Returns:
```

```
        r: The radius of the circumcircle of a triangle.
    """
    T = area(a, b, c)
    n = a * b * c
    d = 4 * T
    r = n / d
    return r

print(circumcircle(3, 4, 5))
```

21.1.4 The Incircle of a Triangle

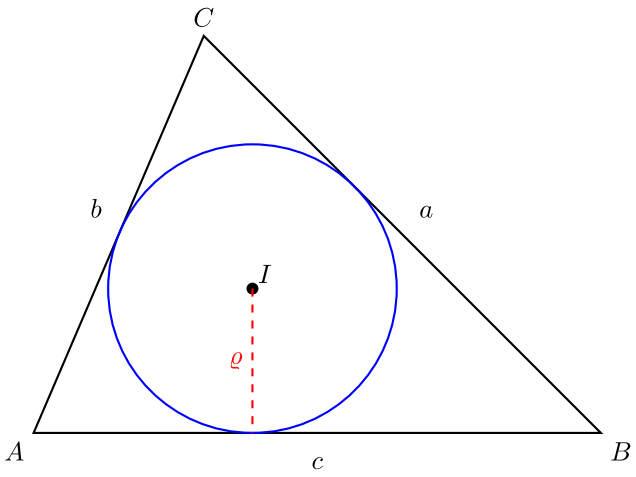


Abbildung 21.4: The Incircle of a Triangle.

The incircle radius ϱ (small greek letter rho) of a triangle is calculated by the formula

$$\varrho = \frac{T}{s}$$

where T denotes the area of the triangle and s the semiperimeter respectively.

Coding Task

Implement a function `incircle()` to calculate the incircle radius of a triangle from its sides. Use as many subfunctions as possible.

```
def incircle(a: float, b: float, c: float) -> float:
    """Calculates the incircle radius.
```

```
3
4       Args:
5           a: Side a of the triangle.
6           b: Side b of the triangle.
7           c: Side c of the triangle.
8
9       Returns:
10          rho: Incircle radius of the triangle.
11      """
12      T = area(a, b, c)
13      s = semiperimeter(a, b, c)
14      rho = T/s
15      return rho
16
17  print(incircle(3, 4, 5))
```

22 Classes: Triangle as Object

22.1 Introduction

Functions are perfect for avoiding repetitive code (DRY principle). But when you model real-world things—like a collection of triangles defined by their three side lengths—a Python class allows you to bundle both the data and the functions together efficiently.

In the following code cell, the functions from the previous chapter are integrated into a class named `TriangleSSS`.

```python
class TriangleSSS:
    """Represents a geometric triangle based on its three side lengths.

    Attributes:
        a (float): The length of the first side.
        b (float): The length of the second side.
        c (float): The length of the third side.
    """

    def __init__(self, a: float, b: float, c: float):
        """Initialises the triangle with the three side lengths.

        Args:
            a (float): Length of side a.
            b (float): Length of side b.
            c (float): Length of side c.
        """
        self.a = a
        self.b = b
        self.c = c

    def _get_semiperimeter(self):
        """Calculates the semiperimeter of the triangle.

        Returns:
            float: The semiperimeter of the triangle.
        """
        return (self.a + self.b + self.c) / 2
```

```
29
30      def get_area(self):
31          """Calculates the area of the triangle using Heron's formula.
32
33          Returns:
34              float: The area of the triangle.
35          """
36          s = self._get_semiperimeter()
37          T = (s * (s - self.a) * (s - self.b) * (s - self.c)) ** (1/2)
38          return T
39
40      def get_altitude(self, base='a'):
41          """Calculates the altitude of the triangle relative to a
            ↪ specific base side.
42
43          Args:
44              base (str, optional): The base side to which the altitude
    ↪ relates
45                  ('a', 'b', or 'c'). Defaults to 'a'.
46
47          Returns:
48              float: The altitude relative to the chosen base side.
49          """
50          T = self.get_area()
51          if base == 'a':
52              h = (2 * T) / self.a
53          elif base == 'b':
54              h = (2 * T) / self.b
55          else:
56              h = (2 * T) / self.c
57          return h
58
59      def get_circumcircle_radius(self):
60          """Calculates the radius of the circumcircle of the triangle.
61
62          Returns:
63              float: The radius of the circumcircle.
64          """
65          T = self.get_area()
66          n = self.a * self.b * self.c
67          d = 4 * T
68          r = n / d
69          return r
70
```

```
    def get_incircle_radius(self):
        """Calculates the radius of the incircle of the triangle.

        Returns:
            float: The radius of the incircle.
        """
        T = self.get_area()
        s = self._get_semiperimeter()
        rho = T / s
        return rho
```

To use the `TriangleSSS` class, we need to create (instantiate) a new triangle object. The following code cell shows how to do this.

```
triangle = TriangleSSS(3, 4, 5)
```

When calling `TriangleSSS(3, 4, 5)`, a very special function is automatically executed behind the scenes: the `__init__()` method (often referred to as the constructor).

The values `3`, `4`, and `5` are passed into the parameters `a`, `b`, and `c`. The self keyword then ensures that these values are bound to the newly created instance and stored in memory as attributes (`self.a`, `self.b`, and `self.c`). It is precisely through this step that the triangle object initialises and "remembers" its individual side lengths.

You can easily verify that these attributes have been successfully saved by accessing them directly using dot notation - for instance, running `print(triangle.a)` will output `3`.

The newly instantiated object `triangle` allows us to access the methods of the `TriangleSSS` class. Methods are simply functions defined inside a class. They are called using *dot notation*, as shown in the example below.

```
triangle.get_area()
```

```
6.0
```

Normally, we call a Python function by passing arguments inside the parentheses. However, thanks to the `self` argument in our method definitions, the method already knows the values of `a`, `b`, and `c` (by accessing `self.a`, `self.b`, and `self.c`). The variable `self` always refers to the specific instance of our object.

22.2 Enhanced Triangle Class

In geometry, a triangle is uniquely defined if we know specific combinations of its side lengths and angles. This is determined by the four classic congruence criteria:

SSS (Side-Side-Side): All three side lengths are known.

SAS (Side-Angle-Side): Two sides and the enclosed angle are known.

ASA / **SAA** (Angle-Side-Angle / Side-Angle-Angle): One side and two angles are known.

SsA (Side-Side-Angle): Two sides and an angle opposite the longer side are known.

While our initial `TriangleSSS` class worked perfectly when given three sides, a truly versatile geometric model should handle any valid combination. Ideally, we want a single, elegant Python class capable of dynamically initialising a triangle based on whichever of these four criteria the user provides.

The following implementation achieves exactly this by identifying the given parameters and validating them against mathematical rules before creating the object.

```
import math

class Triangle:
    def __init__(self, a=None, b=None, c=None, alpha=None, beta=None,
     ↪  gamma=None):
        """Initialise a Triangle instance using valid congruence
         ↪  criteria.

        Args:
            a (float, optional): Length of side a. Defaults to None.
            b (float, optional): Length of side b. Defaults to None.
            c (float, optional): Length of side c. Defaults to None.
            alpha (float, optional): Angle alpha opposite to side a.
    ↪  Defaults to None.
            beta (float, optional): Angle beta opposite to side b.
    ↪  Defaults to None.
            gamma (float, optional): Angle gamma opposite to side c.
    ↪  Defaults to None.

        Raises:
            ValueError: If the number of provided parameters is not
    ↪  exactly three,
                if the parameters do not form a unique triangle, or if
                mathematical rules (e.g. triangle inequality) are
    ↪  violated.
        """
        # 1. Initialise all attributes to prevent subsequent
         ↪  AttributeErrors
        self.a = a
        self.b = b
        self.c = c
```

```
        self.alpha = alpha
        self.beta = beta
        self.gamma = gamma

        # 2. Determine which values were actually passed to the
         ↪ constructor
        given_sides = [k for k, v in [('a', a), ('b', b), ('c', c)] if v
    ↪ is not None]
        given_angles = [k for k, v in [('alpha', alpha), ('beta', beta),
    ↪ ('gamma', gamma)] if v is not None]

        num_sides = len(given_sides)
        num_angles = len(given_angles)

        # A triangle mathematically always requires exactly 3 parameters
        if num_sides + num_angles != 3:
            raise ValueError("A triangle must be defined by exactly 3
             ↪ parameters.")

        # 3. Validate mathematical congruence theorems
        if num_sides == 3:
            # --- SSS (Side-Side-Side) ---
            if not (a + b > c and a + c > b and b + c > a):
                raise ValueError("Triangle inequality violated: The sum
                 ↪ of two sides must be greater than the third.")

        elif num_sides == 2 and num_angles == 1:
            # --- SAS or SsA (Side-Angle-Side / Side-Side-Angle) ---
            angle_name = given_angles[0]
            opposite_mapping = {'alpha': 'a', 'beta': 'b', 'gamma': 'c'}
            opposite_side = opposite_mapping[angle_name]

            if opposite_side in given_sides:
                # Case: SsA (Angle is opposite one of the given sides)
                other_side = [s for s in given_sides if s !=
    ↪ opposite_side][0]
                val_opposite = {'a': a, 'b': b, 'c': c}[opposite_side]
                val_other = {'a': a, 'b': b, 'c': c}[other_side]

                if val_opposite <= val_other:
                    raise ValueError(f"SsA error: Angle '{angle_name}'
                     ↪ must be opposite the longer side.")

        elif num_sides == 1 and num_angles == 2:
```

```
            # --- ASA / SAA (Angle-Side-Angle / Side-Angle-Angle) ---
            angle_values = [v for v in [alpha, beta, gamma] if v is not None]
            if sum(angle_values) >= 180:
                raise ValueError("The sum of the given angles must be less than 180 degrees.")

        elif num_angles == 3:
            # --- AAA (Angle-Angle-Angle) ---
            raise ValueError("AAA (Three angles) does not define a unique triangle due to a lack of scaling.")

        # 4. Mathematically solve the complete triangle to populate all missing sides and angles
        self._solve_triangle()

    def _solve_triangle(self):
        """Private helper method to calculate all missing sides and angles."""
        def rad(deg): return math.radians(deg)
        def deg(rad): return math.degrees(rad)

        # Case 1: SSS
        if self.a and self.b and self.c:
            self.alpha = deg(math.acos((self.b**2 + self.c**2 - self.a**2) / (2 * self.b * self.c)))
            self.beta = deg(math.acos((self.a**2 + self.c**2 - self.b**2) / (2 * self.a * self.c)))
            self.gamma = 180.0 - self.alpha - self.beta

        # Case 2: 2 sides, 1 angle (SAS or SsA)
        elif sum(s is not None for s in [self.a, self.b, self.c]) == 2:
            sides = {'a': self.a, 'b': self.b, 'c': self.c}
            angles = {'alpha': self.alpha, 'beta': self.beta, 'gamma': self.gamma}
            given_s = [k for k, v in sides.items() if v is not None]
            given_a = [k for k, v in angles.items() if v is not None][0]
            angle_val = angles[given_a]

            enclosed_map = {('a', 'b'): 'gamma', ('b', 'a'): 'gamma',
                            ('b', 'c'): 'alpha', ('c', 'b'): 'alpha',
                            ('a', 'c'): 'beta', ('c', 'a'): 'beta'}

            if given_a == enclosed_map[tuple(given_s)]:
```

```
                # Sub-case: SAS
                s1, s2 = given_s[0], given_s[1]
                v1, v2 = sides[s1], sides[s2]
                v3 = math.sqrt(v1**2 + v2**2 - 2 * v1 * v2 *
    ↪ math.cos(rad(angle_val)))
                missing_s = [s for s in ['a', 'b', 'c'] if s not in
    ↪ given_s][0]
                setattr(self, missing_s, v3)
                # Recalculate angles as SSS
                self.alpha = deg(math.acos((self.b**2 + self.c**2 -
                 ↪  self.a**2) / (2 * self.b * self.c)))
                self.beta = deg(math.acos((self.a**2 + self.c**2 -
                 ↪  self.b**2) / (2 * self.a * self.c)))
                self.gamma = 180.0 - self.alpha - self.beta
            else:
                # Sub-case: SsA
                opposite_side = {'alpha': 'a', 'beta': 'b', 'gamma':
    ↪ 'c'}[given_a]
                other_side = [s for s in given_s if s !=
    ↪ opposite_side][0]
                val_opp, val_oth = sides[opposite_side],
    ↪ sides[other_side]

                sin_other = math.sin(rad(angle_val)) * val_oth / val_opp
                other_angle_val = deg(math.asin(sin_other))
                other_angle_name = {'a': 'alpha', 'b': 'beta', 'c':
    ↪ 'gamma'}[other_side]
                setattr(self, other_angle_name, other_angle_val)

                missing_a = [a for a in ['alpha', 'beta', 'gamma'] if
    ↪ getattr(self, a) is None][0]
                setattr(self, missing_a, 180.0 - angle_val -
                 ↪  other_angle_val)

                missing_s = {'alpha': 'a', 'beta': 'b', 'gamma':
    ↪ 'c'}[missing_a]
                missing_s_val = val_opp * math.sin(rad(getattr(self,
    ↪ missing_a))) / math.sin(rad(angle_val))
                setattr(self, missing_s, missing_s_val)

        # Case 3: 1 side, 2 angles (ASA / SAA)
        elif sum(s is not None for s in [self.a, self.b, self.c]) == 1:
            if self.alpha is None: self.alpha = 180.0 - self.beta -
             ↪  self.gamma
```

```
            elif self.beta is None: self.beta = 180.0 - self.alpha -
             ↪ self.gamma
            elif self.gamma is None: self.gamma = 180.0 - self.alpha -
             ↪ self.beta

            known_s_name = [s for s in ['a', 'b', 'c'] if getattr(self,
    ↪ s) is not None][0]
            known_s_val = getattr(self, known_s_name)
            known_a_val = getattr(self, {'a': 'alpha', 'b': 'beta', 'c':
    ↪ 'gamma'}[known_s_name])

            factor = known_s_val / math.sin(rad(known_a_val))
            if self.a is None: self.a = factor *
             ↪ math.sin(rad(self.alpha))
            if self.b is None: self.b = factor *
             ↪ math.sin(rad(self.beta))
            if self.c is None: self.c = factor *
             ↪ math.sin(rad(self.gamma))

    # ========================================================================
    # Methods derived from the formulas in Durandi, Werner, Baoswan
    # Dzung Wong, Markus Kriener, Hansruedi Künsch, Alfred Vogelsanger,
    # Jörg Waldvogel, Samuel Byland, u. a. Formeln, Tabellen, Begriffe:
    # Mathematik - Physik - Chemie. 8., Aktualisierte Auflage.
    # Herausgegeben von Verein Schweizerischer Mathematik- und
    # Physiklehrkräfte und Werner Durandi. Zürich: Orell Füssli Verlag,
    # 2022; pp. 86.
    # ========================================================================

    def get_semiperimeter(self):
        """Calculate the semiperimeter (s) of the triangle.

        Returns:
            float: The semiperimeter value.
        """
        # Formula: s = (a + b + c) / 2
        return (self.a + self.b + self.c) / 2

    def get_exterior_angle(self, vertex='alpha'):
        """Calculate the exterior angle for a given interior angle
         ↪ vertex.

        Args:
```

```
            vertex (str): The interior angle name ('alpha', 'beta',
    ↪ 'gamma'). Defaults to 'alpha'.

        Returns:
            float: The exterior angle in degrees.
        """
        # Formula: alpha' = 180° - alpha
        interior_angle = getattr(self, vertex)
        return 180.0 - interior_angle

    def get_area(self):
        """Calculate the area (A) of the triangle using Heron's formula.

        Returns:
            float: The area of the triangle.
        """
        # Formula: A = sqrt(s * (s - a) * (s - b) * (s - c)) (HERON)
        s = self.get_semiperimeter()
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))

    def get_altitude(self, base='a'):
        """Calculate the altitude (h) relative to a specific base side.

        Args:
            base (str): The base side identifier ('a', 'b', 'c').
    ↪ Defaults to 'a'.

        Returns:
            float: The height (altitude) on the chosen base side.
        """
        # Formula: h_a = c * sin(beta) or h_a = (2 / a) * Area
        base_val = getattr(self, base)
        return (2 * self.get_area()) / base_val

    def get_median(self, side='a'):
        """Calculate the length of the median (Seitenhalbierende) for a
         ↪ side.

        Args:
            side (str): The side identifier ('a', 'b', 'c'). Defaults to
    ↪ 'a'.

        Returns:
            float: The length of the median line.
```

```
        """
        # Formula: s_a = 0.5 * sqrt(2*b^2 + 2*c^2 - a^2)
        if side == 'a':
            return 0.5 * math.sqrt(2 * self.b**2 + 2 * self.c**2 -
             ↪  self.a**2)
        elif side == 'b':
            return 0.5 * math.sqrt(2 * self.a**2 + 2 * self.c**2 -
             ↪  self.b**2)
        else:
            return 0.5 * math.sqrt(2 * self.a**2 + 2 * self.b**2 -
             ↪  self.c**2)

    def get_angle_bisector(self, angle='alpha'):
        """Calculate the length of the internal angle bisector
         ↪  (Winkelhalbierende).

        Args:
            angle (str): The angle identifier ('alpha', 'beta',
↪  'gamma'). Defaults to 'alpha'.

        Returns:
            float: The length of the angle bisector line.
        """
        # Formula: w_alpha = sqrt(b * c * [1 - (a / (b + c))^2])
        if angle == 'alpha':
            return math.sqrt(self.b * self.c * (1 - (self.a / (self.b +
             ↪  self.c))**2))
        elif angle == 'beta':
            return math.sqrt(self.a * self.c * (1 - (self.b / (self.a +
             ↪  self.c))**2))
        else:
            return math.sqrt(self.a * self.b * (1 - (self.c / (self.a +
             ↪  self.b))**2))

    def get_incircle_radius(self):
        """Calculate the radius of the incircle (rho).

        Returns:
            float: The incircle radius.
        """
        # Formula: rho = A / s
        return self.get_area() / self.get_semiperimeter()

    def get_excircle_radius(self, side='a'):
```

```
        """Calculate the radius of the excircle (Ankreisradius rho_i)
        ↪ for a side.

        Args:
            side (str): The side identifier ('a', 'b', 'c') where the
    ↪ circle touches. Defaults to 'a'.

        Returns:
            float: The excircle radius.
        """
        # Formula: rho_a = A / (s - a)
        s = self.get_semiperimeter()
        side_val = getattr(self, side)
        return self.get_area() / (s - side_val)

    def get_circumcircle_radius(self):
        """Calculate the radius of the circumcircle (r).

        Returns:
            float: The circumcircle radius.
        """
        # Formula: r = (a * b * c) / (4 * A)
        return (self.a * self.b * self.c) / (4 * self.get_area())

    def __repr__(self):
        data = {k: round(v, 2) for k, v in self.__dict__.items() if not
    ↪ k.startswith('_')}
        return f"{self.__class__.__name__}({data})"
```

22.2.1 Inspecting the Triangle Instance

When printing a Python object or evaluating it directly inside a notebook cell, the default output is typically a generic string showing the object's class name and its memory address. To provide a clear and meaningful summary of a triangle's properties, we can implement the special `__repr__` method.

The implementation above dynamically inspects the object's internal attribute dictionary (`self.__dict__`). To ensure the output remains clean and legible, it automatically filters out any private helper attributes (those starting with an underscore) and rounds all public side lengths and angles to two decimal places. This guarantees that whenever a `Triangle` instance is invoked, it displays a neat, readable overview of its current geometric state.

To test this, in the cell below a Triangle `triangle` is instantiated and then printed to the command line.

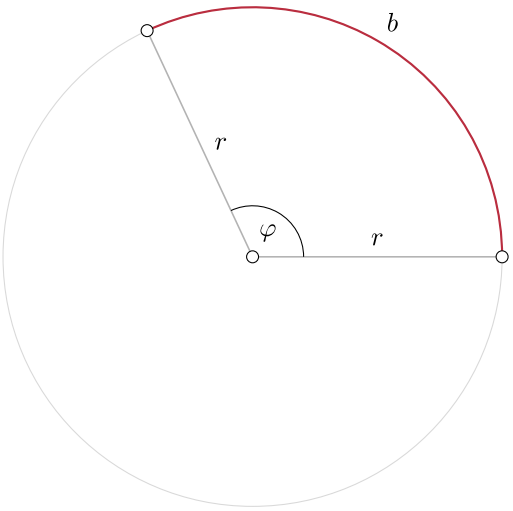
```
triangle = Triangle(3, 4, 5)
print(triangle)
```

```
Triangle({'a': 3, 'b': 4, 'c': 5, 'alpha': 36.87, 'beta': 53.13, 'gamma': 90.0})
```

22.3 Coding Task: Circle Class

After studying the `Triangle` class you should be able, to implement a Python class `Circle` on your own. To do so, you'll find the mathematical properties of a circle below.

Angles in Radian Measure



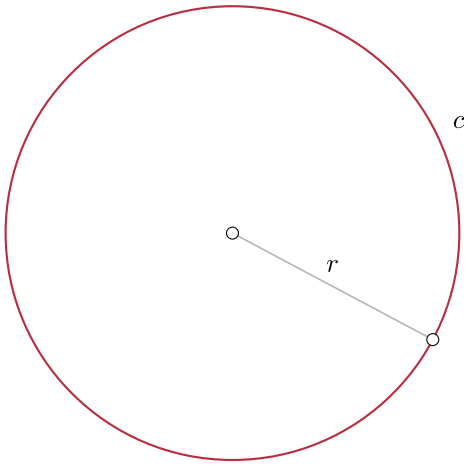
The radian measure $\widehat{\varphi}$ of an angle is the ratio of the length of the arc intersected by the angle φ on a circle to the radius of the circle. It is also denoted by $\operatorname{arc}\varphi$.

$$\widehat{\varphi} = \frac{b}{r} = \operatorname{arc}\varphi = \frac{\pi}{180^\circ}\varphi \iff \varphi = \frac{180^\circ}{\pi}\widehat{\varphi}$$

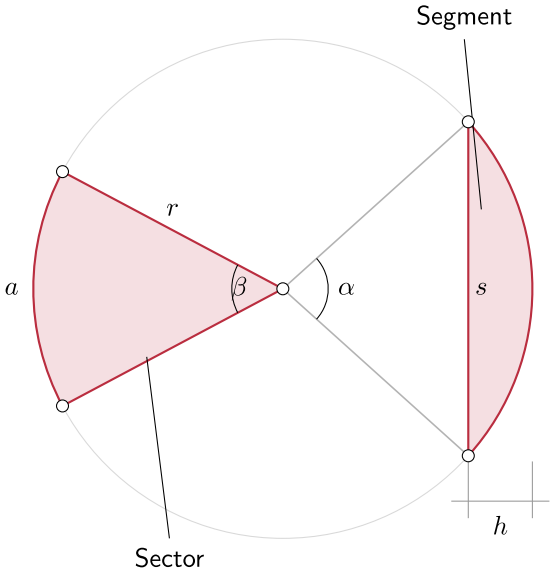
On the unit circle ($r = 1$), we have: $\widehat{\varphi} = b$
The unit of radian measure is the radian (rad). An angle of 1 rad corresponds to a circular arc that has the same length as the radius.[a]

[a] Durandi u. a., *Formeln, Tabellen, Begriffe*, p. 91.

To define a circle, either its radius or its circumference is sufficient. With those data the following elements can be calculated:



- *Circumference*: $c = 2\pi r$
- *Radius*: $r = \frac{c}{2\pi}$



- **Area**: $A = \pi r^2$
- **Arc Length**: $a = \widehat{\beta} r = \frac{\pi}{180^\circ} \beta r$
- **Sector area**: $A = \frac{\widehat{\beta} r^2}{2} = \frac{\pi \beta r^2}{360^\circ} = \frac{ar}{2}$
- **Chord length**: $s = 2r \sin \frac{\alpha}{2}$
- **Segment height**: $h = r\left(1 - \cos \frac{\alpha}{2}\right)$
- **Segment area**: $A = \frac{r^2}{2}(\arc \alpha - \sin \alpha)$

22.3.1 Implement Your own `Circle` Class

In the following cell you'll find the skeleton of a `Circle` class. Implement the missing parts. You don't need to implement your own `__repr__` method.

⚠ Computers calculate angles in **radians** (see box above). To convert angles given in degrees use the function `math.radians()`

```python
import math

class Circle:
    def __init__(self, r=None, c=None):
        """Initialises a Circle via either radius or circumference.

        Raises:
            ValueError: If neither or both parameters are provided.
        """
        if r is not None and c is not None:
            raise ValueError("Provide either radius 'r' OR circumference 'c', not both.")
        if r == None and c == None:
            raise ValueError("A circle must be initialised with either 'r' or 'c'.")

        if r is not None:
            if r < 0:
                raise ValueError("Radius cannot be negative.")
            c = 2 * math.pi * r
        elif c is not None:
            if c < 0:
                raise ValueError("Circumference cannot be negative.")
            r = c / (2 * math.pi)

        self.r = r
        self.c = c

    def get_area(self):
        return math.pi * self.r ** 2

    def get_arc_length(self, beta):
        """Calculates arc length for angle beta in degrees."""
        return (math.pi / 180) * beta * self.r

    def get_sector_area(self, beta):
        """Calculates sector area for angle beta in degrees."""
        a = self.get_arc_length(beta)
        return (a * self.r) / 2

```

```
    def get_chord_length(self, alpha):
        """Calculates chord length for angle alpha in degrees."""
        alpha_rad = math.radians(alpha)
        return 2 * self.r * math.sin(alpha_rad / 2)

    def get_segment_height(self, alpha):
        """Calculates segment height for angle alpha in degrees."""
        alpha_rad = math.radians(alpha)
        return self.r * (1 - math.cos(alpha_rad / 2))

    def get_segment_area(self, alpha):
        """Calculates segment area for angle alpha in degrees."""
        alpha_rad = math.radians(alpha)  # arc alpha is just the angle
    ↪ in radians!
        return (self.r**2 / 2) * (alpha_rad - math.sin(alpha_rad))
```

```
# test
circle = Circle(r=1)
alpha = 180
beta = 90
print(round(circle.get_area(), 2))
print(round(circle.get_arc_length(beta), 2))
print(round(circle.get_chord_length(beta), 2))
print(round(circle.get_segment_area(alpha), 2))
print(round(circle.get_segment_height(alpha), 2))
```

23 Functions: Exercises

23.1 Area of a trapezium

Given the height h and the parallel sides a and b of a trapezium, its area is calculated by the following formula:

$$A = \frac{a+b}{2} \cdot h$$

Implement this formula as a Python function in the cell below.

```
def trapezium_area(a: float, b: float, h: float) -> float:
    """Calculate the area of a trapezium.

    Args:
        a (float): The length of the first parallel side.
        b (float): The length of the second parallel side.
        h (float): The perpendicular height of the trapezium.

    Returns:
        float: The calculated area of the trapezium.
    """
    return ((a + b) / 2) * h

# test case

print(trapezium_area(2, 4, 6))
```

23.2 Volume of a right circular cylinder

The volume of a right circular cylinder is calculated by the formula

$$V = \pi \cdot r^2 \cdot h$$

where

- π is a constant with a value of 3.14,

- r denotes the radius of the base, and
- h denotes the height of the right circular cylinder.

Implement the formula as a Python function in the cell below.

```python
def get_cylinder_volume(r: float, h: float) -> float:
    """Calculate the volume of a cylinder.

    Args:
        r (float): The radius of the cylinder's base.
        h (float): The height of the cylinder.

    Returns:
        float: The calculated volume of the cylinder.
    """
    PI = 3.14 # Approximated value of pi to two decimal
              # places as a local constant
    return PI * r**2 * h

# test case

get_cylinder_volume(1, 1)
```

23.3 Progression decisions

The ordinance for promoting students of a Gymnasium in the canton of Zurich reads as follows:

> § 9. Bedingungen
> Die Bedingungen für die definitive Aufnahme bzw. Promotion sind erfüllt, wenn in allen Promotionsfächern, die im betreffenden Semester unterrichtet werden,
>
> a. die doppelte Summe aller Notenabweichungen von 4 nach unten nicht grösser ist als die Summe aller Notenabweichungen von 4 nach oben und
>
> b. nicht mehr als drei Noten unter 4 erteilt werden.

Implement the check of the conditions for a progression as a Python function in the cell below.

```python
def does_the_student_pass(grades: list[float]) -> bool:
    """Determine whether a student qualifies for definitive promotion.

    This function implements specific academic regulations based on a
```

```
    grading scale where 4.0 is the passing mark. A student passes if
    both of the following conditions are met:
    1. Twice the sum of all grade deviations below 4.0 is not greater
       than the sum of all grade deviations above 4.0.
    2. No more than three grades are below 4.0.

    Args:
        grades (list[float]): A list of grades achieved in the
            subjects relevant for promotion in the semester.

    Returns:
        bool: True if the promotion conditions are fulfilled,
              False otherwise.
    """
    sub_fours = [g - 4 for g in grades if g < 4]
    sup_fours = [g - 4 for g in grades if g >= 4]
    aggregated_sub_fours = abs(2 * sum(sub_fours))
    aggregated_sup_fours = sum(sup_fours)
    compensation = aggregated_sub_fours <= aggregated_sup_fours
    number_sub_fours = len(sub_fours)
    return compensation and (number_sub_fours <= 3)

# test cases
progress_secured = [5, 4.5, 4.5, 3.5, 5, 4, 4, 4, 4.5, 5, 5, 4]
failed_a = [5, 4.5, 4.5, 3.5, 4, 4, 4, 3, 4.5, 4, 4, 4]
failed_b = [5, 5, 4.5, 3.5, 4, 3.5, 4, 3, 4.5, 3.5, 5, 5.5]

print(f'The student progresses:
 ↪  {does_the_student_pass(progress_secured)}')
print(f'The student progresses a): {does_the_student_pass(failed_a)}')
print(f'The student progresses b): {does_the_student_pass(failed_b)}')
```

24 Triangulation: Inheriting from Triangle

Land surveying has traditionally rested on the application of trigonometric functions to triangles — which is why it is also called *triangulation*. In this chapter we survey a distance across inaccessible terrain and, in doing so, revisit the whole progression of this course:

1. **Helper functions** that call one another (as in Kapitel 21.1, *Functions within functions*).
2. **Bundling** those helpers into a class (as with `TriangleSSS`, see Kapitel 22.1).
3. **Inheritance** — the new idea: a survey triangle *is a* `Triangle`, so it can inherit everything the `Triangle` class already knows and add only what surveying needs.

Trigonometric Fundamentals

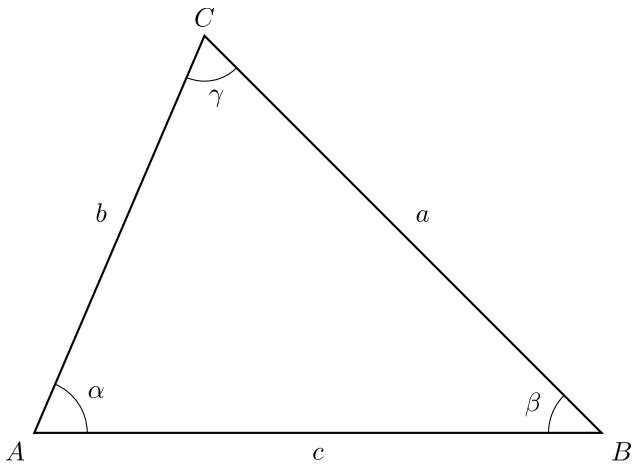


Abbildung 24.1: General Triangle.

In a general triangle (Abbildung 24.1) the sine rule and the cosine rule hold:
Sine rule

$$\frac{a}{\sin\alpha} = \frac{b}{\sin\beta} = \frac{c}{\sin\gamma}$$

Cosine rule

$$c^2 = a^2 + b^2 - 2\,a\,b\,\cos\gamma$$

24.1 The Surveying Problem

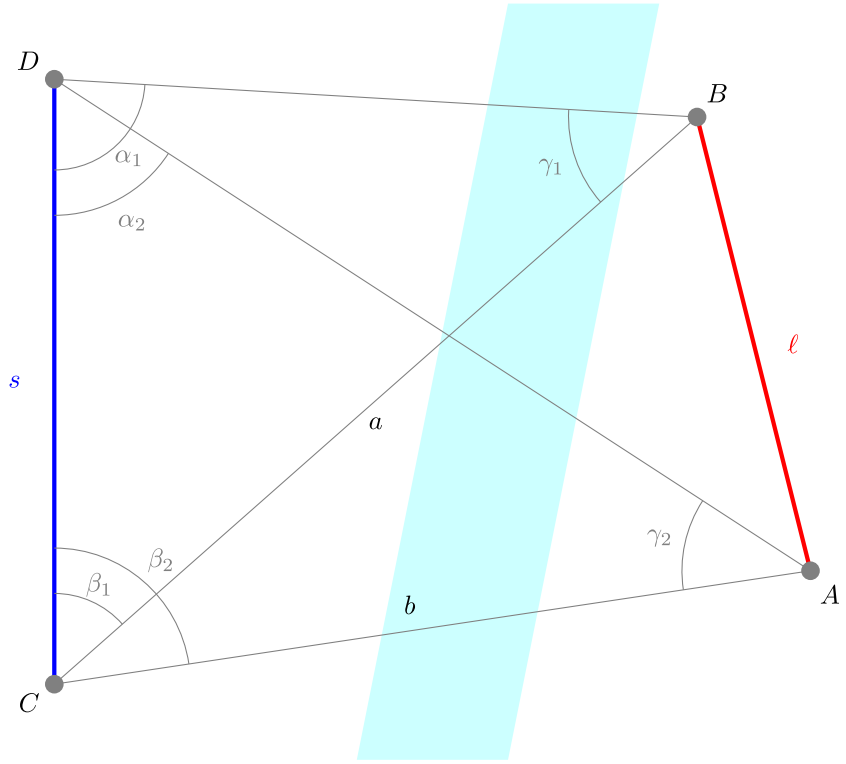


Abbildung 24.2: Schematic diagram.

We are looking for the distance $\overline{AB}$ (ℓ) across inaccessible terrain (Abbildung 24.2). What we *can* measure is the baseline $\overline{CD}$ (s) and the angles α_1, α_2 (at point D) and β_1, β_2 (at point C).

The idea is to link the baseline $s = \overline{CD}$ to the sought distance by means of **two auxiliary triangles**. From triangle $C\,D\,B$ we obtain the leg $\overline{CB}$, from triangle $C\,D\,A$ the leg $\overline{CA}$. Both share vertex C; the angle enclosed between them is $\beta_2 - \beta_1$. The target distance then follows from the cosine rule in triangle $C\,A\,B$:

$$\ell = \sqrt{a^2 + b^2 - 2ab\cos(\beta_2 - \beta_1)}$$

24.2 Stage 1 — Helper Functions

We begin exactly as in the Heron example: a *main* function `triangulation()` that calls several *helper* functions. This keeps each helper small, readable and independently testable.

Three helpers are needed:

- `angle_converter()` — bearings may be read from a compass calibrated in degrees, in artillery **mils** (A‰) or in **gon**. This helper normalises them to degrees.
- `angle_diff()` — each interior angle is the *modular* difference of two compass bearings. Using `% 360` automatically handles the case where the two readings straddle north.
- `sine_rule()` — returns the side opposite a given angle.

Default Values and Modular Subtraction

`angle_diff()` relies on modular arithmetic: with a positive divisor, Python's `%` operator always returns a value in $[0, 360)$, so the classical extra step *"if the result is negative, add 360"* becomes unnecessary. For example $40° - 110° = -70 \equiv 290 \pmod{360}$.

```
import math
```

```
def angle_converter(angle: float,
                    mil: bool = False,
                    gon: bool = False) -> float:
    """Normalises an angle to degrees.

    Supports input in degrees (default), artillery mils or gon. Only one
    alternative mode may be active at a time.

    Args:
        angle (float): The angle value to convert.
        mil (bool, optional): If True,
            interpret `angle` as artillery mils
            (6400 mil = 360°). Defaults to False.
        gon (bool, optional): If True, interpret `angle` as gon
            (400 gon = 360°). Defaults to False.

    Returns:
        float: The angle expressed in degrees.

    Raises:
        ValueError: If both `mil` and `gon` are True.
    """
    if mil and gon:
        raise ValueError("Only 'mil=True' (artillery) OR \
                         'gon=True' may be selected.")

    if mil:
        return angle * (360 / 6400)
    elif gon:
        return angle * (360 / 400)
```

```
    else:
        return angle
```

```
def angle_diff(bearing1: float, bearing2: float) -> float:
    """Modular difference (bearing1 - bearing2) modulo 360 degrees.

    Automatically accounts for crossing north by mapping negative
    differences into the positive cycle [0, 360).

    Args:
        bearing1 (float): The first compass bearing, in degrees.
        bearing2 (float): The bearing to subtract, in degrees.

    Returns:
        float: The interior angle in the range [0, 360) degrees.
    """
    return (bearing1 - bearing2) % 360
```

```
def sine_rule(length: float,
              opposite_angle: float,
              ref_angle: float) -> float:
    """Calculates a side of a triangle using the sine rule.

    Determines the side opposite `opposite_angle`, starting from the
    known side `length` opposite `ref_angle`:

        required side = length * sin(opposite_angle) / sin(ref_angle)

    Args:
        length (float): The known side.
        opposite_angle (float): Angle opposite the side we are looking
        for, in degrees.
        ref_angle (float): Angle opposite the known side `length`, in
        degrees.

    Returns:
        float: The length of the side we are looking for.
    """
    return (length * math.sin(math.radians(opposite_angle)) /
            math.sin(math.radians(ref_angle)))
```

With the helpers in place, the main function forms the four interior angles from the eight bearings, builds the two auxiliary triangles via the sine rule, and closes with the cosine

rule.

```
def triangulation(s: float,
                  alpha1_a: float, alpha1_b: float,
                  alpha2_a: float, alpha2_b: float,
                  beta1_a: float, beta1_b: float,
                  beta2_a: float, beta2_b: float,
                  mil: bool = False, gon: bool = False) -> float:
    """Calculates an inaccessible distance by triangulation from compass
     ↪ bearings.

    From the accessible baseline CD (`s`) and four pairs of azimuth
    bearings, two auxiliary triangles are formed. The sine rule yields
    the legs CB and CA; the cosine rule in triangle C-A-B then gives
    the target distance, the enclosed angle at C being beta2 - beta1.

    Args:
        s (float): The measured, accessible baseline (CD).
        alpha1_a, alpha1_b (float): Bearings for partial
        angle alpha 1 (at D).
        alpha2_a, alpha2_b (float): Bearings for partial
        angle alpha 2 (at D).
        beta1_a, beta1_b (float): Bearings for partial
        angle beta 1 (at C).
        beta2_a, beta2_b (float): Bearings for partial
        angle beta 2 (at C).
        mil (bool, optional): Interpret all bearings as mils.
        Defaults to False.
        gon (bool, optional): Interpret all bearings as gon.
        Defaults to False.

    Returns:
        float: The length of the inaccessible target distance (l).
    """
    # Each pair of bearings gives one interior angle (modular
    # difference), normalised to degrees.
    bearings = [alpha1_a, alpha1_b, alpha2_a, alpha2_b,
                beta1_a, beta1_b, beta2_a, beta2_b]
    partial = [angle_diff(bearings[i], bearings[i + 1])
               for i in range(0, len(bearings), 2)]
    alpha1, alpha2, beta1, beta2 = (angle_converter(a, mil, gon)
                                    for a in partial)

    # Auxiliary triangles on the baseline s: gamma lies opposite s.
```

```
    gamma1 = 180 - (alpha1 + beta1)
    gamma2 = 180 - (alpha2 + beta2)

    # Sine rule -> the legs CB and CA (opposite alpha1 and alpha2).
    side_cb = sine_rule(s, alpha1, gamma1)
    side_ca = sine_rule(s, alpha2, gamma2)

    # Cosine rule in triangle C-A-B; enclosed angle at
    # C is beta2 - beta1.
    return math.sqrt(side_cb**2 + side_ca**2
                     - 2 * side_cb * side_ca
                     * math.cos(math.radians(beta2 - beta1)))
```

```
# Worked example: alpha1 = 50°, alpha2 = 30°, beta1 = 60°, beta2 = 100°
distance = triangulation(100,
                         50, 0, 30, 0,   # alpha1, alpha2 at D
                         60, 0, 100, 0)  # beta1, beta2 at C
print(round(distance, 2))
```

```
52.48
```

24.3 Stage 2 — Bundling into a Class

The helper functions work, but the measured data (baseline and bearings) travel through the code as a long list of loose arguments. Just as we moved from free functions to the `TriangleSSS` class, we can **bundle** the data and the calculation into a single object. The helpers become *private methods* (leading underscore), mirroring `_get_semiperimeter()`.

```
class Survey:
    """Bundles a triangulation measurement into a single object.

    Attributes:
        baseline (float): The accessible baseline CD (s).
        bearings_D (tuple): Four bearings measured at
        D (alpha1, alpha2).
        bearings_C (tuple): Four bearings measured at C (beta1, beta2).
    """

    def __init__(self, baseline, bearings_D, bearings_C,
                 mil=False, gon=False):
        """Stores the measured quantities on the instance.
```

```
14
15          Args:
16              baseline (float): The accessible baseline CD.
17              bearings_D (tuple[float, float, float, float]):
18              Bearing pairs for alpha1 and alpha2, measured at D.
19              bearings_C (tuple[float, float, float, float]):
20              Bearing pairs for beta1 and beta2, measured at C.
21              mil (bool, optional): Interpret bearings as mils.
22              Defaults to False.
23              gon (bool, optional): Interpret bearings as gon.
24              Defaults to False.
25          """
26          self.baseline = baseline
27          self.bearings_D = bearings_D
28          self.bearings_C = bearings_C
29          self.mil = mil
30          self.gon = gon
31
32      def _angle(self, bearing1, bearing2):
33          """Private helper: interior angle from two bearings,
34             in degrees."""
35          return angle_converter(angle_diff(bearing1, bearing2),
36                                 self.mil, self.gon)
37
38      def get_distance(self):
39          """Calculates the inaccessible target distance AB.
40
41          Returns:
42              float: The length of the target distance (l).
43          """
44          alpha1 = self._angle(self.bearings_D[0], self.bearings_D[1])
45          alpha2 = self._angle(self.bearings_D[2], self.bearings_D[3])
46          beta1 = self._angle(self.bearings_C[0], self.bearings_C[1])
47          beta2 = self._angle(self.bearings_C[2], self.bearings_C[3])
48
49          side_cb = sine_rule(self.baseline, alpha1, 180 -
50                              (alpha1 + beta1))
51          side_ca = sine_rule(self.baseline, alpha2, 180 -
52                              (alpha2 + beta2))
53
54          return math.sqrt(side_cb**2 + side_ca**2
55                           - 2 * side_cb * side_ca *
56                          math.cos(math.radians(beta2 - beta1)))
```

```
survey = Survey(baseline=100,
                bearings_D=(50, 0, 30, 0),
                bearings_C=(60, 0, 100, 0))
print(round(survey.get_distance(), 2))
```

```
52.48
```

24.4 Stage 3 — Inheritance: an Auxiliary Triangle *is a* Triangle

Look closely at what `get_distance()` does: it builds two triangles on the baseline and reads a leg off each. But we *already have* a fully featured `Triangle` class from the previous chapter — it can solve a triangle from a side and two angles (the **ASA** case) and exposes `get_area()`, `get_semiperimeter()`, `__repr__` and more.

Rather than re-deriving the legs by hand, we let a new class **inherit** from `Triangle`. An auxiliary survey triangle *is a* triangle — the textbook signal for inheritance.

> **The `is-a` Relationship and `super()`**
>
> When a class `AuxiliaryTriangle(Triangle)` is written, `AuxiliaryTriangle` becomes a **subclass** (child) of `Triangle`. It automatically gains every method of the parent. Inside `__init__`, the call `super().__init__(...)` runs the parent's constructor — here the parent even *solves* the triangle for us, filling in all missing sides and angles. We then add only the one small method surveying needs.
> Contrast this with *composition* (a `Survey` that merely *has* two triangles): inheritance is the right tool when the new type genuinely **is a** specialised kind of the old one.

First, the `Triangle` class from the previous chapter must be available. In your book this cell is already present; it is repeated here so the notebook runs on its own.

```
class Triangle:
    """Represents a triangle, solvable from valid congruence criteria
     ↪  (abridged).

    Only the ASA/SAA and SSS paths needed for surveying are shown here;
    the full class with all four congruence cases is developed in the
    previous chapter.
    """

    def __init__(self, a=None, b=None, c=None,
                 alpha=None, beta=None, gamma=None):
        self.a, self.b, self.c = a, b, c
        self.alpha, self.beta, self.gamma = alpha, beta, gamma

```

```
        given_sides = [k for k, v in
                       [('a', a), ('b', b), ('c', c)] if v is not None]
        given_angles = [k for k, v in
                        [('alpha', alpha), ('beta', beta), ('gamma',
    gamma)]
                        if v is not None]
        if len(given_sides) + len(given_angles) != 3:
            raise ValueError("A triangle must be defined by exactly \
                              3 parameters.")

        self._solve_triangle()

    def _solve_triangle(self):
        """Fills in all missing sides and angles
           (SSS and ASA/SAA paths)."""
        def rad(d): return math.radians(d)
        def deg(r): return math.degrees(r)

        if self.a and self.b and self.c:  # SSS
            self.alpha = deg(math.acos((self.b**2 +
                                        self.c**2 - self.a**2)
                                       / (2 * self.b * self.c)))
            self.beta = deg(math.acos((self.a**2 + self.c**2 -
                                       self.b**2)
                                      / (2 * self.a * self.c)))
            self.gamma = 180.0 - self.alpha - self.beta

        elif sum(s is not None for s in [self.a, self.b, self.c]) == 1:
            # ASA / SAA
            if self.alpha is None:
                self.alpha = 180.0 - self.beta - self.gamma
            elif self.beta is None:
                self.beta = 180.0 - self.alpha - self.gamma
            elif self.gamma is None:
                self.gamma = 180.0 - self.alpha - self.beta

            known_s = [s for s in ['a', 'b', 'c']
                       if getattr(self, s) is not None][0]
            known_a = {'a': 'alpha', 'b': 'beta', 'c': 'gamma'}[known_s]
            factor = getattr(self, known_s) / math.sin(rad(getattr(self,
    known_a)))
            if self.a is None:
                self.a = factor * math.sin(rad(self.alpha))
            if self.b is None:
```

```
                self.b = factor * math.sin(rad(self.beta))
            if self.c is None:
                self.c = factor * math.sin(rad(self.gamma))

    def get_semiperimeter(self):
        return (self.a + self.b + self.c) / 2

    def get_area(self):
        s = self.get_semiperimeter()
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))

    def __repr__(self):
        data = {k: round(v, 2) for k, v in self.__dict__.items()
                if not k.startswith('_')}
        return f"{self.__class__.__name__}({data})"
```

Now the subclass. Each auxiliary triangle sits on the baseline s (side c) with the two measured base angles α (at D) and β (at C) — precisely the **ASA** case. `super().__init__()` hands these to the parent, which solves the triangle completely. The leg we want, $\overline{CB}$ resp. $\overline{CA}$, is the side opposite α, i.e. side a.

```
class AuxiliaryTriangle(Triangle):
    """A survey triangle on the shared baseline — a specialised
     ↪ Triangle.

    Constructed from the baseline and the two base angles
    (the ASA case), it inherits everything Triangle can do and
    adds one surveying method.
    """

    def __init__(self, baseline: float, alpha: float, beta: float):
        """Builds the auxiliary triangle from baseline and base angles.

        Args:
            baseline (float): The shared baseline s (becomes side c).
            alpha (float): Base angle at D, opposite the leg we seek.
            beta (float): Base angle at C.
        """
        # ASA: side c = baseline, adjacent angles alpha and beta.
        super().__init__(c=baseline, alpha=alpha, beta=beta)

    def leg_from_C(self) -> float:
        """The leg from C to the far target point
           (opposite alpha, i.e. side a)."""
```

```
        return self.a
```

The `Survey` class becomes noticeably simpler: it no longer computes legs itself but delegates to two `AuxiliaryTriangle` objects, then closes with the cosine rule.

```
class TriangulationSurvey:
    """Chains two AuxiliaryTriangles on one baseline to find AB."""

    def __init__(self, baseline, bearings_D, bearings_C,
                 mil=False, gon=False):
        self.baseline = baseline
        self.bearings_D = bearings_D
        self.bearings_C = bearings_C
        self.mil = mil
        self.gon = gon

    def _angle(self, bearing1, bearing2):
        return angle_converter(angle_diff(bearing1, bearing2),
                               self.mil, self.gon)

    def get_distance(self):
        """Calculates the target distance via two
           inherited triangles."""
        alpha1 = self._angle(self.bearings_D[0], self.bearings_D[1])
        alpha2 = self._angle(self.bearings_D[2], self.bearings_D[3])
        beta1 = self._angle(self.bearings_C[0], self.bearings_C[1])
        beta2 = self._angle(self.bearings_C[2], self.bearings_C[3])

        triangle_cdb = AuxiliaryTriangle(self.baseline, alpha1, beta1)
        triangle_cda = AuxiliaryTriangle(self.baseline, alpha2, beta2)

        side_cb = triangle_cdb.leg_from_C()
        side_ca = triangle_cda.leg_from_C()

        return math.sqrt(side_cb**2 + side_ca**2
                         - 2 * side_cb * side_ca *
                         math.cos(math.radians(beta2 - beta1)))
```

```
survey = TriangulationSurvey(baseline=100,
                             bearings_D=(50, 0, 30, 0),
                             bearings_C=(60, 0, 100, 0))
print(round(survey.get_distance(), 2))
```

```
52.48
```

Because `AuxiliaryTriangle` *is a* `Triangle`, every inherited method just works — and `isinstance()` confirms the relationship.

```python
aux = AuxiliaryTriangle(100, 50, 60)
print(aux)                                # inherited __repr__
print("Area:", round(aux.get_area(), 2))  # inherited get_area()
print("Is a Triangle:", isinstance(aux, Triangle))
```

```
AuxiliaryTriangle({'a': 81.52, 'b': 92.16, 'c': 100, 'alpha': 50, 'beta': 60, 'gamma': 70.0}
Area: 3529.95
Is a Triangle: True
```

24.5 Coding Task

Coding Task: A Height by Triangulation

The same principle measures the **height of an inaccessible tower**. From two points P and Q on level ground, a distance d apart and in line with the tower's base, you measure the elevation angles φ_P and φ_Q to the top.
Implement a class `TowerSurvey` that inherits from `Triangle`. Build the auxiliary triangle PQT (where T is the top) using `super().__init__()` with the appropriate ASA data, inherit the sine-rule solution for the slant distance $\overline{QT}$, and add one method `get_height()` that returns the tower height. Reuse — do not re-derive — the machinery you already inherit.

25 Interest-Bearing Accounts: Inheriting from Account

Double-entry bookkeeping records every business transaction as an *entry* that debits one account and credits another. In this chapter we build a small ledger in Python and, in doing so, revisit the whole progression of this course:

1. **A data-carrying class** that models a single booking (`Entry`).
2. **Bundling** state and behaviour into a class that keeps a running balance (`Account`).
3. **Inheritance** — the new idea: a current account *is an* `Account`, so it can inherit everything the `Account` class already knows and add only what interest-bearing accounts need.

Bookkeeping Fundamentals

Every entry touches two accounts. By convention an account's balance (its *Saldo*) moves according to which side the entry lands on:

- a **debit** on the account increases its balance;
- a **credit** on the account decreases it.

The *interest* on a balance held over a period follows the simple day-count formula

$$\text{interest} = \text{balance} \times \frac{\text{rate}}{360} \times \text{days},$$

where the year is taken as 360 days (the *30/360* convention common in Swiss practice).

25.1 The Design at a Glance

Before writing any code, it helps to picture where we are heading. Abbildung 25.1 is a **UML class diagram** of the three classes in this chapter and how they relate.

25.2 The Design at a Glance

Before writing any code, it helps to picture where we are heading. Abbildung 25.1 is a **UML class diagram** of the three classes in this chapter and how they relate.

Two relationships appear here, and telling them apart is the whole point of the chapter:

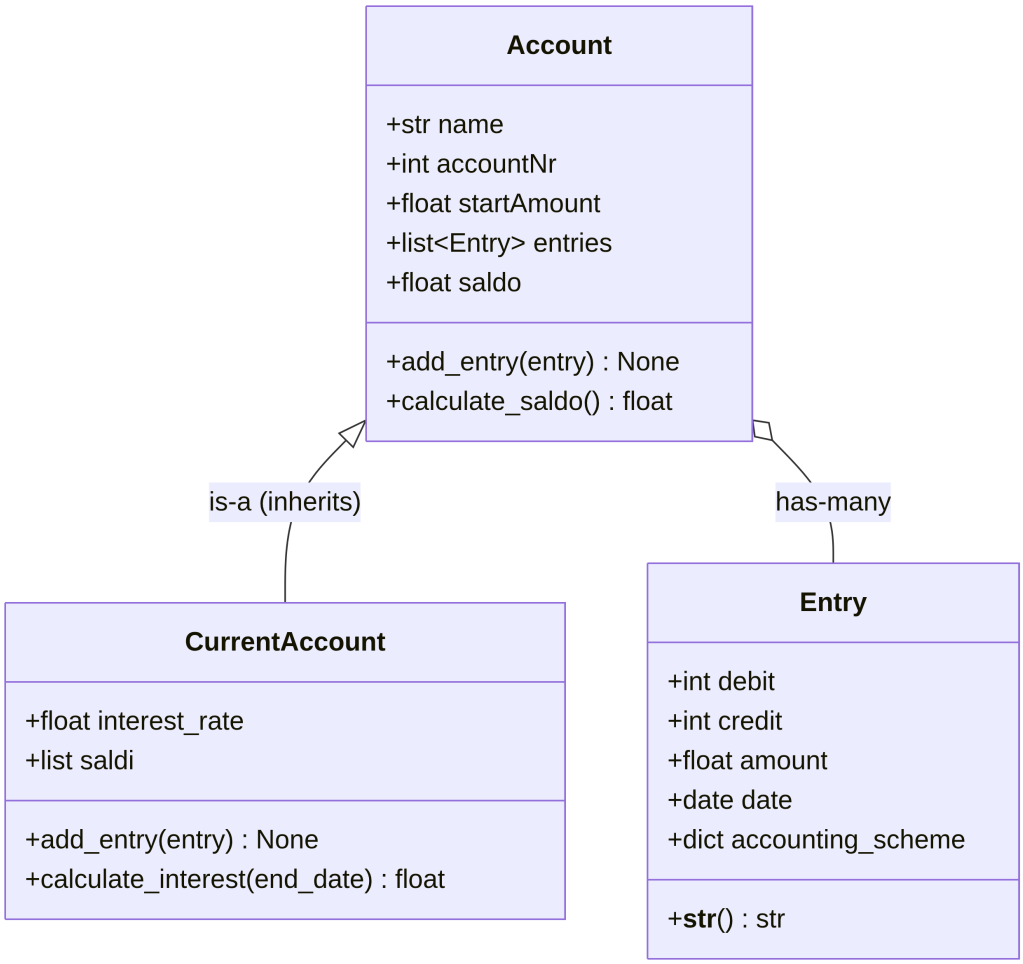


Abbildung 25.1: UML class diagram: CurrentAccount inherits from Account; each Account holds Entry objects.

- The **hollow triangle** arrow (`Account <|-- CurrentAccount`) is *inheritance*: a `CurrentAccount` **is an** `Account`. It inherits every attribute and method of the parent and adds `interest_rate`, `saldi` and `calculate_interest`. Note that `add_entry` appears in *both* boxes — the subclass **overrides** it.
- The **hollow diamond** arrow (`Account o-- Entry`) is *aggregation*: an `Account` **has** a list of `Entry` objects. This is composition, not inheritance — an account is not a kind of entry.

Reading a UML Class Diagram

Each box has three compartments: the class **name**, its **attributes** (data) and its **methods** (behaviour). A `+` marks a public member. The arrow *type* encodes the relationship — an *is-a* link (inheritance) uses a hollow triangle pointing at the parent, whereas a *has-a* link (aggregation) uses a hollow diamond at the containing class.

25.3 Stage 1 — A Class for a Single Entry

We begin with the smallest unit of the ledger: one booking. An `Entry` stores the debit account number, the credit account number, an amount and a date. It also carries the chart of accounts (*Kontenrahmen*) so that it can print itself in a human-readable form.

Dunder Methods

Methods whose names begin and end with two underscores — `__init__`, `__str__`, `__len__`, `__eq__` — are called **dunder** methods (from *double underscore*). They are also known as *magic* or *special* methods. The double underscores mark them as part of Python's own protocol, so they never clash with method names you choose yourself.
You rarely call a dunder method by name. Instead, Python invokes it for you when you use ordinary syntax or a built-in function: creating an `Entry` runs `__init__`; `print(entry)` and `str(entry)` run `__str__`; `len(x)` runs `__len__`; and `x == y` runs `__eq__`. (We do not need `__len__` or `__eq__` here — they only show the pattern.)
Defining a dunder method is how a class connects to that syntax and lets its objects behave like built-in types. In this chapter we implement two. `__init__` sets a new object up. `__str__` must return a string; defining it lets an `Entry` describe itself — date, both account names and the amount — instead of showing the default `<Entry object at 0x…>`.

What `self.` Does

A class bundles data and the methods that act on it. The class definition is only a *template*; from it, individual objects are created at run time, and each object carries its own set of attribute values.
The first parameter of every method refers to the object the method was called on. It is named `self` by convention, not by language rule — but always write `self`. When Python runs `first_entry = Entry(1000, 1020, 50.00, date(2026, 8,`

28)), it runs `__init__` with `self` bound to the new object. Later, `print(first_entry)` calls `first_entry.__str__()` with `self` bound to that same object.
So inside `__init__`, `self.debit = debit` stores the value *on that one object*, and inside `__str__`, `self.debit` and `self.accounting_scheme` read the data back from *the same* object. Any value the object should still have after the method finishes must be stored on `self`. A plain local name like `text` in `__str__` is gone as soon as the method returns.

```
from datetime import date
```

```
class Entry:
    """A single double-entry booking (one debit, one credit).

    Attributes:
        debit (int): Account number debited.
        credit (int): Account number credited.
        amount (float): The amount booked.
        date (date): The date of the booking.
    """

    def __init__(self, debit: int,
                 credit: int,
                 amount: float,
                 date: date) -> None:
        self.debit = debit
        self.credit = credit
        self.amount = amount
        self.date = date
        # Abridged chart of accounts (Kontenrahmen KMU) for readable
        ↪ output.
        self.accounting_scheme = {
            1000: "Kasse",
            1020: "Bankguthaben",
            2000: "Verbindlichkeiten aus Lieferungen und Leistungen
            ↪ (Kreditoren)",
            6950: "Finanzertrag",
        }

    def __str__(self) -> str:
        text = (
            f"{self.date.strftime('%d %b %Y')}: "
            f"{self.accounting_scheme[self.debit]} "
            f"({self.debit}) - {self.accounting_scheme[self.credit]} "
            f"({self.credit}) {self.amount}"
```

```
        )
        return " ".join(text.split())
```

A first booking: 50.00 moved from *Bankguthaben* (1020) to *Kasse* (1000). Printing the object triggers `__str__`.

```
first_entry = Entry(1000, 1020, 50.00, date(2026, 8, 28))
print(first_entry)
```

25.4 Stage 2 — Bundling into an Account

A single entry is of little use on its own. An **account** collects the entries that touch it and can compute its balance at any time. We bundle that state (name, number, opening balance, list of entries) and the behaviour (`add_entry`, `calculate_saldo`) into one class.

`calculate_saldo` walks through every entry: an entry that **debits** this account raises the balance, one that **credits** it lowers the balance, and entries touching neither are skipped.

```
class Account:
    """A ledger account that collects entries and computes its balance.

    Attributes:
        name (str): Human-readable account name.
        accountNr (int): The account number.
        startAmount (float): Opening balance.
        entries (list[Entry]): All entries booked to this account.
        saldo (float | None): Most recently computed balance.
    """

    def __init__(self,
                 name: str,
                 accountNr: int,
                 startAmount: float) -> None:
        self.name = name
        self.accountNr = accountNr
        self.startAmount = startAmount
        self.entries = []
        self.saldo = None

    def add_entry(self, entry: Entry) -> None:
        """Appends an entry to the account and reports it.

```

```
        Args:
            entry (Entry): The booking to add.
        """
        self.entries.append(entry)
        print(f'Entry "{entry}" added.')

    def calculate_saldo(self) -> float:
        """Computes the balance from the opening amount and all entries.

        A debit on this account increases the balance; a credit
        decreases it.

        Returns:
            float: The resulting balance.
        """
        saldo = self.startAmount
        for entry in self.entries:
            if entry.debit == self.accountNr:
                saldo += entry.amount
            elif entry.credit == self.accountNr:
                saldo -= entry.amount
            else:
                continue
        print(f'Saldo: {saldo}')
        self.saldo = saldo
        return saldo
```

We open the cash account 1000 with 10.00, book two entries and read off the balance.

```
a1000 = Account('Cash', 1000, 10.00)
a1000.add_entry(Entry(1000, 1020, 50.0, date(2026, 8, 28)))
a1000.add_entry(Entry(2000, 1000, 5.00, date(2026, 8, 28)))
a1000.calculate_saldo()
```

25.5 Stage 3 — Inheritance: a Current Account *is an* Account

A **current account** (*Kontokorrent*) behaves exactly like an ordinary account — it collects entries and has a balance — but it does something extra: it earns **interest**. That is the textbook signal for inheritance: a current account *is an* account, plus a little more. In the UML diagram (Abbildung 25.1) this is the hollow-triangle link from `CurrentAccount` to `Account`.

Rather than copying all of `Account`, we let a new class **inherit** from it and add only what

interest requires.

The `is-a` Relationship and `super()`

Writing `class CurrentAccount(Account)` makes `CurrentAccount` a **subclass** (child) of `Account`. It automatically gains every method of the parent — `add_entry`, `calculate_saldo` and the constructor. Inside its own `__init__`, the call `super().__init__(...)` runs the parent's constructor, so `name`, `accountNr`, `startAmount`, `entries` and `saldo` are set up for us. We then add just the new attributes `interest_rate` and `saldi`.
Contrast this with *composition* (the hollow-diamond `has-a` link between `Account` and `Entry`): inheritance is the right tool when the new type genuinely **is a** specialised kind of the old one.

25.5.1 A First Version

We build the subclass step by step. The constructor takes one extra argument, `interest_rate`, hands the rest to the parent via `super().__init__(...)`, and keeps a dated history of balances (`saldi`) so that interest can later be computed period by period. `add_entry` is **overridden**: it first calls `super().add_entry(entry)` to keep the inherited behaviour, then records the new balance together with the entry's date.

For `calculate_interest` we write the calculation out in full, in the most direct way we can think of — index by index. It works; we will scrutinise it afterwards.

```
class CurrentAccount(Account):
    """An interest-bearing account – first version.

    Inherits entry handling and balance calculation from Account and
    adds an interest rate together with a dated history of balances.

    Attributes:
        interest_rate (float): Annual interest rate (e.g. 0.05 for 5%).
        saldi (list[tuple[float, date]]): Balance history as
            (balance, date) pairs, opened with the start amount.
    """

    def __init__(self,
                 name: str,
                 accountNr: int,
                 startAmount: float,
                 interest_rate: float) -> None:
        super().__init__(name, accountNr, startAmount)  # parent sets up
        ↪ the account
```

```
        self.interest_rate = interest_rate              # the new attributes
        self.saldi = []
        self.saldi.append((startAmount, date(2026, 1, 1)))

    def add_entry(self, entry: Entry) -> None:
        """Extends Account.add_entry by recording the new dated balance."""
        super().add_entry(entry)                                #  inherited behaviour
        self.saldi.append((self.calculate_saldo(), entry.date))  # plus history

    def calculate_interest(self, end_date=date(2026, 12, 31)) -> float:
        """Computes 30/360 interest across the balance history.

        Each balance earns interest over the number of days it was held,
        i.e. until the next balance change (and finally until end_date).
        If end_date is the year-end, the total is booked as Finanzertrag.
        """
        number_of_days = []
        interests = []
        end_date = end_date
        i = 0
        saldi = [saldo[0] for saldo in self.saldi]
        day_list = [saldo[1] for saldo in self.saldi]
        while i < (len(day_list) - 1):
            d = (day_list[i+1] - day_list[i]).days
            number_of_days.append(d)
            i += 1
        rest_days = (end_date - day_list[-1]).days
        number_of_days.append(rest_days)
        for i in range(len(saldi)):
            interest = (saldi[i] * self.interest_rate / 360) * number_of_days[i]
            interests.append(interest)
        if end_date == date(2026, 12, 31):
            entry = Entry(1020, 6950, sum(interests), end_date)
            self.add_entry(entry)
        return sum(interests)
```

We open a bank current account 1020 with 100.00 at 5 % interest, book a movement in March, and compute the interest to the year-end.

```
bank = CurrentAccount('Bank', 1020, 100.00, 0.05)
bank.add_entry(Entry(1020, 1000, 150.00, date(2026, 3, 1)))
print(bank.calculate_interest())
```

25.5.2 Refactoring `calculate_interest`

The first version is correct, but it does more bookkeeping of its own than the problem requires. Reading it closely, a few things stand out:

- Two accumulator lists (`number_of_days`, `interests`) are filled with `append` inside loops.
- A manual index `i` and a `while` loop walk through consecutive dates — a classic place for off-by-one errors.
- The line `end_date = end_date` does nothing.

Python lets us say the same thing more directly. The days between consecutive balances are just the differences of **adjacent pairs**, which `zip(day_list, day_list[1:])` produces without any index. Both loops then collapse into **list comprehensions**, and the intent — *"each balance, times its days, times the daily rate"* — becomes readable in one line.

> **Pairing Adjacent Elements with `zip`**
>
> `zip(seq, seq[1:])` pairs each element with its successor: from `[d0, d1, d2]` it yields `(d0, d1)` and `(d1, d2)`. Iterating over those pairs replaces the manual index arithmetic `day_list[i+1] - day_list[i]` and removes the risk of mis-counting the loop bounds.

The behaviour is identical — only `calculate_interest` changes; everything inherited from `Account` stays untouched.

```
class CurrentAccount(Account):
    """An interest-bearing account – refactored version.

    Identical behaviour to the first version; only calculate_interest
    is rewritten using zip and list comprehensions.
    """

    def __init__(self,
                 name: str,
                 accountNr: int,
                 startAmount: float,
                 interest_rate: float) -> None:
        super().__init__(name, accountNr, startAmount)
        self.interest_rate = interest_rate
        self.saldi = []
```

```
        self.saldi.append((startAmount, date(2026, 1, 1)))

    def add_entry(self, entry: Entry) -> None:
        """Extends Account.add_entry by recording the new dated
         ↪ balance."""
        super().add_entry(entry)
        self.saldi.append((self.calculate_saldo(), entry.date))

    def calculate_interest(self, end_date=date(2026, 12, 31)) -> float:
        """Computes 30/360 interest across the balance history.

        Args:
            end_date (date, optional): End of the interest period.
                Defaults to 31 December 2026.

        Returns:
            float: The total interest over the period.
        """
        saldi = [saldo[0] for saldo in self.saldi]
        day_list = [saldo[1] for saldo in self.saldi]

        # Days between consecutive balances (adjacent pairs via zip)...
        number_of_days = [
            (nxt - cur).days
            for cur, nxt in zip(day_list, day_list[1:])
        ]
        # ... plus the remaining days from the last balance to end_date.
        number_of_days.append((end_date - day_list[-1]).days)

        # Charge each balance interest over the days it was held.
        interests = [
            saldo * self.interest_rate / 360 * days
            for saldo, days in zip(saldi, number_of_days)
        ]

        if end_date == date(2026, 12, 31):
            entry = Entry(1020, 6950, sum(interests), end_date)
            self.add_entry(entry)
        return sum(interests)
```

Running the same example confirms that the refactored version yields exactly the same interest — same result, clearer code.

```
bank = CurrentAccount('Bank', 1020, 100.00, 0.05)
bank.add_entry(Entry(1020, 1000, 150.00, date(2026, 3, 1)))
print(bank.calculate_interest())
```

Because `CurrentAccount` *is an* `Account`, every inherited method just works — and `isinstance()` confirms the relationship.

```
print("Is an Account:", isinstance(bank, Account))
print("Balance:", bank.saldo)
```

Teil V

Cyber Securtity

26 Caesar Chiffre

In der Kryptologie wird mit Verschlüsselung jener Prozess beschrieben, der eine Nachricht so transformiert, dass sie idealerweise nur von autorisierten Parteien gelesen werden kann. Dieser Prozess wandelt die ursprüngliche Darstellung der Informationen, bekannt als Klartext, in eine alternative Form, bekannt als Chiffretext, um. Trotz seines Ziels verhindert die Verschlüsselung nicht selbst die Interferenz, sondern verweigert dem potenziellen Abhörer den verständlichen Inhalt.

Aus Wikipedia

Das erste Beispiel für eine Chiffre, das wir uns ansehen werden, ist die Caesar-Chiffre.

Die Caesar-Chiffre ist eine einfache Substitutionsverschlüsselungstechnik, bei der jeder Buchstabe des zu verschlüsselnden Textes durch einen Buchstaben ersetzt wird, der sich um eine feste Anzahl von Positionen im Alphabet verschiebt. Zum Beispiel würde bei einer Verschiebung um vier Positionen nach rechts `A` durch `E` ersetzt werden, und das Wort `CIPHER` würde zu `GMTLIV`. Die Technik ist nach Julius Caesar benannt, der sie in seinen Briefen verwendete. Die Einfachheit der Caesar-Chiffre macht sie zu einer beliebten Quelle für Freizeit-Kryptogramme.

Aus Encyclopedia Britannica

26.1 Python Implementation

Um die Caesar-Chiffre in Python zu implementieren, bauen wir auf den von Python angebotenen String-Methoden auf. Die Funktion `ord` gibt den Unicode-Codepunkt für ein gegebenes Zeichen zurück, und die Funktion `chr` gibt das Zeichen zurück, das dem gegebenen Unicode-Codepunkt entspricht.

```
1  # Was ist der Unicode Wert des Buchstabens 'A'?
```

26.1.1 Simple (naive) Implementation

Als erstes definieren wir eine Funktion, die einen gegebenen Klartext verschlüsselt, indem sie jeden Buchstaben um eine bestimmte Anzahl von Positionen im Alphabet verschiebt.

```
1  plain = "CIPHER"
2  shift = 4
```

```python
def caesar_encode(plain, shift):
    cipher = ""

    for char in plain:
        shifted = ord(char) + shift
        cipher += chr(shifted)

    return cipher
```

26.1.2 Verbesserte Implementation

Was geschieht aber, wenn wir das Ende des Alphabets erreichen? Zum Beispiel, wenn wir `Z` um 4 Positionen verschieben, würden wir über `Z` hinausgehen. Um dies zu handhaben, können wir den Modulo-Operator `%` verwenden, um im Alphabet von vorne zu beginnen. Der Modulo-Operator gibt den Rest einer Division zurück, was es uns ermöglicht, wieder bei Null zu beginnen, wenn wir die Länge des Alphabets überschreiten. Im Fall der Caesar-Chiffre können wir ihn verwenden, um sicherzustellen, dass unsere verschobenen Positionen innerhalb der Grenzen des Alphabets bleiben.

Daher verwenden wir modulo 26 (die Anzahl der Buchstaben im lateinischen Alphabet), um sicherzustellen, dass unsere verschobenen Positionen korrekt "herumwickeln". Wenn wir `Z` um 4 Positionen verschieben, landen wir bei `D`. Dieses Wickelverhalten ist für die Funktionsweise der Caesar-Chiffre von entscheidender Bedeutung. Die Berechnung der neuen Position eines Buchstabens kann wie folgt ausgedrückt werden:

$$x' = (x + n) \mod 26$$

oder in einer kompakteren Form mit dem Modulo-Additionsoperator:

$$x' = x \oplus_{26} n$$

In Python können wir dies wie folgt implementieren:

```python
def caesar_encrypt_mod(plain, shift):
    cipher = ""

    for char in plain:
        shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
        cipher += chr(shifted)

    return cipher
```

Weil

```
ord('A')
```

65 zurückgibt, müssen wir 65 von dem Ergebnis von `ord('A')` subtrahieren, um 0 zu erhalten. Dies gibt uns den 0-basierten Index des Buchstabens im Alphabet, der für unsere Berechnungen nützlich ist.

Daher die oben gezeigte Berechnung.

Für die Entschlüsselung können wir einfach den Verschiebungswert subtrahieren anstatt ihn zu addieren:

```
def caesar_decrypt_mod(plain, shift):
    cipher = ""

    for char in plain:
        shifted = (ord(char) - ord('A') - shift) % 26 + ord('A')
        cipher += chr(shifted)

    return cipher
```

Für die Bequemlichkeit implementieren wir eine Funktion die sowohl ver- als auch entschlüsselt.

```
def caesar(text : str, shift : int, encrypt=True) -> str:
    text = text.upper()
    result = ""

    if encrypt:
        for char in text:
            shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
            result += chr(shifted)
    else:
        for char in text:
            shifted = (ord(char) - ord('A') - shift) % 26 + ord('A')
            result += chr(shifted)

    return result
```

26.2 Caesar Chiffre brechen

Es gibt zwei Hauptmethoden, um eine Caesar-Chiffre zu brechen:

1. Brute Force Attack: Versuchen Sie alle möglichen Verschiebungen (1-25) und sehen Sie, welche eine sinnvolle Ausgabe erzeugt.

2. Frequency Analysis: Analysieren Sie die Häufigkeit von Buchstaben im Chiffretext und vergleichen Sie sie mit der erwarteten Häufigkeit von Buchstaben in der Sprache.

26.2.1 Brute Force Attack

```python
def caesar_bruteforce(ciphertext: str) -> None:
    for shift in range(1, 26):
        decrypted_text = caesar(ciphertext, shift, encrypt=False)
        print(f"Shift {shift}: {decrypted_text}")
```

26.2.2 Häufigkeitsanalyse

In der deutschen Sprache erscheinen bestimmte Buchstaben häufiger als andere. Zum Beispiel ist der Buchstabe ‘E’ der häufigste Buchstabe in deutschen Texten, gefolgt von ‘N’, ‘I’, ‘S’, ‘R’ und ‘A’. Durch die Analyse der Häufigkeit von Buchstaben im Chiffretext können wir fundierte Vermutungen darüber anstellen, welche Buchstaben im Klartext welchen im Chiffretext entsprechen.

Für eine genauere Analyse siehe die folgende Häufigkeitstabelle:

Platz	Buchstabe	Relative Häufigkeit
1.	E	17,40 %
2.	N	9,78 %
3.	I	7,55 %
4.	S	7,27 %
5.	R	7,00 %
6.	A	6,51 %
7.	T	6,15 %
8.	D	5,08 %
9.	H	4,76 %
10.	U	4,35 %
11.	L	3,44 %
12.	C	3,06 %
13.	G	3,01 %
14.	M	2,53 %
15.	O	2,51 %
16.	B	1,89 %
17.	W	1,89 %
18.	F	1,66 %
19.	K	1,21 %
20.	Z	1,13 %
21.	P	0,79 %
22.	V	0,67 %

Platz	Buchstabe	Relative Häufigkeit
23.	J	0,27 %
24.	Y	0,04 %
25.	X	0,03 %
26.	Q	0,02 %

Quelle: Wikipedia

Wir brauchen also eine Funktion, die die Häufigkeit jedes Buchstabens im Chiffretext zählt. Das Ergebnis sollte ein Dictionary sein, bei dem die Schlüssel die Buchstaben und die Werte die Zählungen jedes Buchstabens sind. Oder noch besser, die Häufigkeiten in Prozent.

Um die Sache einfach zu halten, nehmen wir an, dass der Eingabetext nur Großbuchstaben von `A` bis `Z` und keine Leerzeichen oder Satzzeichen enthält.

```
def letter_frequency(text: str) -> dict:
    frequency = {}
    total_letters = 0

    for char in text:
        if char not in frequency:
            frequency[char] = 1
        else:
            frequency[char] += 1
        total_letters += 1

    for key, value in frequency.items():
        frequency[key] = (value / total_letters) * 100


    return frequency
```

Nachdem wir den häufigsten Buchstaben im Chiffretext gefunden haben, können wir annehmen, dass er dem Buchstaben ‘E’ im Klartext entspricht. Indem wir die Verschiebung berechnen, die erforderlich ist, um den häufigsten Buchstaben in ‘E’ zu verwandeln, können wir dann den gesamten Chiffretext mit diesem Verschiebungswert entschlüsseln.

```
def find_shift(char: str, e = 'E') -> int:
    return (ord(char) - ord(e)) % 26
```

27 File Handling in Python

Um Textdateien in Python zu bearbeiten, schreiben wir eine Funktion, mit welcher der Inhalt einer Datei einer Variabel als String zugewiesen wird.

```
def file_reader(path : str) -> str:

    with open(path, mode='r', encoding='utf-8') as f:
        text = f.read()

    return text
```

Um ver- oder entschlüsselte Texte in eine Datei zu schreiben, schreiben wir eine Funktion, die einen String in eine Datei schreibt.

```
def file_writer(path : str, text : str) -> None:
    i = 0
    grouped_text = ""
    for c in text:
        i += 1
        if i % 50 == 0:
            grouped_text += c + "\n"
        elif i % 5 == 0:
            grouped_text += c + " "
        else:
            grouped_text += c

    with open(path, mode='w', encoding='utf-8') as f:
        f.write(grouped_text)
```

Damit Texte ausschliesslich aus ASCII Grossbuchstaben bestehen, schreiben wir eine Funktion, die alle Kleinbuchstaben in Grossbuchstaben umwandelt und alle Umlaute in ihre aequivalenten Buchstaben umwandelt. Alle anderen Zeichen werden entfernt.

Damit alle Methoden für die Bearbeitung von Strings zur Verfügung stehen, muss das String-Modul importiert werden.

```
import string
```

```python
def text_cleaning(text : str) -> str:
    clean = text.upper() \
                .replace('Ä', 'AE') \
                .replace('Ö', 'OE') \
                .replace('Ü', 'UE') \
                .replace('ß', 'SS') \
                .replace(' ', '') \

    cleaned_text = ''

    for c in clean:
        if c.isalpha():
            cleaned_text += c

    return cleaned_text
```

28 Caesar-Chiffre knacken

In der letzten Lektion haben wir die Caesar-Chiffre kennengelernt und in Python implementiert. Wer den Schlüssel kennt, also die Anzahl Positionen, um die jeder Buchstabe verschoben wurde, kann einen Chiffretext ohne Mühe entschlüsseln. Spannender ist die Frage, was passiert, wenn wir den Schlüssel **nicht** kennen. Genau in dieser Situation befindet sich ein Angreifer, der eine verschlüsselte Nachricht abfängt.

Das Brechen einer Verschlüsselung ohne Kenntnis des Schlüssels wird als *Kryptoanalyse* bezeichnet. Für die Caesar-Chiffre gibt es zwei grundsätzlich verschiedene Vorgehensweisen:

1. **Brute Force**: Alle möglichen Schlüssel werden ausprobiert.
2. **Statistische Analyse**: Aus Eigenschaften des Chiffretextes wird auf den Schlüssel geschlossen.

Beide Verfahren setzen wir im Folgenden in Python um.

28.1 Brute Force

Der Begriff *Brute Force* (englisch für «rohe Gewalt») bezeichnet ein Verfahren, bei dem schlicht alle möglichen Schlüssel der Reihe nach ausprobiert werden, bis der richtige gefunden ist. Raffinesse ist dabei nicht gefragt, sondern nur Ausdauer, und davon haben Computer reichlich.

Bei der Caesar-Chiffre ist dieser Ansatz besonders erfolgversprechend, weil der *Schlüsselraum*, also die Menge aller möglichen Schlüssel, winzig ist. Das lateinische Alphabet hat 26 Buchstaben. Eine Verschiebung um 26 Positionen führt wieder zum Klartext, ebenso eine Verschiebung um 0. Es gibt deshalb nur **25 sinnvolle Schlüssel**. Diese von Hand durchzuprobieren, dauert ein paar Minuten, ein Computer erledigt es in Bruchteilen einer Sekunde.

Das Vorgehen ist denkbar einfach:

1. Der Chiffretext wird mit jedem möglichen Schlüssel von 1 bis 25 entschlüsselt.
2. Die 25 Ergebnisse werden ausgegeben.
3. Unter diesen Kandidaten wird derjenige gesucht, der einen lesbaren Text ergibt.

Der letzte Schritt ist der Haken an der Sache: Der Computer liefert zwar alle Kandidaten, aber er weiss nicht, welcher davon der richtige ist. Bei einem kurzen Text kann ein Mensch das auf einen Blick erkennen. Soll der Computer diese Entscheidung selbst treffen, braucht er ein Kriterium, mit dem er beurteilen kann, wie sehr ein Text nach Deutsch «aussieht». Ein solches Kriterium liefert die statistische Analyse.

Brute Force zeigt ausserdem, worauf es bei einer sicheren Verschlüsselung ankommt: Der Schlüsselraum muss so gross sein, dass das Ausprobieren aller Schlüssel auch mit den schnellsten Computern praktisch unmöglich ist. Moderne Verfahren wie AES verwenden Schlüssel mit 128 Bit oder mehr, was rund $3{,}4 \cdot 10^{38}$ möglichen Schlüsseln entspricht.

28.1.1 Hilfsfunktion

Für beide Verfahren brauchen wir die Funktion `caesar` aus der letzten Lektion. Wir passen sie leicht an: Zeichen, die keine Buchstaben von `A` bis `Z` sind (Leerzeichen, Satzzeichen, Umlaute), werden nicht verschoben, sondern unverändert übernommen. So bleibt ein entschlüsselter Text besser lesbar.

```
def caesar(text: str, shift: int, encrypt=True) -> str:
    text = text.upper()
    if not encrypt:
        shift = -shift
    result = ""

    for char in text:
        if 'A' <= char <= 'Z':
            shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
            result += chr(shifted)
        else:
            result += char

    return result
```

Als Beispiel verschlüsseln wir den Anfang des Johannesevangeliums mit einem Schlüssel, den wir anschliessend «vergessen».

```
plain = """Im Anfang war das Wort, und das Wort war bei Gott, und das
 ↪ Wort war
Gott. Dieses war im Anfang bei Gott. Alle Dinge sind durch dasselbe
geworden, und ohne das Wort ist auch nicht eines geworden, das geworden
ist. In ihm war Leben, und das Leben war das Licht für die Menschen."""

ciphertext = caesar(plain, 7)
print(ciphertext)
```

28.1.2 Implementation

Die Funktion `brute_force` entschlüsselt den Chiffretext mit jedem der 25 möglichen Schlüssel und gibt die Ergebnisse als Liste von Tupeln `(schlüssel, kandidat)` zurück. Weil die

Kandidaten zurückgegeben und nicht direkt ausgegeben werden, können wir sie später weiterverarbeiten, zum Beispiel statistisch bewerten.

```
def brute_force(ciphertext: str) -> list[tuple[int, str]]:
    candidates = []

    for shift in range(1, 26):
        candidate = caesar(ciphertext, shift, encrypt=False)
        candidates.append((shift, candidate))

    return candidates
```

Um die Ausgabe übersichtlich zu halten, zeigen wir von jedem Kandidaten nur die ersten 40 Zeichen an.

```
for shift, candidate in brute_force(ciphertext):
    print(f"{shift:2}: {candidate[:40]}")
```

Ein Blick auf die Liste genügt, um den richtigen Schlüssel zu finden. Der Computer kann das aber noch nicht. Dazu brauchen wir die statistische Analyse.

28.2 Statistische Analyse

Die statistische Analyse, genauer die *Häufigkeitsanalyse*, nutzt eine Schwäche der Caesar-Chiffre aus: Jeder Buchstabe des Klartextes wird immer durch denselben Buchstaben im Chiffretext ersetzt. Die Buchstaben werden also zwar umbenannt, aber ihre Häufigkeit bleibt erhalten.

In jeder natürlichen Sprache kommen die Buchstaben unterschiedlich oft vor. Im Deutschen ist `E` mit rund 17 % mit Abstand der häufigste Buchstabe, gefolgt von `N` (knapp 10 %) und `I` (gut 7.5 %). Buchstaben wie `Q`, `X` oder `Y` kommen dagegen kaum vor. Diese Verteilung ist für eine Sprache so charakteristisch, dass sie sich in jedem genügend langen Text wiederfindet.

Daraus ergibt sich folgendes Vorgehen:

1. Im Chiffretext wird gezählt, wie oft jeder Buchstabe vorkommt.
2. Der häufigste Buchstabe im Chiffretext entspricht sehr wahrscheinlich dem `E` im Klartext.
3. Der Abstand zwischen diesem Buchstaben und `E` im Alphabet ist der gesuchte Schlüssel.

Ist im Chiffretext beispielsweise `I` der häufigste Buchstabe, liegt eine Verschiebung um vier Positionen nahe, denn `E` + 4 = `I`.

Dieses Verfahren hat seine Grenzen. Bei sehr kurzen Texten weicht die Häufigkeitsverteilung oft deutlich vom Durchschnitt ab, und der häufigste Buchstabe ist dann nicht unbedingt ein `E`. Zuverlässiger wird die Analyse, wenn nicht nur ein einzelner Buchstabe, sondern die

gesamte Verteilung betrachtet wird: Für jeden der 25 möglichen Schlüssel wird geprüft, wie gut die Buchstabenhäufigkeiten des entschlüsselten Textes mit den bekannten Häufigkeiten der deutschen Sprache übereinstimmen. Der Schlüssel mit der besten Übereinstimmung gewinnt.

Damit schliesst sich der Kreis zu Brute Force: Die Kombination beider Ansätze, also alle Schlüssel ausprobieren und das Ergebnis statistisch bewerten, erlaubt es dem Computer, eine Caesar-Chiffre vollständig selbständig zu knacken.

Die Häufigkeitsanalyse ist übrigens keine moderne Erfindung. Sie wurde bereits im 9. Jahrhundert vom arabischen Gelehrten al-Kindi beschrieben und funktioniert nicht nur bei der Caesar-Chiffre, sondern bei allen einfachen Substitutionschiffren.

28.2.1 Buchstabenhäufigkeit zählen

Als erstes brauchen wir die bekannten Häufigkeiten der deutschen Sprache. Wir übernehmen die Werte (in Prozent) aus der Häufigkeitstabelle der letzten Lektion in ein Dictionary.

```
GERMAN_FREQUENCY = {
    'A': 6.51, 'B': 1.89, 'C': 3.06, 'D': 5.08, 'E': 17.40, 'F': 1.66,
    'G': 3.01, 'H': 4.76, 'I': 7.55, 'J': 0.27, 'K': 1.21, 'L': 3.44,
    'M': 2.53, 'N': 9.78, 'O': 2.51, 'P': 0.79, 'Q': 0.02, 'R': 7.00,
    'S': 7.27, 'T': 6.15, 'U': 4.35, 'V': 0.67, 'W': 1.89, 'X': 0.03,
    'Y': 0.04, 'Z': 1.13,
}
```

Die Funktion `letter_frequency` zählt, wie oft jeder Buchstabe in einem Text vorkommt, und rechnet die Anzahl in Prozent um. Im Unterschied zur letzten Lektion werden alle anderen Zeichen ignoriert, und das Dictionary enthält jeden Buchstaben von `A` bis `Z`, auch wenn er gar nicht vorkommt. Das brauchen wir später für den Vergleich mit `GERMAN_FREQUENCY`.

```
def letter_frequency(text: str) -> dict[str, float]:
    counts = {}
    for code in range(ord('A'), ord('Z') + 1):
        counts[chr(code)] = 0

    total_letters = 0
    for char in text.upper():
        if char in counts:
            counts[char] += 1
            total_letters += 1

    frequency = {}
    for letter, count in counts.items():
        if total_letters > 0:
```

```
            frequency[letter] = count / total_letters * 100
        else:
            frequency[letter] = 0.0

    return frequency
```

```
frequency = letter_frequency(ciphertext)

for letter, percent in sorted(frequency.items(), key=lambda item:
    item[1], reverse=True)[:5]:
    print(f"{letter}: {percent:5.2f} %")
```

28.2.2 Der häufigste Buchstabe

Der häufigste Buchstabe im Chiffretext entspricht vermutlich dem `E`. Die Funktion `find_shift` aus der letzten Lektion berechnet den Abstand zwischen zwei Buchstaben. Die Funktion `crack_by_e` sucht den häufigsten Buchstaben und gibt den daraus berechneten Schlüssel zurück.

```
def find_shift(char: str, e='E') -> int:
    return (ord(char) - ord(e)) % 26


def crack_by_e(ciphertext: str) -> int:
    frequency = letter_frequency(ciphertext)
    most_common = max(frequency, key=frequency.get)
    return find_shift(most_common)
```

```
shift = crack_by_e(ciphertext)
print(f"Vermuteter Schlüssel: {shift}")
print(caesar(ciphertext, shift, encrypt=False))
```

Bei einem kurzen Text geht das schief. Im folgenden Beispiel ist `S` und nicht `E` der häufigste Buchstabe des Klartextes.

```
short_cipher = caesar("Alle Truppen sammeln sich am Fluss", 3)
shift = crack_by_e(short_cipher)

print(f"Vermuteter Schlüssel: {shift}")
print(caesar(short_cipher, shift, encrypt=False))
```

Variante: Die gesamte Verteilung vergleichen

Zuverlässiger ist es, die gesamte Häufigkeitsverteilung zu betrachten. Um zu messen, wie gut die beobachteten Häufigkeiten B_i eines Textes zu den erwarteten Häufigkeiten E_i der deutschen Sprache passen, verwenden wir die *Chi-Quadrat-Summe*:

$$\chi^2 = \sum_{i=A}^{Z} \frac{(B_i - E_i)^2}{E_i}$$

Für jeden Buchstaben wird die Abweichung zwischen beobachtetem und erwartetem Wert quadriert und durch den erwarteten Wert geteilt. Das Quadrieren sorgt dafür, dass sich positive und negative Abweichungen nicht gegenseitig aufheben. Das Teilen durch E_i bewirkt, dass eine Abweichung bei einem seltenen Buchstaben stärker ins Gewicht fällt: Ein Text mit 5 % `Q` ist viel verdächtiger als einer mit 5 % zu wenig `E`. Je **kleiner** χ^2, desto ähnlicher ist der Text einem deutschen Text.

```
def chi_squared(text: str) -> float:
    observed = letter_frequency(text)
    total = 0.0

    for letter, expected in GERMAN_FREQUENCY.items():
        total += (observed[letter] - expected) ** 2 / expected

    return total
```

Nun verbinden wir die beiden Verfahren: `crack` erzeugt mit `brute_force` alle Kandidaten, bewertet jeden mit `chi_squared` und gibt den Kandidaten mit dem kleinsten Wert zurück.

```
def crack(ciphertext: str) -> tuple[int, str]:
    best_shift = 0
    best_text = ciphertext
    best_score = float('inf')

    for shift, candidate in brute_force(ciphertext):
        score = chi_squared(candidate)
        if score < best_score:
            best_shift = shift
            best_text = candidate
            best_score = score

    return best_shift, best_text
```

```
shift, text = crack(ciphertext)
print(f"Gefundener Schlüssel: {shift}")
print(text)
```

Und auch der kurze Text, an dem die erste Variante gescheitert ist, wird nun richtig entschlüsselt.

```
shift, text = crack(short_cipher)
print(f"Gefundener Schlüssel: {shift}")
print(text)
```

Ganz ohne Grenzen ist aber auch dieses Verfahren nicht. Bei einem Text wie «Hallo Welt, alles klar?» mit nur 18 Buchstaben, davon sechs `L`, ist die Verteilung so untypisch, dass auch die Chi-Quadrat-Summe den falschen Schlüssel liefert. Je länger der Chiffretext, desto zuverlässiger funktioniert die statistische Analyse.
Wer nachvollziehen möchte, wie die Entscheidung zustande kommt, kann sich die Werte für alle Schlüssel ausgeben lassen. Der richtige Schlüssel sticht deutlich hervor.

```
for shift, candidate in brute_force(ciphertext):
    print(f"{shift:2}: {chi_squared(candidate):10.1f}
     ↪  {candidate[:30]}")
```

29 Die Vigenère-Chiffre

Die Caesar-Chiffre hat zwei grosse Schwächen, die wir beim Knacken ausgenutzt haben:

1. Es gibt nur 25 mögliche Schlüssel. Ein Computer probiert sie in Bruchteilen einer Sekunde alle durch.
2. Jeder Buchstabe wird immer durch denselben Buchstaben ersetzt. Deshalb bleibt die Häufigkeitsverteilung der Buchstaben erhalten, und der häufigste Buchstabe im Geheimtext verrät meistens das `E`.

Die Vigenère-Chiffre behebt beide Schwächen mit einer einfachen Idee: Statt alle Buchstaben um denselben Betrag zu verschieben, wird **jeder Buchstabe um einen anderen Betrag** verschoben. Welcher Betrag das ist, legt ein Schlüssel**wort** fest.

29.1 Ein wenig Geschichte

Das Verfahren wurde 1553 vom italienischen Kryptologen Giovan Battista Bellaso beschrieben. Seinen Namen verdankt es aber dem französischen Diplomaten Blaise de Vigenère, der 1586 ein Buch über Verschlüsselung veröffentlichte und dem die Erfindung später irrtümlich zugeschrieben wurde.[1]

Die Chiffre galt rund 300 Jahre lang als unknackbar und wurde in Frankreich *le chiffre indéchiffrable* genannt, die nicht zu entziffernde Chiffre. Erst um 1854 fand der Engländer Charles Babbage einen Weg, sie zu brechen, veröffentlichte ihn aber nie. 1863 beschrieb der preussische Offizier Friedrich Wilhelm Kasiski unabhängig davon ein Verfahren, mit dem sich die Chiffre systematisch angreifen lässt.

29.2 Die Idee: Ein Schlüsselwort statt einer Zahl

Bei der Caesar-Chiffre ist der Schlüssel eine Zahl, zum Beispiel 3. Bei der Vigenère-Chiffre ist der Schlüssel ein Wort. In unserem Beispiel verwenden wir das Schlüsselwort

BUELRAIN

Weil die Chiffre nur die Buchstaben `A` bis `Z` kennt, wird das `Ü` als `UE` geschrieben.

[1] Die Fehlzuschreibung ist sogar doppelt ironisch. In seinem *Traicté des chiffres* beschrieb Vigenère nämlich ein stärkeres Verfahren, die sogenannte *Autokey-Chiffre*. Dort wird nach einem kurzen Startschlüssel der Klartext selbst als Schlüssel verwendet. Dadurch wiederholt sich der Schlüssel nie, und genau diese Wiederholung ist, wie wir sehen werden, die grosse Schwäche der Chiffre, die heute seinen Namen trägt.

Jeder Buchstabe des Schlüsselworts steht für eine Verschiebung. Dabei gilt dieselbe Nummerierung wie bei der Caesar-Chiffre in Python: `A` = 0, `B` = 1, `C` = 2 und so weiter bis `Z` = 25. Der Buchstabe gibt also an, um wie viele Positionen verschoben wird:

Schlüsselbuchstabe	`B`	`U`	`E`	`L`	`R`	`A`	`I`	`N`
Verschiebung	1	20	4	11	17	0	8	13

Unser Schlüsselwort besteht damit aus acht verschiedenen Caesar-Verschiebungen. Der erste Buchstabe des Klartextes wird um 1 verschoben, der zweite um 20, der dritte um 4 und so weiter. Nach dem achten Buchstaben ist das Schlüsselwort aufgebraucht, und es beginnt wieder von vorne: Der neunte Buchstabe wird wieder um 1 verschoben, der zehnte um 20 usw.

Man kann sich die Vigenère-Chiffre deshalb als **acht Caesar-Chiffren** vorstellen, die sich reihum abwechseln.

29.3 Verschlüsseln Schritt für Schritt

Wir verschlüsseln die Nachricht

ANGRIFF IM MORGENGRAUEN

Wie bei der Caesar-Chiffre lassen wir Leerzeichen weg und schreiben nur Grossbuchstaben. Leerzeichen würden einem Angreifer verraten, wo ein Wort aufhört, und damit wertvolle Hinweise liefern.

Schritt 1: Schlüsselwort unter den Klartext schreiben

Das Schlüsselwort wird so oft wiederholt, bis es gleich lang ist wie der Klartext:

```
Klartext:   ANGRIFFIMMORGENGRAUEN
Schlüssel:  BUELRAINBUELRAINBUELR
```

Schritt 2: Jeden Buchstaben einzeln verschieben

Nun wird jeder Klartextbuchstabe um den Wert des Schlüsselbuchstabens darunter verschoben. Sehen wir uns die ersten drei Buchstaben genau an:

- `A` (0) mit Schlüssel `B` (1): 0 + 1 = 1, das ergibt `B`.
- `N` (13) mit Schlüssel `U` (20): 13 + 20 = 33. Das Alphabet hat nur 26 Buchstaben, also beginnen wir wie bei der Caesar-Chiffre wieder vorne: 33 − 26 = 7, das ergibt `H`.
- `G` (6) mit Schlüssel `E` (4): 6 + 4 = 10, das ergibt `K`.

Die folgende Tabelle zeigt die Rechnung für den ganzen Text:

Nr.	Klartext	Schlüssel	Verschiebung	Rechnung	Geheimtext
1	A	B	1	$0 + 1 = 1$	B
2	N	U	20	$13 + 20 = 33 \rightarrow 33 - 26 = 7$	H
3	G	E	4	$6 + 4 = 10$	K
4	R	L	11	$17 + 11 = 28 \rightarrow 28 - 26 = 2$	C
5	I	R	17	$8 + 17 = 25$	Z
6	F	A	0	$5 + 0 = 5$	F
7	F	I	8	$5 + 8 = 13$	N
8	I	N	13	$8 + 13 = 21$	V
9	M	B	1	$12 + 1 = 13$	N
10	M	U	20	$12 + 20 = 32 \rightarrow 32 - 26 = 6$	G
11	O	E	4	$14 + 4 = 18$	S
12	R	L	11	$17 + 11 = 28 \rightarrow 28 - 26 = 2$	C
13	G	R	17	$6 + 17 = 23$	X
14	E	A	0	$4 + 0 = 4$	E
15	N	I	8	$13 + 8 = 21$	V
16	G	N	13	$6 + 13 = 19$	T
17	R	B	1	$17 + 1 = 18$	S
18	A	U	20	$0 + 20 = 20$	U
19	U	E	4	$20 + 4 = 24$	Y
20	E	L	11	$4 + 11 = 15$	P
21	N	R	17	$13 + 17 = 30 \rightarrow 30 - 26 = 4$	E

Schritt 3: Geheimtext aufschreiben

Der fertige Geheimtext lautet:

BHKCZFNVNGSCXEVTSUYPE

Damit er trotzdem gut lesbar bleibt, wird der Geheimtext üblicherweise in Fünfergruppen geschrieben. Die Gruppen haben nichts mit den Wörtern des Klartextes zu tun:

BHKCZ FNVNG SCXEV TSUYP E

29.4 Das Vigenère-Quadrat

Ohne Computer ist das ständige Rechnen mühsam und fehleranfällig. Deshalb benutzte man früher eine Tabelle, das sogenannte *Vigenère-Quadrat* (lateinisch *tabula recta*). Jede Zeile enthält das Alphabet, aber jede Zeile ist um eine Position weiter verschoben als die vorherige. Jede Zeile ist also eine eigene Caesar-Chiffre.

Abbildung 29.1 zeigt das vollständige Quadrat.

So wird die Tabelle benutzt:

1. Die **Zeile** wird durch den Schlüsselbuchstaben bestimmt.

Klartextbuchstabe (Spalte)

Schlüsselbuchstabe (Zeile)	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Abbildung 29.1: Das Vigenère-Quadrat. Hervorgehoben ist die Verschlüsselung des Klartextbuchstabens `N` mit dem Schlüsselbuchstaben `U`: Im Schnittpunkt von Zeile `U` und Spalte `N` steht der Geheimtextbuchstabe `H`.

2. Die **Spalte** wird durch den Klartextbuchstaben bestimmt.
3. Im Schnittpunkt steht der Geheimtextbuchstabe.

Probieren wir es mit dem zweiten Buchstaben unseres Beispiels aus, dem `N` mit dem Schlüsselbuchstaben `U`. In Abbildung 29.1 ist dieser Fall hervorgehoben: In der Zeile `U` und der Spalte `N` steht ein `H`. Das stimmt mit der Rechnung oben überein.

Für unser Schlüsselwort `BUELRAIN` brauchen wir nur acht der 26 Zeilen. Wer von Hand verschlüsselt, kann sich diese Zeilen herausschreiben:

Schlüssel	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**	**S**	**T**	**U**	**V**	**W**	**X**	**Y**	**Z**
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M

29.5 Die Rechnung als Formel

Bei der Caesar-Chiffre haben wir die Verschlüsselung so geschrieben:

$$x' = (x + n) \mod 26$$

Bei der Vigenère-Chiffre hängt die Verschiebung n von der Position im Text ab. Ist p_i der i-te Buchstabe des Klartextes, k das Schlüsselwort mit der Länge m und c_i der i-te Buchstabe des Geheimtextes, dann gilt (die Positionen werden ab 0 gezählt):

$$c_i = (p_i + k_{i \bmod m}) \mod 26$$

Der Ausdruck $i \bmod m$ sorgt dafür, dass das Schlüsselwort immer wieder von vorne beginnt. Bei unserem Schlüssel ist $m = 8$. Für den Buchstaben an Position $i = 9$ wird also der Schlüsselbuchstabe an Position $9 \bmod 8 = 1$ verwendet, das ist das `U`.

⚠ Achtung: Zwei Zählweisen

In der Tabelle oben haben wir die Buchstaben so nummeriert, wie man es im Alltag tut: Der erste Buchstabe hat die Nr. 1. In der Formel und in Python beginnt die Zählung dagegen bei 0. Die Position i ist deshalb immer um eins kleiner als die Nr. in der Tabelle.
Die Position $i = 9$ ist also der **zehnte** Buchstabe, in der Tabelle die Nr. 10. Dort steht tatsächlich der Klartextbuchstabe `M` mit dem Schlüssel `U`. Bei Nr. 9 in der Tabelle steht der Schlüssel `B`, das entspricht $i = 8$ und $8 \bmod 8 = 0$, also dem ersten Buchstaben des Schlüssels.

29.6 Entschlüsseln

Zum Entschlüsseln wird die Verschiebung wieder rückgängig gemacht. Statt zu addieren, wird der Wert des Schlüsselbuchstabens subtrahiert:

$$p_i = (c_i - k_{i \bmod m}) \mod 26$$

Für die ersten drei Buchstaben des Geheimtextes `BHK`:

- `B` (1) mit Schlüssel `B` (1): $1 - 1 = 0$, das ergibt `A`.
- `H` (7) mit Schlüssel `U` (20): $7 - 20 = -13$. Ist das Ergebnis negativ, zählen wir 26 dazu: $-13 + 26 = 13$, das ergibt `N`.
- `K` (10) mit Schlüssel `E` (4): $10 - 4 = 6$, das ergibt `G`.

Im Vigenère-Quadrat sucht man dazu in der Zeile des Schlüsselbuchstabens den Geheimtextbuchstaben. Die Spalte, in der er steht, ist der Klartextbuchstabe.

Wichtig ist: Wer entschlüsseln will, muss **dasselbe Schlüsselwort** kennen wie der Absender. Man spricht deshalb von einem *symmetrischen* Verschlüsselungsverfahren.

29.7 Warum die Häufigkeitsanalyse versagt

Vergleichen wir Klartext und Geheimtext genauer:

```
Klartext:   ANGRIFFIMMORGENGRAUEN
Schlüssel:  BUELRAINBUELRAINBUELR
Geheimtext: BHKCZFNVNGSCXEVTSUYPE
```

Dabei fallen einige Dinge auf:

- **Gleiche Klartextbuchstaben werden verschieden verschlüsselt.** Das `N` kommt dreimal vor und wird zu `H`, `V` und `E`. Das `G` wird zu `K`, `X` und `T`.

- **Gleiche Geheimtextbuchstaben stehen für verschiedene Klartextbuchstaben.** Das `V` im Geheimtext steht einmal für ein `I` und einmal für ein `N`.
- **Doppelbuchstaben verschwinden.** Aus `FF` in `ANGRIFF` wird `FN`, aus `MM` wird `NG`. Das liegt daran, dass bei unserem Schlüssel keine zwei gleichen Buchstaben nebeneinander stehen. Zwei benachbarte Buchstaben werden deshalb immer verschieden weit verschoben.

Genau darauf hat sich die Häufigkeitsanalyse verlassen. Bei der Caesar-Chiffre konnten wir annehmen, dass der häufigste Buchstabe im Geheimtext ein `E` ist. Bei der Vigenère-Chiffre wird das `E` je nach Position verschieden verschlüsselt. Bei unserem Schlüssel, dessen acht Buchstaben alle verschieden sind, wird es zu einem von acht verschiedenen Buchstaben. Die typische Häufigkeitsverteilung der deutschen Sprache wird dadurch verwischt, und der häufigste Buchstabe im Geheimtext verrät nichts mehr.

29.7.1 Ein Experiment: Häufigkeiten sichtbar machen

Ob die Verteilung tatsächlich verwischt wird, können wir mit einem Experiment überprüfen. Dazu brauchen wir einen Text, der lang genug ist, damit sich die typischen Häufigkeiten der deutschen Sprache zeigen. Wir verwenden den Anfang des Johannesevangeliums (Johannes 1, 1–14) in der Fassung der Zürcher Bibel von 1931. Die ersten vier Verse kennen wir bereits aus der Lektion über die Caesar-Chiffre.

Das Experiment hat vier Schritte:

1. Den Klartext vorbereiten.
2. Den Klartext einmal mit der Caesar-Chiffre und einmal mit der Vigenère-Chiffre verschlüsseln.
3. In allen drei Texten die Buchstaben zählen.
4. Die Häufigkeiten in drei Histogrammen nebeneinanderstellen.

29.7.1.1 Schritt 1: Den Klartext vorbereiten

Wir speichern den Text in einer Variablen. Die drei Anführungszeichen `"""` erlauben einen String über mehrere Zeilen.

```
text = """Im Anfang war das Wort, und das Wort war bei Gott, und von
 ↪ Gottes Wesen war das Wort.
Dieses war im Anfang bei Gott.
Alle Dinge sind durch das Wort geworden, und ohne dasselbe ist nicht
 ↪ eines geworden, das geworden ist.
In ihm war Leben, und das Leben war das Licht der Menschen.
Und das Licht scheint in der Finsternis, und die Finsternis hat es nicht
 ↪ begriffen.
Es trat ein Mensch auf, von Gott gesandt, sein Name war Johannes.
```

```
Dieser kam zum Zeugnis, um Zeugnis abzulegen von dem Licht, damit alle durch ihn glaubten.
Nicht er war das Licht, sondern er sollte Zeugnis ablegen von dem Licht.
Das wahre Licht, das jeden Menschen erleuchtet, war im Kommen in die Welt.
Es war in der Welt, und die Welt ist durch dasselbe geworden, und die Welt hat es nicht erkannt.
Es kam in sein Eigentum, und die Seinen nahmen es nicht auf.
Allen denen aber, die es aufnahmen, gab es Macht, Kinder Gottes zu werden, denen, die an seinen Namen glauben;
die nicht aus dem Blut, noch aus dem Willen des Fleisches, noch aus dem Willen des Mannes, sondern aus Gott geboren sind.
Und das Wort wurde Fleisch und wohnte unter uns, und wir schauten seine Herrlichkeit, eine Herrlichkeit als des Eingeborenen vom Vater,
voller Gnade und Wahrheit."""
```

Wie in der Lektion über die Caesar-Chiffre arbeiten wir nur mit den Grossbuchstaben von `A` bis `Z`. Die Funktion `clean` wandelt den Text in Grossbuchstaben um und behält nur die Buchstaben. Leerzeichen, Satzzeichen und Zeilenumbrüche fallen weg.

Umlaute dürfen wir aber nicht einfach weglassen, sonst würde aus «Jürg» der Name «JRG». Wie oben beschrieben, ersetzen wir sie deshalb durch zwei Buchstaben: `Ä` durch `AE`, `Ö` durch `OE` und `Ü` durch `UE`. Die Ersetzungen speichern wir in einem Dictionary `replacements`. Um das `ß` müssen wir uns nicht kümmern: `"ß".upper()` ergibt in Python bereits `"SS"`.

```
def clean(text: str) -> str:
    replacements = {"Ä": "AE", "Ö": "OE", "Ü": "UE"}
    result = ""

    for char in text.upper():
        if char in replacements:
            result += replacements[char]
        elif 'A' <= char <= 'Z':
            result += char

    return result
```

Ein kurzer Test zeigt, ob die Umlaute richtig ersetzt werden:

```
print(clean("Jürg grüsst: Äpfel, Öl und Spaß!"))
```

```
JUERGGRUESSTAEPFELOELUNDSPASS
```

Nun bereiten wir den Bibeltext vor. Er enthält keine Umlaute, die Ersetzung kommt hier

also nicht zum Einsatz.

```
plain = clean(text)
print(len(plain), "Buchstaben")
```

```
965 Buchstaben
```

29.7.1.2 Schritt 2: Verschlüsseln

Die Funktion `caesar` kennen wir bereits. Weil der Text nur noch Buchstaben enthält, genügt hier die einfache Version.

Die Funktion `vigenere` funktioniert fast gleich. Der Unterschied: Die Verschiebung ist nicht für alle Buchstaben gleich, sondern wird für jeden Buchstaben aus dem Schlüsselwort berechnet. Dafür brauchen wir die Position `i` des Buchstabens im Text. Die Funktion `enumerate` liefert in der `for`-Schleife zu jedem Zeichen auch seine Position. Mit `i % len(key)` beginnt das Schlüsselwort nach dem letzten Buchstaben wieder von vorne, genau wie in der Formel $c_i = (p_i + k_{i \bmod m}) \mod 26$.

```
def caesar(text: str, shift: int) -> str:
    result = ""

    for char in text:
        shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
        result += chr(shifted)

    return result


def vigenere(text: str, key: str) -> str:
    result = ""

    for i, char in enumerate(text):
        shift = ord(key[i % len(key)]) - ord('A')
        shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
        result += chr(shifted)

    return result
```

Wir verschlüsseln den Klartext mit der Caesar-Chiffre und dem Schlüssel 3 sowie mit der Vigenère-Chiffre und dem Schlüssel `BUELRAIN`.

```
caesar_text = caesar(plain, 3)
vigenere_text = vigenere(plain, "BUELRAIN")
```

Damit die Texte lesbar bleiben, geben wir sie in Fünfergruppen aus, zehn Gruppen pro Zeile. Die Funktion `print_groups` schneidet dazu den Text mit `text[start:start + 5]` in Stücke von fünf Buchstaben.

```
def print_groups(text: str, size=5, per_line=10) -> None:
    groups = []
    for start in range(0, len(text), size):
        groups.append(text[start:start + size])

    for start in range(0, len(groups), per_line):
        print(" ".join(groups[start:start + per_line]))
```

```
print("Klartext")
print_groups(plain)
```

```
Klartext
IMANF ANGWA RDASW ORTUN DDASW ORTWA RBEIG OTTUN DVONG OTTES
WESEN WARDA SWORT DIESE SWARI MANFA NGBEI GOTTA LLEDI NGESI
NDDUR CHDAS WORTG EWORD ENUND OHNED ASSEL BEIST NICHT EINES
GEWOR DENDA SGEWO RDENI STINI HMWAR LEBEN UNDDA SLEBE NWARD
ASLIC HTDER MENSC HENUN DDASL ICHTS CHEIN TINDE RFINS TERNI
SUNDD IEFIN STERN ISHAT ESNIC HTBEG RIFFE NESTR ATEIN MENSC
HAUFV ONGOT TGESA NDTSE INNAM EWARJ OHANN ESDIE SERKA MZUMZ
EUGNI SUMZE UGNIS ABZUL EGENV ONDEM LICHT DAMIT ALLED URCHI
HNGLA UBTEN NICHT ERWAR DASLI CHTSO NDERN ERSOL LTEZE UGNIS
ABLEG ENVON DEMLI CHTDA SWAHR ELICH TDASJ EDENM ENSCH ENERL
EUCHT ETWAR IMKOM MENIN DIEWE LTESW ARIND ERWEL TUNDD IEWEL
TISTD URCHD ASSEL BEGEW ORDEN UNDDI EWELT HATES NICHT ERKAN
NTESK AMINS EINEI GENTU MUNDD IESEI NENNA HMENE SNICH TAUFA
LLEND ENENA BERDI EESAU FNAHM ENGAB ESMAC HTKIN DERGO TTESZ
UWERD ENDEN ENDIE ANSEI NENNA MENGL AUBEN DIENI CHTAU SDEMB
LUTNO CHAUS DEMWI LLEND ESFLE ISCHE SNOCH AUSDE MWILL ENDES
MANNE SSOND ERNAU SGOTT GEBOR ENSIN DUNDD ASWOR TWURD EFLEI
SCHUN DWOHN TEUNT ERUNS UNDWI RSCHA UTENS EINEH ERRLI CHKEI
TEINE HERRL ICHKE ITALS DESEI NGEBO RENEN VOMVA TERVO LLERG
NADEU NDWAH RHEIT
```

```
print("Caesar-Chiffre (Schlüssel 3)")
print_groups(caesar_text)
```

```
Caesar-Chiffre (Schlüssel 3)
LPDQI DQJZD UGDVZ RUWXQ GGDVZ RUWZD UEHLJ RWWXQ GYRQJ RWWHV
ZHVHQ ZDUGD VZRUW GLHVH VZDUL PDQID QJEHL JRWWD OOHGL QJHVL
QGGXU FKGDV ZRUWJ HZRUG HQXQG RKQHG DVVHO EHLVW QLFKW HLQHV
JHZRU GHQGD VJHZR UGHQL VWLQL KPZDU OHEHQ XQGGD VOHEH QZDUG
DVOLF KWGHU PHQVF KHQXQ GGDVO LFKWV FKHLQ WLQGH UILQV WHUQL
VXQGG LHILQ VWHUQ LVKDW HVQLF KWEHJ ULIIH QHVWU DWHLQ PHQVF
KDXIY RQJRW WJHVD QGWVH LQQDP HZDUM RKDQQ HVGLH VHUND PCXPC
HXJQL VXPCH XJQLV DECXO HJHQY RQGHP OLFKW GDPLW DOOHG XUFKL
KQJOD XEWHQ QLFKW HUZDU GDVOL FKWVR QGHUQ HUVRO OWHCH XJQLV
DEOHJ HQYRQ GHPOL FKWGD VZDKU HOLFK WGDVM HGHQP HQVFK HQHUO
HXFKW HWZDU LPNRP PHQLQ GLHZH OWHVZ DULQG HUZHO WXQGG LHZHO
WLVWG XUFKG DVVHO EHJHZ RUGHQ XQGGL HZHOW KDWHV QLFKW HUNDQ
QWHVN DPLQV HLQHL JHQWX PXQGG LHVHL QHQQD KPHQH VQLFK WDXID
OOHQG HQHQD EHUGL HHVDX IQDKP HQJDE HVPDF KWNLQ GHUJR WWHVC
XZHUG HQGHQ HQGLH DQVHL QHQQD PHQJO DXEHQ GLHQL FKWDX VGHPE
OXWQR FKDXV GHPZL OOHQG HVIOH LVFKH VQRFK DXVGH PZLOO HQGHV
PDQQH VVRQG HUQDX VJRWW JHERU HQVLQ GXQGG DVZRU WZXUG HIOHL
VFKXQ GZRKQ WHXQW HUXQV XQGZL UVFKD XWHQV HLQHK HUUOL FKNHL
WHLQH KHUUO LFKNH LWDOV GHVHL QJHER UHQHQ YRPYD WHUYR OOHUJ
QDGHX QGZDK UKHLW
```

```
print("Vigenère-Chiffre (Schlüssel BUELRAIN)")
print_groups(vigenere_text)
```

```
Vigenère-Chiffre (Schlüssel BUELRAIN)
JGEYW AVTXU VORSE BSNYY UDIFX IVENA ZOFCK ZKTCA EPSYX OBGFM
APJEV JBLHL JWWEU XMPJE AJBLM XRNNN OAFPZ GWGUU PWVDQ AHYWT
EDLHS WLORS EBSNK PNOZQ FHYYU OPAFX EDJET OFCWE EIKUU YMYVS
ORXIV OVNLN TAIHF RLROC WEZNQ UNQEC CEJRO OROUA AYFVI YNAZQ
BMPTT HBQFL QPESK UFHYY UDIFM CGSKS KUFCR EZNLR SZMYJ TMEOC
WFEDL VFZMY JTMEO CWSRT MFOCG SKBMT SCJQV NMFUL EEVIV ZFHWN
YACSW IRRFT BTFME YUTAR JHRLD EENSD SSRNV RTXMP JEZXB GDFDZ
MHHHM DLMHR VARTJ AJMVF IRVND BOXIX CIKUU XEXZT IYMYH FICPV
IHKWR UJGFH RTTHB RSQEC UAAYJ WLEJO VQFLR PISWY MNIKV UOAJM
EMCEO ROPSY UEUYJ WLEUA AJBBV PCIKU UXEDA ELROG IYJCP ROYVW
VUKUU YXHRR QZLIQ XVNQA ECIHV LBRTQ ECZNL RSQIW KUVQE CIHVL
BVTNH FICPQ BMWPC BMTFQ SCUEV HOXHT VWMYU BEEVS VVDBX PIKIA
ONIDB AUVOM ITEEQ TFHXF DUVQE CIDVI VROHE SDEVR THMNY TIHGU
PWVNL ROYRL SEZQJ YIDRU NABBQ PEGIO FMQLT HBXJH HPIGW GUYWK
LWMEE YROVN MAECI LESMV OYRYR MMAHF EFSEV QJYRT THBNV MHPDB
THUHS NYACF EYQHZ LTROX IDWLM VTWLP JNWPI UYDUE UJJFP PEDMF
NURYV SABOX ICEAC FHIXE XEJBS YRDZN LHOXH LJWWE UQYCU ENYFC
WNYUV QXILY KECAU YVFES CAEQM CJCPN VNIYJ EQAFB ICILQ PIEIT
```

```
KEQAF BICIL QPIEI TKATF EYWPZ NORCI VPEEV IPGZL KEZIP FPPIG
VNEYY YUWIU SBITK
```

Schon von blossem Auge fällt etwas auf: Der Klartext beginnt mit `IMANF ANGWA RDASW ORT…`, die Caesar-Chiffre entsprechend mit `LPDQI DQJZD UGDVZ RUW…`. Bei der Caesar-Chiffre wird jedes `A` zu `D`, die Muster des Klartextes bleiben also erhalten. Bei der Vigenère-Chiffre sind sie verschwunden.

29.7.1.3 Schritt 3: Buchstaben zählen

Die Funktion `letter_frequency` kennen wir aus der Lektion über das Knacken der Caesar-Chiffre. Sie gibt für jeden Buchstaben von `A` bis `Z` an, wie viel Prozent des Textes er ausmacht.

```
def letter_frequency(text: str) -> dict[str, float]:
    counts = {}
    for code in range(ord('A'), ord('Z') + 1):
        counts[chr(code)] = 0

    for char in text:
        counts[char] += 1

    frequency = {}
    for letter, count in counts.items():
        frequency[letter] = count / len(text) * 100

    return frequency
```

```
plain_frequency = letter_frequency(plain)
caesar_frequency = letter_frequency(caesar_text)
vigenere_frequency = letter_frequency(vigenere_text)
```

29.7.1.4 Schritt 4: Die Histogramme zeichnen

Für die Darstellung verwenden wir die Bibliothek matplotlib. Sie ist die bekannteste Python-Bibliothek zum Zeichnen von Diagrammen. Üblicherweise wird ihr Teil `pyplot` unter dem kurzen Namen `plt` importiert.

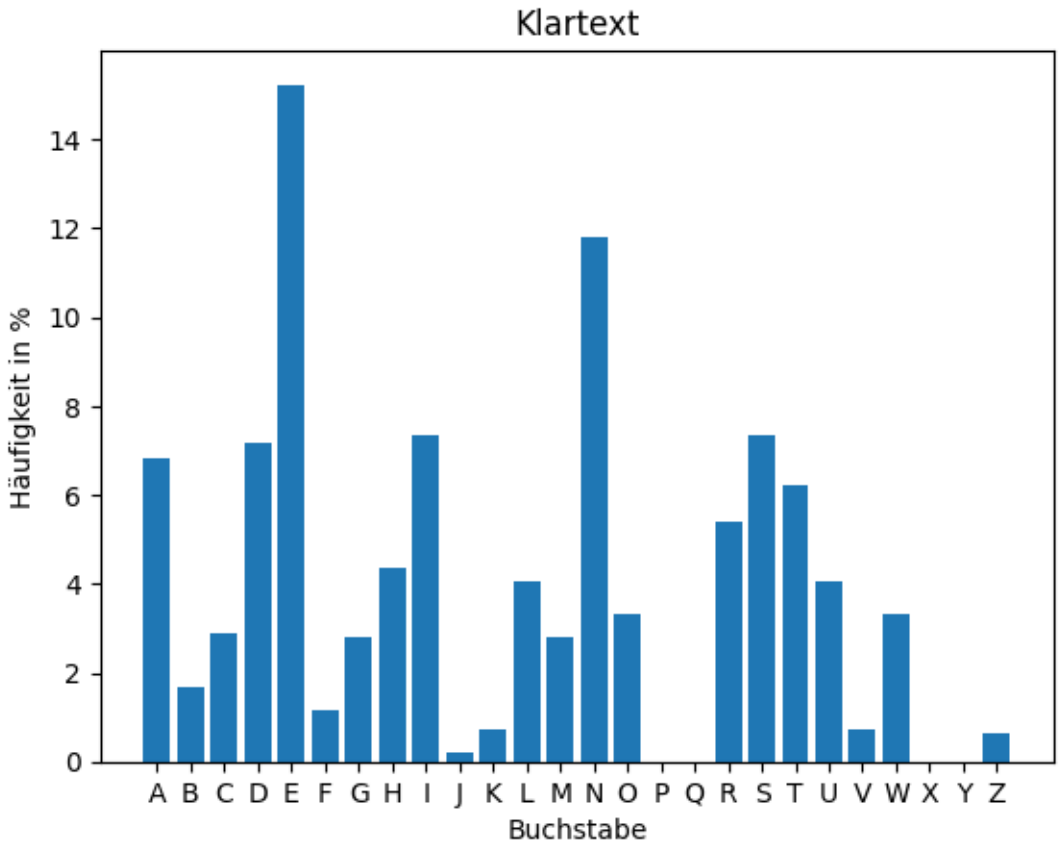
```
import matplotlib.pyplot as plt
```

Beginnen wir mit einem einzelnen Histogramm für den Klartext. Ein Histogramm zeigt für jeden Buchstaben einen Balken, dessen Höhe der Häufigkeit entspricht. In matplotlib zeichnet die Funktion `bar` solche Balken. Sie braucht zwei Listen gleicher Länge:

- die Beschriftungen der Balken, hier die Buchstaben (`letters`),
- die Höhen der Balken, hier die Häufigkeiten in Prozent (`values`).

Ein Dictionary liefert uns beides: `keys()` gibt die Buchstaben, `values()` die Häufigkeiten. Mit `list(...)` machen wir daraus Listen.

```
1  letters = list(plain_frequency.keys())
2  values = list(plain_frequency.values())
3
4  plt.bar(letters, values)
5  plt.title("Klartext")
6  plt.xlabel("Buchstabe")
7  plt.ylabel("Häufigkeit in %")
8  plt.show()
```



Die einzelnen Zeilen bewirken Folgendes:

- `plt.bar(letters, values)` zeichnet die Balken.
- `plt.title`, `plt.xlabel` und `plt.ylabel` beschriften das Diagramm und die beiden Achsen.
- `plt.show()` zeigt das fertige Diagramm an.

Man erkennt deutlich das typische Bild der deutschen Sprache: Das `E` ragt heraus, gefolgt von `N`, und die Buchstaben `P`, `Q`, `X` und `Y` kommen gar nicht vor.

Für den Vergleich wollen wir die drei Histogramme untereinander anordnen. Dafür erstellt `plt.subplots(3, 1)` eine Abbildung (`fig`) mit drei Diagrammen in drei Zeilen und einer Spalte. Die drei Diagramme erhalten wir in der Liste `axes`. Zwei Einstellungen helfen beim Vergleichen:

- `sharey=True` sorgt dafür, dass alle drei Diagramme dieselbe Skala auf der y-Achse haben. Nur so lassen sich die Höhen der Balken direkt vergleichen.
- `figsize=(8, 9)` legt die Grösse der Abbildung in Zoll fest (Breite, Höhe).

Weil wir dreimal fast dasselbe zeichnen, schreiben wir dafür eine Funktion `plot_frequency`. Sie erhält das Diagramm `ax`, in das gezeichnet werden soll, die Häufigkeiten und einen Titel. Bei einem einzelnen Diagramm heissen die Methoden für die Beschriftung `set_title`, `set_xlabel` und `set_ylabel` statt `title`, `xlabel` und `ylabel`.

Zusätzlich zeichnet `ax.axhline` eine waagrechte gestrichelte Linie auf der Höhe $100/26 \approx 3.85$ %. So häufig wäre jeder Buchstabe, wenn alle Buchstaben gleich oft vorkämen. Je näher die Balken an dieser Linie liegen, desto flacher ist die Verteilung.

```
def plot_frequency(ax, frequency: dict[str, float], title: str) -> None:
    letters = list(frequency.keys())
    values = list(frequency.values())

    ax.bar(letters, values)
    ax.axhline(100 / 26, color="gray", linestyle="--",
    ↪ label="Gleichverteilung")
    ax.set_title(title)
    ax.set_ylabel("Häufigkeit in %")
    ax.legend()
```

```
fig, axes = plt.subplots(3, 1, sharey=True, figsize=(8, 9))

plot_frequency(axes[0], plain_frequency, "Klartext")
plot_frequency(axes[1], caesar_frequency, "Caesar-Chiffre (Schlüssel
↪ 3)")
plot_frequency(axes[2], vigenere_frequency, "Vigenère-Chiffre (Schlüssel
↪ BUELRAIN)")

axes[2].set_xlabel("Buchstabe")
fig.tight_layout()
plt.show()
```

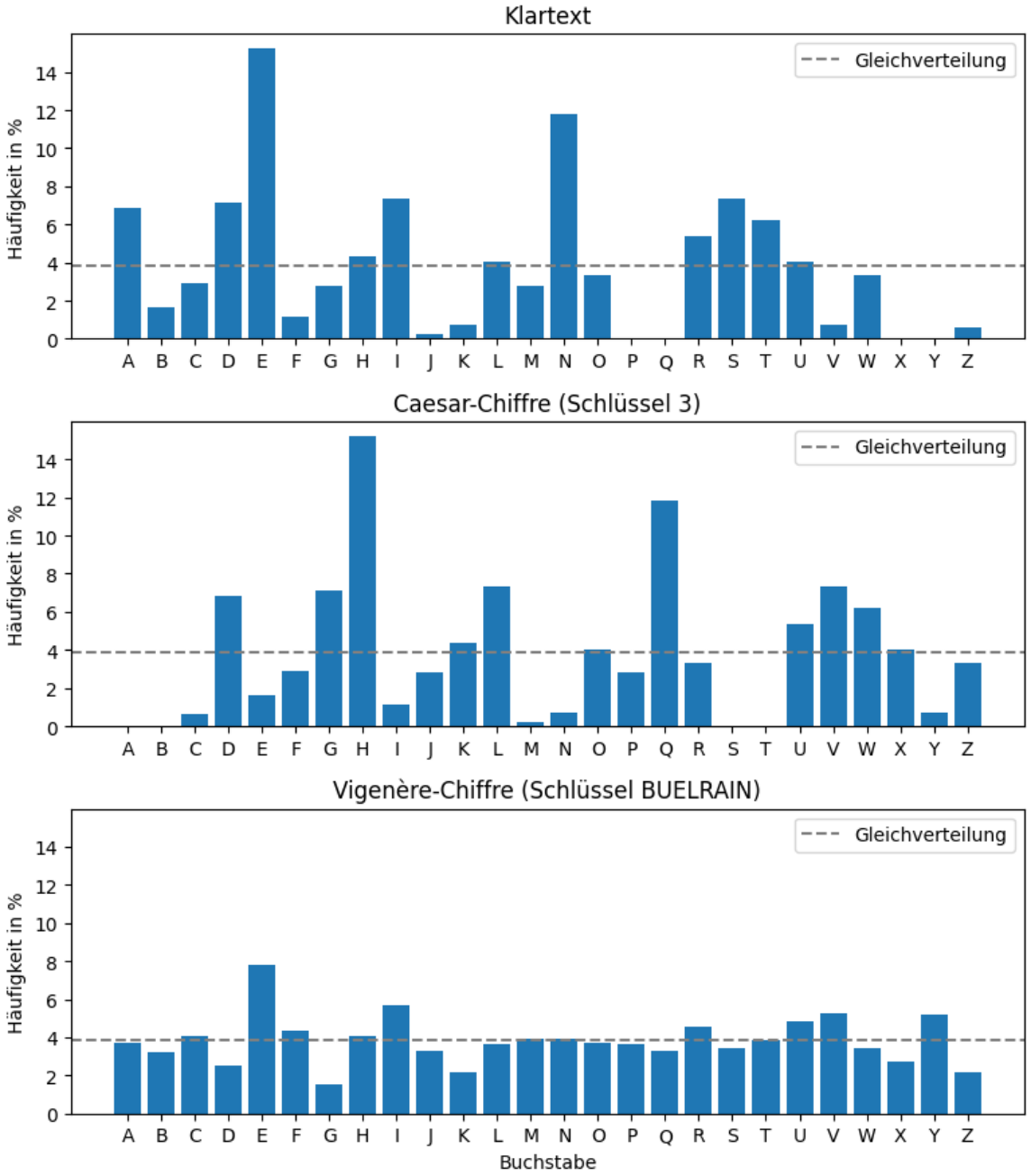


Abbildung 29.2: Buchstabenhäufigkeiten im Klartext, in der Caesar-Chiffre und in der Vigenère-Chiffre.

`fig.tight_layout()` ordnet die drei Diagramme so an, dass sich Titel und Beschriftungen nicht überlappen.

29.7.1.5 Auswertung

Abbildung 29.2 zeigt das Ergebnis des Experiments:

- **Caesar-Chiffre:** Das Histogramm sieht genau gleich aus wie beim Klartext, es ist nur um drei Buchstaben nach rechts verschoben. Der höchste Balken steht nun beim `H` statt beim `E`, und auch die Lücken wandern mit: Statt `P`, `Q`, `X` und `Y` fehlen nun `S`, `T`, `A` und `B`. Genau das nutzt die Häufigkeitsanalyse aus: Wer den höchsten Balken findet, kennt den Schlüssel.
- **Vigenère-Chiffre:** Die Verteilung ist deutlich flacher. Es gibt keinen Balken mehr, der so klar heraussticht wie das `E` im Klartext, und die meisten Balken liegen nahe bei der gestrichelten Linie. Auch `P`, `Q`, `X` und `Y`, die im Klartext gar nicht vorkommen, tauchen nun auf. Es gibt keine Lücken mehr.

Dass die Verteilung flacher wird, lässt sich auch in Zahlen ausdrücken. Wir suchen in jedem Text den häufigsten Buchstaben. Dafür verwenden wir die Funktion `max`. Wird ihr ein Dictionary übergeben, vergleicht sie die Schlüssel, also die Buchstaben. `max(frequency)` würde deshalb einfach den Buchstaben liefern, der im Alphabet zuletzt kommt, das `Z`. Mit dem Argument `key=frequency.get` sagen wir `max`, wonach verglichen werden soll: Für jeden Buchstaben wird `frequency.get(buchstabe)` aufgerufen, also seine Häufigkeit nachgeschlagen. Zurückgegeben wird der Buchstabe mit der grössten Häufigkeit.

```
for name, frequency in [("Klartext", plain_frequency),
                        ("Caesar", caesar_frequency),
                        ("Vigenère", vigenere_frequency)]:
    most_common = max(frequency, key=frequency.get)
    print(f"{name:10} häufigster Buchstabe: {most_common}
     ↪  ({frequency[most_common]:.1f} %)")
```

```
Klartext   häufigster Buchstabe: E (15.2 %)
Caesar     häufigster Buchstabe: H (15.2 %)
Vigenère   häufigster Buchstabe: E (7.8 %)
```

Im f-String steuern die Angaben nach dem Doppelpunkt die Formatierung:

- `{name:10}` reserviert für den Namen 10 Zeichen. Kürzere Namen werden mit Leerzeichen aufgefüllt, so stehen die Ausgaben schön untereinander.
- `{frequency[most_common]:.1f}` gibt die Zahl als Dezimalzahl (`f`) mit einer Stelle nach dem Komma (`.1`) aus.

Bei der Caesar-Chiffre ist der höchste Balken genau gleich hoch wie im Klartext. Bei der Vigenère-Chiffre ist er nur noch etwa halb so hoch.

Auffällig ist, dass auch bei der Vigenère-Chiffre das `E` der häufigste Buchstabe ist. Ist die Häufigkeitsanalyse also doch erfolgreich? Prüfen wir das nach. Zuerst schauen wir, zu welchen Buchstaben das `E` des Klartextes verschlüsselt wurde. Dazu gehen wir mit `enumerate` durch

den Klartext und merken uns bei jedem `E` den Buchstaben, der an derselben Position im Geheimtext steht. Mit `not in` prüfen wir, ob der Buchstabe schon in der Liste steht, damit jeder Buchstabe nur einmal vorkommt. `sorted` gibt die Liste am Schluss alphabetisch sortiert aus.

```
encrypted_e = []

for i, char in enumerate(plain):
    if char == "E" and vigenere_text[i] not in encrypted_e:
        encrypted_e.append(vigenere_text[i])

print(sorted(encrypted_e))
```

```
['E', 'F', 'I', 'M', 'P', 'R', 'V', 'Y']
```

Das `E` des Klartextes ist zu acht verschiedenen Buchstaben geworden, einem für jeden Buchstaben des Schlüssels. Das ist so, weil alle acht Buchstaben von `BUELRAIN` verschieden sind. Nur dort, wo im Schlüssel das `A` steht, ist es ein `E` geblieben.

Umgekehrt können wir zählen, aus welchen Klartextbuchstaben die `E` im Geheimtext entstanden sind. Dafür verwenden wir wieder ein Dictionary. Die Methode `origin_of_e.get(letter, 0)` liefert die bisherige Anzahl für `letter`. Kommt der Buchstabe zum ersten Mal vor, steht er noch nicht im Dictionary. Dann liefert `get` den Ersatzwert `0`, statt wie `origin_of_e[letter]` einen Fehler zu melden.

```
origin_of_e = {}

for i, char in enumerate(vigenere_text):
    if char == "E":
        letter = plain[i]
        origin_of_e[letter] = origin_of_e.get(letter, 0) + 1

print(origin_of_e)
```

```
{'A': 13, 'W': 3, 'T': 9, 'D': 10, 'E': 22, 'R': 5, 'N': 11, 'K': 2}
```

Von den 75 `E` im Geheimtext waren nur 22 schon im Klartext ein `E`, also weniger als ein Drittel. Die übrigen stammen von `A`, `N`, `D`, `T` und weiteren Buchstaben. Bei unserem Schlüssel ist jeder Buchstabe im Geheimtext eine Mischung aus acht verschiedenen Klartextbuchstaben. Beim `E` im Geheimtext sind das zufällig `E`, `A`, `N`, `D`, `T` und `R`, also besonders häufige Buchstaben, sowie `W` und `K`. Deshalb ist sein Balken etwas höher als die anderen. Dass das `E` selbst darunter ist, liegt am `A` im Schlüssel, einer Schwäche, die wir weiter unten noch genauer betrachten.

Wer annimmt, dass jedes `E` im Geheimtext auch im Klartext ein `E` ist, liegt also in mehr als zwei von drei Fällen falsch. Die Häufigkeitsanalyse, wie wir sie bei der Caesar-Chiffre verwendet haben, funktioniert hier nicht mehr.

29.7.2 Ein grösserer Schlüsselraum

Auch Brute Force wird schwieriger. Ein Schlüsselwort mit acht Buchstaben erlaubt

$$26^8 = 208\,827\,064\,576$$

also rund 209 Milliarden verschiedene Schlüssel. Für die Menschen des 16. Jahrhunderts war es völlig undenkbar, so viele Schlüssel von Hand auszuprobieren.

Für einen heutigen Computer sieht das anders aus. Nehmen wir an, er schaffe hundert Millionen Schlüssel pro Sekunde. Die folgende Zelle berechnet, wie lange er für alle Schlüssel einer bestimmten Länge braucht. Damit die Zeiten lesbar bleiben, wählt die Funktion `format_duration` je nach Grösse eine passende Einheit. `26 ** length` ist die Potenz 26^{length}. Die Formatangabe `:,` setzt Tausendertrennzeichen, die wir mit `.replace(",", "'")` durch das in der Schweiz übliche Hochkomma ersetzen.

```
def format_duration(seconds: float) -> str:
    if seconds < 1:
        return "unter 1 Sekunde"
    elif seconds < 3600:
        return f"{seconds / 60:.1f} Minuten"
    elif seconds < 24 * 3600:
        return f"{seconds / 3600:.1f} Stunden"
    elif seconds < 365 * 24 * 3600:
        return f"{seconds / (24 * 3600):.1f} Tage"
    else:
        return f"{seconds / (365 * 24 * 3600):.1f} Jahre"


keys_per_second = 100_000_000

for length in range(1, 13):
    keys = 26 ** length
    seconds = keys / keys_per_second
    keys_text = f"{keys:,}".replace(",", "'")
    print(f"{length:2} Buchstaben: {keys_text:>26} Schlüssel,
     ↪ {format_duration(seconds)}")
```

```
 1 Buchstaben:                         26 Schlüssel, unter 1 Sekunde
 2 Buchstaben:                        676 Schlüssel, unter 1 Sekunde
```

```
 3 Buchstaben:                    17'576 Schlüssel, unter 1 Sekunde
 4 Buchstaben:                   456'976 Schlüssel, unter 1 Sekunde
 5 Buchstaben:                11'881'376 Schlüssel, unter 1 Sekunde
 6 Buchstaben:               308'915'776 Schlüssel, 0.1 Minuten
 7 Buchstaben:             8'031'810'176 Schlüssel, 1.3 Minuten
 8 Buchstaben:           208'827'064'576 Schlüssel, 34.8 Minuten
 9 Buchstaben:         5'429'503'678'976 Schlüssel, 15.1 Stunden
10 Buchstaben:       141'167'095'653'376 Schlüssel, 16.3 Tage
11 Buchstaben:     3'670'344'486'987'776 Schlüssel, 1.2 Jahre
12 Buchstaben:    95'428'956'661'682'176 Schlüssel, 30.3 Jahre
```

Für unseren Schlüssel mit acht Buchstaben braucht der Computer also nur gut eine halbe Stunde. Dabei haben wir sogar angenommen, dass der Angreifer die Länge des Schlüssels kennt. Kennt er sie nicht, muss er alle Längen ausprobieren. Weil jeder zusätzliche Buchstabe die Anzahl Schlüssel versechsundzwanzigfacht, wächst der Aufwand sehr schnell: Bei zwölf Buchstaben wäre der Computer bereits rund 30 Jahre beschäftigt.

Brute Force ist aber gar nicht der klügste Angriff:

- **Wörterbuchangriff:** `BUELRAIN` ist kein zufälliger Buchstabensalat, sondern ein Name. Viele Menschen wählen Wörter oder Namen als Schlüssel, weil sie sich diese gut merken können. Ein Angreifer probiert deshalb zuerst alle Einträge einer Wortliste aus. Selbst eine Liste mit einigen Millionen Wörtern und Namen ist bei hundert Millionen Versuchen pro Sekunde in Sekundenbruchteilen abgearbeitet.
- **Kasiski-Test:** Wie wir im nächsten Abschnitt sehen, lässt sich die Länge des Schlüssels aus dem Geheimtext herauslesen. Danach muss der Angreifer nur noch acht einzelne Caesar-Chiffren knacken, also höchstens $8 \cdot 25 = 200$ Schlüssel ausprobieren statt 209 Milliarden.

29.8 Die Schwächen der Vigenère-Chiffre

Ganz so sicher, wie man 300 Jahre lang glaubte, ist die Chiffre aber nicht.

Der Buchstabe `A` im Schlüssel. Da `A` für eine Verschiebung um 0 steht, bleibt jeder Buchstabe an dieser Position unverändert. In unserem Beispiel steht das `A` an der sechsten Stelle des Schlüssels, und tatsächlich bleiben das `F` (Nr. 6 in der Tabelle) und das `E` (Nr. 14) im Geheimtext stehen. Ein `A` im Schlüssel ist also ein kleines Leck.

Die Wiederholung des Schlüssels. Die eigentliche Schwäche liegt darin, dass sich das Schlüsselwort wiederholt. Alle Buchstaben mit den Nummern 1, 9, 17, … werden mit derselben Verschiebung (`B`) verschlüsselt, alle mit den Nummern 2, 10, 18, … mit derselben Verschiebung (`U`) und so weiter. In Python, wo ab 0 gezählt wird, sind das die Positionen 0, 8, 16, … und 1, 9, 17, … Kennt ein Angreifer die Länge des Schlüssels, kann er den Geheimtext in acht Gruppen aufteilen. Jede Gruppe ist eine gewöhnliche Caesar-Chiffre und lässt sich mit der Häufigkeitsanalyse aus der letzten Lektion knacken. Damit das klappt, muss der Geheimtext allerdings lang genug sein, denn jede Gruppe enthält nur jeden achten Buchstaben.

Bei unserem Bibeltext mit 965 Buchstaben sind das immerhin rund 120 Buchstaben pro Gruppe.

29.8.1 Die Schlüssellänge finden: der Kasiski-Test

Bleibt die Frage, wie man die Länge des Schlüssels herausfindet. Hier setzt das Verfahren von Kasiski an: Kommt in einem langen Text dieselbe Buchstabenfolge (zum Beispiel `DER` oder `UND`) mehrmals vor und trifft sie zufällig auf dieselbe Stelle im Schlüssel, entsteht im Geheimtext zweimal dieselbe Buchstabenfolge. Der Abstand zwischen solchen Wiederholungen ist dann ein Vielfaches der Schlüssellänge.

Probieren wir das am verschlüsselten Bibeltext aus. Wir suchen nach *Trigrammen*, also Folgen von drei Buchstaben, die im Geheimtext mehrmals vorkommen. Für jedes Trigramm merken wir uns in einem Dictionary die Liste der Positionen, an denen es beginnt. Die Schleife läuft nur bis `len(vigenere_text) - 2`, weil ab dort keine drei Buchstaben mehr übrig sind.

```
positions = {}

for i in range(len(vigenere_text) - 2):
    trigram = vigenere_text[i:i + 3]
    if trigram not in positions:
        positions[trigram] = []
    positions[trigram].append(i)
```

Nun berechnen wir für jedes Trigramm, das mehrmals vorkommt, die Abstände zwischen zwei aufeinanderfolgenden Vorkommen. Alle Abstände sammeln wir in der Liste `distances`. Die Trigramme, die mindestens dreimal vorkommen, geben wir zur Kontrolle aus.

```
distances = []

for trigram, starts in positions.items():
    gaps = []
    for j in range(len(starts) - 1):
        gaps.append(starts[j + 1] - starts[j])
    distances += gaps

    if len(starts) >= 3:
        print(f"{trigram}: Positionen {starts}, Abstände {gaps}")

print(len(distances), "Abstände gefunden")
```

```
YYU: Positionen [18, 122, 218, 954], Abstände [104, 96, 736]
PJE: Positionen [51, 67, 339], Abstände [16, 272]
```

```
IKU: Positionen [141, 381, 477], Abstände [240, 96]
KUU: Positionen [142, 382, 478, 502], Abstände [240, 96, 24]
LRO: Positionen [166, 486, 654], Abstände [320, 168]
OCW: Positionen [168, 248, 264], Abstände [80, 16]
QEC: Positionen [177, 417, 529, 543, 623], Abstände [240, 112, 14, 80]
UAA: Positionen [188, 420, 468], Abstände [232, 48]
TTH: Positionen [203, 411, 739], Abstände [208, 328]
THB: Positionen [204, 412, 684, 740], Abstände [208, 272, 56]
LQP: Positionen [209, 893, 909], Abstände [684, 16]
ZNL: Positionen [236, 532, 828], Abstände [296, 296]
NLR: Positionen [237, 533, 653], Abstände [296, 120]
EEV: Positionen [290, 586, 932], Abstände [296, 346]
OXI: Positionen [376, 768, 808], Abstände [392, 40]
ECI: Positionen [520, 544, 624, 712], Abstände [24, 80, 88]
UVQ: Positionen [541, 621, 853], Abstände [80, 232]
CIL: Positionen [713, 891, 907], Abstände [178, 16]
ITK: Positionen [898, 914, 962], Abstände [16, 48]
154 Abstände gefunden
```

Die meisten Abstände sind Vielfache von 8: 16, 24, 48, 96, 240 und so weiter. Ein paar wenige passen nicht ins Bild, etwa der Abstand 14 beim Trigramm `QEC`. Hier sind zufällig zwei verschiedene Stellen des Klartextes (`WAR` und `DDI`) zum gleichen Geheimtext geworden. Solche Zufallstreffer sind selten, sie stören das Gesamtbild aber kaum.

Um die Schlüssellänge zu bestimmen, zählen wir für jede mögliche Länge von 2 bis 12, wie viele Abstände ohne Rest durch diese Länge teilbar sind. Dafür verwenden wir wieder den Modulo-Operator: `distance % length == 0` bedeutet «ohne Rest teilbar».

```
for length in range(2, 13):
    count = 0
    for distance in distances:
        if distance % length == 0:
            count += 1
    print(f"Länge {length:2}: {count:3} von {len(distances)} Abständen
    ↪ teilbar")
```

```
Länge  2: 151 von 154 Abständen teilbar
Länge  3:  48 von 154 Abständen teilbar
Länge  4: 143 von 154 Abständen teilbar
Länge  5:  28 von 154 Abständen teilbar
Länge  6:  47 von 154 Abständen teilbar
Länge  7:  12 von 154 Abständen teilbar
Länge  8: 142 von 154 Abständen teilbar
Länge  9:   8 von 154 Abständen teilbar
Länge 10:  28 von 154 Abständen teilbar
```

```
Länge 11:   2 von 154 Abständen teilbar
Länge 12:  43 von 154 Abständen teilbar
```

Drei Längen stechen heraus: 2, 4 und 8. Das ist kein Widerspruch. Jedes Vielfache von 8 ist automatisch auch ein Vielfaches von 4 und von 2. Kürzere Längen, die Teiler von 8 sind, schneiden deshalb mindestens so gut ab. Gesucht ist die **grösste** Länge, durch die fast alle Abstände teilbar sind, und das ist 8. Damit haben wir die Länge von `BUELRAIN` allein aus dem Geheimtext bestimmt, ohne einen einzigen Schlüssel auszuprobieren.

Der Rest ist Handwerk: Den Geheimtext in acht Gruppen zerlegen und jede Gruppe mit der Häufigkeitsanalyse knacken. Das ist eine der Programmieraufgaben am Ende dieses Kapitels.

Daraus folgt auch, wie man die Vigenère-Chiffre sicherer machen kann: Je länger der Schlüssel im Vergleich zum Text, desto weniger Wiederholungen gibt es und desto kürzer werden die Gruppen. Ist der Schlüssel zufällig, gleich lang wie der Text und wird er nur ein einziges Mal verwendet, ist die Verschlüsselung sogar beweisbar unknackbar. Dieses Verfahren heisst *One-Time-Pad*.

Aufgaben

Von Hand

1. Verschlüsseln Sie Ihren Vornamen mit dem Schlüssel `BUELRAIN`. Benutzen Sie dazu das Vigenère-Quadrat. Umlaute schreiben Sie wie die Funktion `clean` als `AE`, `OE` und `UE`.
2. Entschlüsseln Sie die folgende Nachricht, die ebenfalls mit dem Schlüssel `BUELRAIN` verschlüsselt wurde:

   WCKPE EZRJM XYZCP GVHOY RCSOB L
3. Verschlüsseln Sie den Text `AAAAAAAAAAAAAAAA` (16-mal `A`) mit dem Schlüssel `BUELRAIN`. Was fällt Ihnen auf? Was bedeutet das für die Sicherheit, wenn ein Angreifer einen Teil des Klartextes kennt?
4. Weshalb ist `AAAA` ein schlechter Schlüssel? Welcher Chiffre entspricht ein Schlüssel wie `DDDD`? Verschlüsseln Sie `ANGRIFF` mit dem Schlüssel `BOOT`. Was geschieht mit dem Doppelbuchstaben `FF`, und weshalb?

Programmieren

5. Schreiben Sie eine Funktion `vigenere_decrypt(text, key)`, die einen mit `vigenere` verschlüsselten Text wieder entschlüsselt. Testen Sie sie mit der Nachricht aus Aufgabe 2 und mit `vigenere_text`.
6. Schreiben Sie eine Funktion `split_groups(text, length)`, die einen Text in `length` Gruppen zerlegt und als Liste zurückgibt. Die erste Gruppe enthält die Buchstaben an den Positionen 0, `length`, 2 · `length`, …, die zweite die Buchstaben an den Positionen 1, `length` + 1, … und so weiter. *Tipp:* `text[1::8]` liefert jeden achten Buchstaben ab Position 1.

7. *Für Fortgeschrittene:* Knacken Sie `vigenere_text`, ohne den Schlüssel zu verwenden. Zerlegen Sie den Text mit `split_groups` in acht Gruppen und bestimmen Sie für jede Gruppe mit der Funktion `crack` aus der Lektion «Caesar-Chiffre knacken» die Verschiebung. Aus den acht Verschiebungen ergibt sich der Schlüssel. Erhalten Sie `BUELRAIN`?

Literaturverzeichnis

Durandi, Werner, Baoswan Dzung Wong, Markus Kriener, u. a. *Formeln, Tabellen, Begriffe: Mathematik - Physik - Chemie*. 8., aktualisierte Auflage. Herausgegeben von Verein Schweizerischer Mathematik- und Physiklehrkräfte und Werner Durandi. Zürich: Orell Füssli Verlag, 2022.